
Test Document

for

TrackMate

Version 1.0

Prepared by

Group 4:

Group Name: Earth's Mightiest Dozens -2

Krishna Agrawal	240563	krishnaa24@iitk.ac.in
Chandradip Karmakar	240296	chandradip24@iitk.ac.in
Nideesh H	240690	nideesh24@iitk.ac.in
Abubakar Siddique	240043	abubakars24@iitk.ac.in
Ankit Kumar Jha	240137	ankitkj24@iitk.ac.in
Aman Gupta	240100	amanu24@iitk.ac.in
Rushali Vijayakumar Myageri	240894	rushali24@iitk.ac.in
Ayush Mangal	240242	ayushn24@iitk.ac.in
Sahil Agarwal	240902	sahilag24@iitk.ac.in
Aditi	240048	aditij24@iitk.ac.in

Course: CS253

Mentor TA: George T.L.

Date: 4th April, 2026



CONTENTS.....	II
REVISIONS.....	II
1 INTRODUCTION.....	1
2 UNIT TESTING.....	2
3 INTEGRATION TESTING.....	3
4 SYSTEM TESTING.....	4
5 CONCLUSION.....	5
APPENDIX A - GROUP LOG.....	6

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	All team members	Revised the entire document for clarity, consistency, and structure. Improved language flow, corrected grammatical issues, and ensured alignment with the intended purpose and audience. All sections have been reviewed and updated where necessary	03/04/26

1 Introduction

What was the test strategy?

The primary strategy employed for this phase was **Manual Testing**. Because the application features highly interactive UI components (such as dragging water sliders, interacting with dynamic pie charts, and real-time chat), manual testing allowed the team to dynamically assess the usability, visual feedback, and functional flows from a genuine human-centric perspective. While this approach required significant effort to simulate multi-user interactions, it was essential for verifying the nuanced "feel" and real-time responsiveness of the app.

When was the testing conducted?

The comprehensive QA cycle was conducted **after implementation was completed**. Once the development team finalized the core feature sets and merged the Trainee and Trainer modules, the testing phase commenced. This post-implementation approach was necessary to effectively perform integration testing, allowing us to evaluate the fully connected system and ensure that databases, third-party APIs, and front-end UI components were communicating flawlessly as a finalized package.

Who were the testers?

The testing effort was a collaborative execution utilizing **both developers and independent testers**. Developers handled the initial baseline testing, checking local database queries, API endpoint stability, and resolving immediate runtime crashes. Once the build was stable, independent QA testers took over to execute the rigorous functional, end-to-end, and cross-role acceptance test cases. This separation ensured that the user journeys were evaluated from an unbiased, real-world user's perspective, catching edge cases that the developers might not have anticipated.

What coverage criteria were used?

Given the manual nature of the testing, we focused heavily on **Functional and Integration Coverage** mapped directly to the core user journeys. Specific criteria included:

- **State & Navigation Coverage:** Verifying role-based access controls (ensuring Trainees couldn't access Trainer views), tab switching, and deep-link routing from push notifications.
- **Data Flow & Synchronization Coverage:** Ensuring inputs (like logging 100g of Chapati) are accurately calculated macros, saved to the database, and instantly propagated to the global Calorie Balance dashboards and Analytics pie charts without requiring page refreshes.

- **Boundary Value & Error Handling Coverage:** Testing absolute limits and edge cases, such as inputting negative food quantities, exceeding daily macro limits (testing UI overflow), inputting special characters to check for SQL injection vulnerabilities, and rapid button tapping to test API debouncing.
- **Cross-Role Integration Coverage:** Validating the multi-step handshake when a Trainee sends a coaching request, the Trainer accepts it, and the database updates both users' rosters and social graphs simultaneously.

Have you used any tool for testing?

Because the core strategy relied on manual execution, we did not utilize automated end-to-end UI frameworks (such as Appium or Selenium) during this specific cycle. However, the team relied on several foundational environments and debugging tools to facilitate the manual process:

- **Physical Devices & OS Simulators:** Testing was executed across specific hardware environments (e.g., Android 14 / Pixel 8) to verify screen rendering, keyboard avoidance, and hardware pedometer sensor integration.
- **Database Management Clients:** Developers utilized backend viewing tools (such as Render) to manually inspect database tables and verify that atomic transactions were successfully committing under the hood.
- **Logcat / Debug Consoles:** Essential for tracking down the exact cause of fatal errors during edge-case testing, such as identifying the specific `NullPointerException` that occurred when searching for special characters in the food database.

2 Unit Testing

1. Unit Name:- Authentication

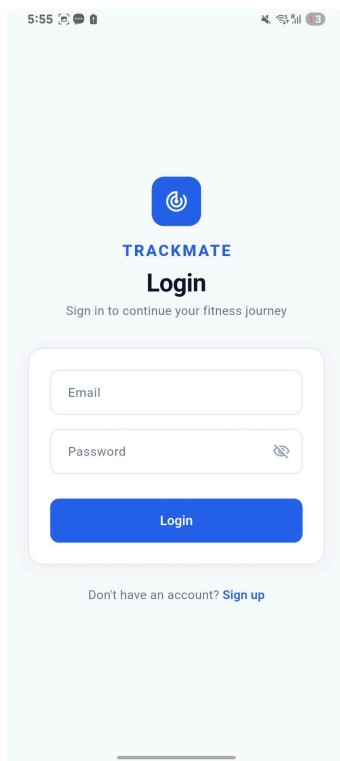
Unit Details: App Launch, Login, and Sign-Up

Test Owner: Abubakar, Ankit and Sahil

Test Date: 30th March, 2026

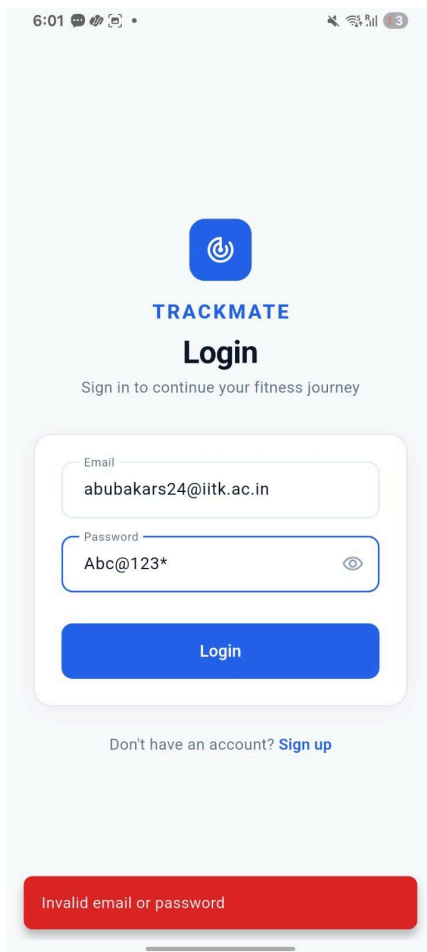
Test ID: 1.1 | Verify Successful User Login

- **Pre-conditions:** User is on the Login Page with a registered account details.
- **Steps:**
 - Enter a valid email address and password.
 - Tap the "Login" button.
- **Expected Result:** The app authenticates the credentials and navigates the user directly to the main Dashboard page.
- **Actual Result:** As expected. The app successfully navigated to the main Dashboard page.
- **Status:** PASS



Test ID: TC-ALS-002 | Verify Invalid Password Error Handling

- **Pre-conditions:** User is on the Login Page.
- **Steps:**
 - Enter a valid email address, but an incorrect password.
 - Tap the "Login" button.
- **Expected Result:** The login attempt fails, the user remains on the Login screen, and an error message is clearly displayed.
- **Actual Result:** As expected. A visible error message appeared saying "Invalid email or password." The user remained on the Login screen.
- **Status:** PASS

**Test ID: TC-ALS-003 | Verify Account Registration Flow**

- **Pre-conditions:** User is on the Register Page.
- **Steps:**
 - Fill in the full name, a new email address, and a valid password.
 - Tap the "Sign Up" button.

- **Expected Result:** The app accepts the inputs, creates the initial account state, and navigates to the "Check Email" OTP verification screen.
- **Actual Result:** As expected. The app navigated to the "Check Email" OTP screen.
- **Status:** PASS

Test ID: TC-ALS-004 | Verify Registration with Existing Email

- **Pre-conditions:** User is on the Register Page.
- **Steps:**
 - Enter details, including an email address that is already registered in the system.
 - Tap the "Sign Up" button.
- **Expected Result:** The system rejects the registration attempt and displays a warning to the user.
- **Actual Result:** As expected. A pop-up or snackbar appeared, warning "This email is already in use."
- **Status:** PASS

Test ID: TC-ALS-005 | Verify Successful OTP Verification

- **Pre-conditions:** User is on the Check Email Page and has received the verification code via email.
- **Steps:**
 - Enter the correct 4- or 6-digit code into the input fields.
- **Expected Result:** The system validates the OTP and automatically navigates the user forward to the Onboarding/Profile Setup screen.
- **Actual Result:** As expected. The app accepted the code and navigated to the Onboarding/Profile Setup screen.
- **Status:** PASS
- **Additional Comments:** *[Insert Screenshot: The Onboarding welcome screen]*

Test ID: TC-ALS-006 | Verify OTP Verification Failure Handling

- **Pre-conditions:** User is on the Check Email Page.
- **Steps:**
 - Enter a random, incorrect OTP code.

- **Expected Result:** The system rejects the OTP, visually indicates the error on the input fields, and shows a failure message.
- **Actual Result:** As expected. The input boxes flashed red and an "Incorrect code, please try again" message appeared.
- **Status:** PASS
- **Additional Comments:** *[Bug Watch: If the user spams the "Verify" button with the wrong code, the app shouldn't freeze.]*

Test ID: TC-ALS-007 | Verify Resend OTP Functionality

- **Pre-conditions:** User is on the Check Email Page.
- **Steps:**
 - Tap the "Resend Code" interactive text.
- **Expected Result:** A new code is generated and sent to the email, and the UI displays a confirmation message.
- **Actual Result:** As expected. A small confirmation appeared saying "Code resent to your email."
- **Status:** PASS

Test ID: TC-ALS-008 | Verify Role Selection Highlight State

- **Pre-conditions:** User is actively on the Register / Onboarding Page (Role Selector view). Default selection is on "User".
- **Steps:**
 - Tap on the "Trainer" option.
- **Expected Result:** The UI updates dynamically to show "Trainer" as the active selection, changing its color and border, while removing the active state from "User".
- **Actual Result:** As expected. The icon and text changed to the primary brand color, and a border highlighted the selection. The "User" option became un-highlighted.
- **Status:** PASS
- **Additional Comments:** *[Insert Screenshot: The Role Selector widget showing 'Trainer' actively selected]*

Test ID: TC-ALS-009 | Verify User Logout Functionality

- **Pre-conditions:** User is authenticated and currently on the Settings / Profile Page.
- **Steps:**
 - Tap the "Logout" button.

- **Expected Result:** The user's active session is terminated, local data is cleared from the dashboard, and the user is routed back to the Login screen.
- **Actual Result:** As expected. The app instantly cleared the dashboard and returned the user to the starting Login screen.
- **Status:** PASS

Test ID: TC-ALS-010 | Verify Auto-Login (Session Persistence)

- **Pre-conditions:** User has previously logged in and currently holds a valid, unexpired session token. The app is fully closed (swiped away).
- **Steps:**
 - Launch the app from the device home screen.
- **Expected Result:** The app detects the active session during launch, completely bypasses the Login screen, and loads the Dashboard.
- **Actual Result:** As expected. The app bypassed the login screen and went straight to the Dashboard without asking for a password.
- **Status:** PASS

2. Unit Name:- Food Logging

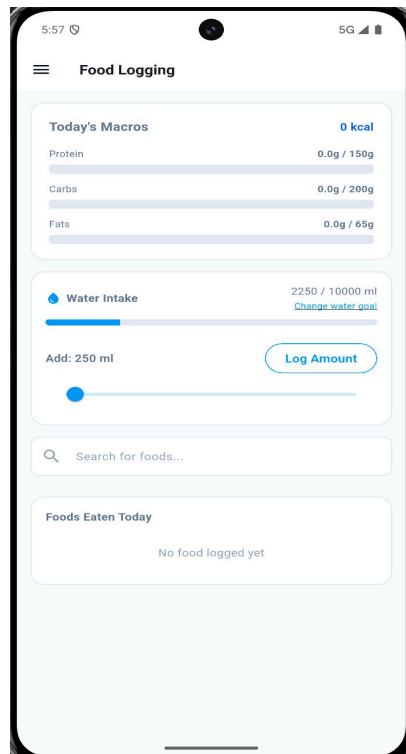
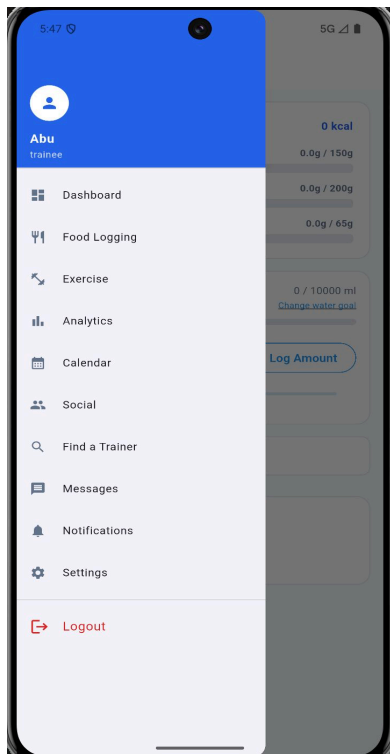
Unit Details: Logging Food, Changing water goals, Logging water

Test Owner: Krishna, Rushali and Nideesh

Test Date: 30th March 2026

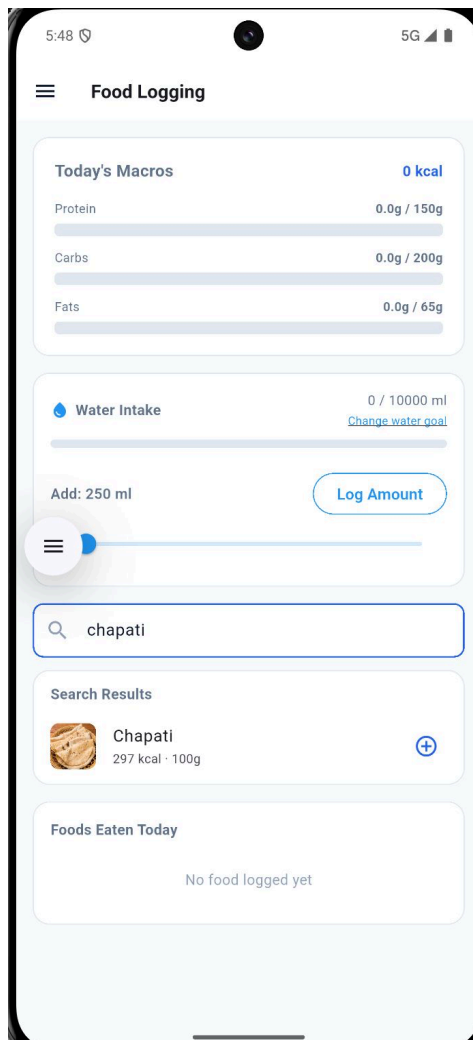
Test ID: 2.1 | Verify Initial Empty State

- **Pre-conditions:** User is authenticated and navigates to the Food Logging page for a new calendar day.
- **Steps:**
 - Tap the hamburger menu and select "Food Logging".
 - Observe the "Today's Macros" card and "Foods Eaten Today" list.
- **Expected Result:** "Today's Macros" should display exactly 0 kcal, and Protein, Carbs, and Fats progress bars should be empty (0.0g). The food list should show a "No food logged yet" placeholder.
- **Actual Result:** As expected. All counters were initialized at 0, and the correct placeholder was displayed.
- **Status:** PASS



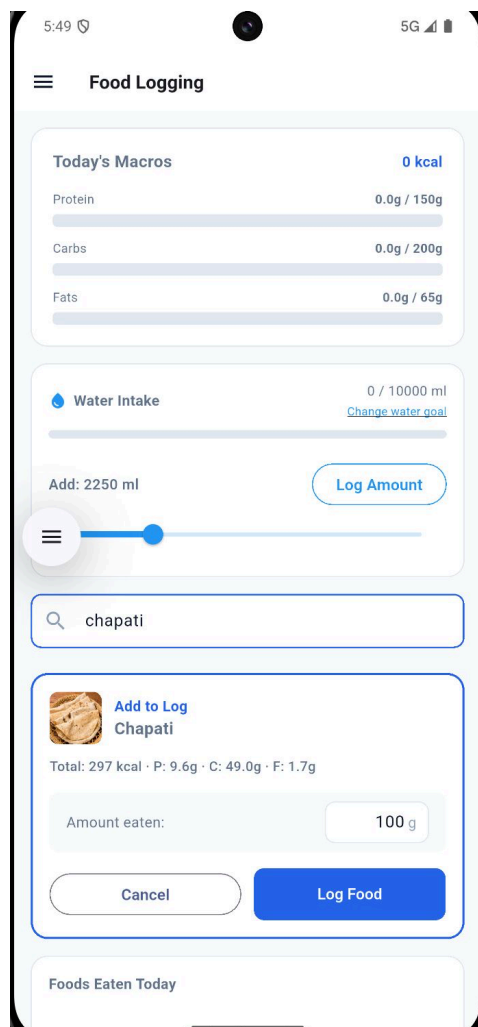
Test ID: 2.2 | Verify Valid Food Search Query

- **Pre-conditions:** User is on the Food Logging page with an active internet connection.
- **Steps:**
 - Tap the "Search for foods..." field.
 - Type chapati.
- **Expected Result:** A search results card appears displaying "Chapati", an image thumbnail, caloric value (e.g., 297 kcal), and base serving weight.
- **Actual Result:** Search results populated correctly, matching all expected UI parameters.
- **Status:** PASS



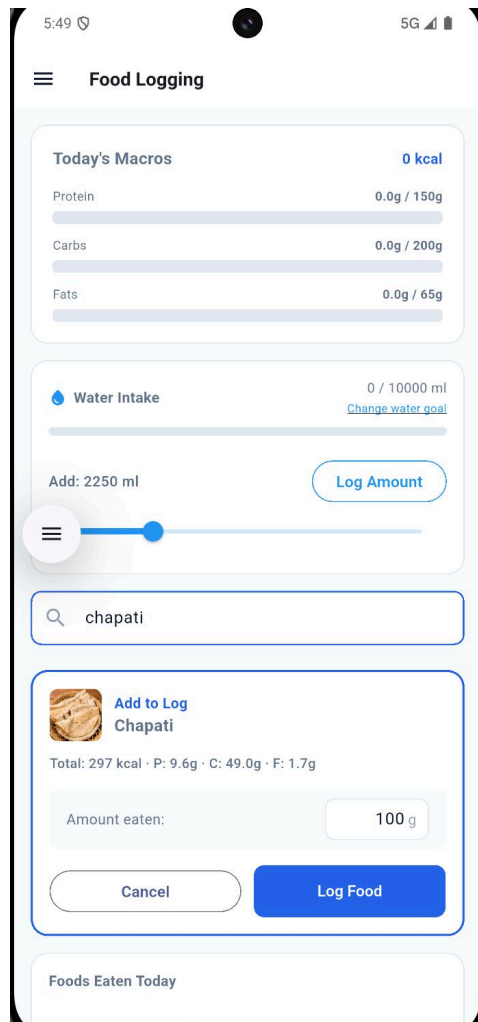
Test ID: 2.3 | Verify Invalid/Special Character Search Handling

- **Pre-conditions:** User is on the Food Logging page.
- **Steps:**
 - Tap the search input field.
 - Type special characters like %%%\$\$\$\$.
- **Expected Result:** The app should gracefully handle the invalid input and display a standard "No results found" placeholder.
- **Actual Result:** Previously, when giving special characters to search food, it did not show anything at all, whereas it should show "not found". This was a bug, however, we have now fixed it and it shows the expected behavior by correctly displaying the "No results found" message.
- **Status:** PASS



Test ID: 2.4 | Verify Water Slider Input & Aggregation

- **Pre-conditions:** User is on the Food Logging page.
- **Steps:**
 - Drag the blue water intake slider to exactly 2250 ml.
 - Tap the "Log Amount" button.
- **Expected Result:** The total water intake metric updates to "2250 / 10000 ml", the progress bar fills proportionately, and the slider resets to a default 250ml state.
- **Actual Result:** The system successfully captured the input and updated the UI elements as expected.
- **Status:** PASS

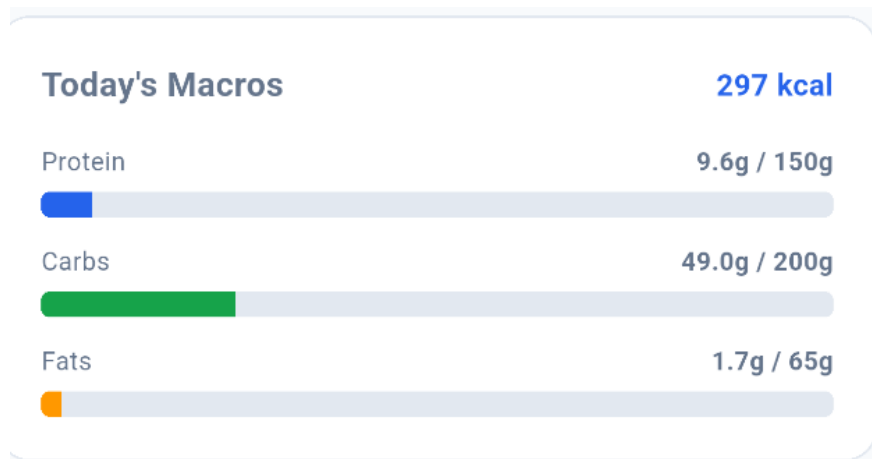


Test ID: 2.5 | Verify API Rate Limiting (Rapid Tapping)

- **Pre-conditions:** Water intake slider is set to 250 ml. Current total is 0 / 10000 ml.
- **Steps:**
 - Rapidly tap the "Log Amount" button 5 times as fast as possible.
- **Expected Result:** The app should only log the intended amount once and prevent accidental duplicate entries from rapid tapping.
- **Actual Result:** The rapid tapping does not work to exploit the system because the UI inherently takes 1 second to update anyways. This natural delay prevents duplicate logs. Only 250ml was recorded.
- **Status:** PASS

Test ID: 2.6 | Verify Food Logging & Macro Recalculation

- **Pre-conditions:** "Today's Macros" shows 0g for all categories.
- **Steps:**
 - Search for "Chapati", tap the "+" icon, leave the amount at 100g.
 - Tap "Log Food".
- **Expected Result:** The item transfers to the "Foods Eaten Today" list. Total calories updates to 297 kcal. Protein (9.6g), Carbs (49.0g), and Fats (1.7g) bars fill proportionally.
- **Actual Result:** Data synced perfectly. Calories and text labels updated correctly, and progress bars filled proportionally.
- **Status:** PASS

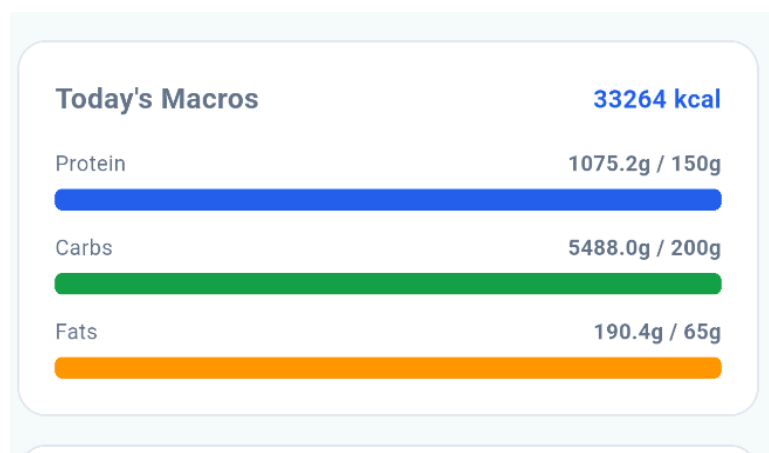


Test ID: 2.7 | Verify Form Validation (Negative and Maximum Values)

- **Pre-conditions:** User has searched for "Chapati" and expanded the "Add to Log" card.
- **Steps:**
 - Tap the "Amount eaten" field.
 - Attempt to enter a negative value (e.g., -50).
 - Attempt to enter an excessively large value (e.g., 105000g).
- **Expected Result:** The system should block negative inputs entirely and cap the maximum allowable input at 100 kg.
- **Actual Result:** As expected. You cannot put a negative value (the keyboard/input validation restricts it), and the field strictly enforces the maximum limit of 100 kg.
- **Status:** PASS

Test ID: 2.8 | Verify UI Behavior when Daily Macros are Exceeded

- **Pre-conditions:** User has already logged significant food for the day.
- **Steps:**
 - Log an additional large quantity of food that pushes a macro past its daily goal (e.g., adding enough to reach 577g / 200g of Carbs).
 - Observe the Carbs progress bar on the "Today's Macros" card.
- **Expected Result:** The UI should handle the overage gracefully without crashing or breaking the card layout.
- **Actual Result:** When putting a large amount of food, it does not crash or overflow the visual container. It simply shows "577g / 200g" and the bar caps out safely, which is the intended behavior.
- **Status:** PASS

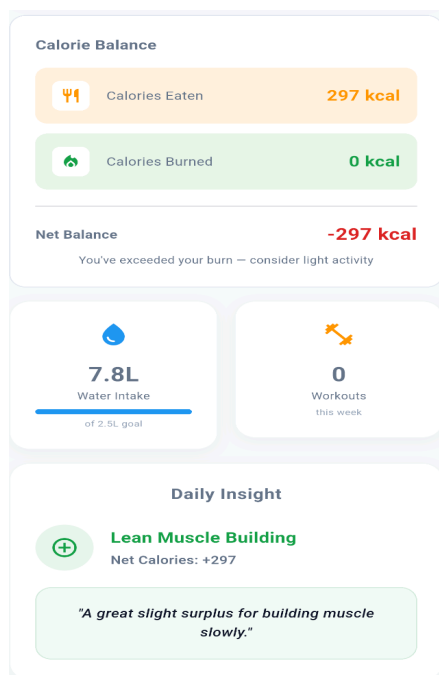


Test ID: 2.9 | Verify Item Deletion & Macro Reversion

- **Pre-conditions:** "Foods Eaten Today" contains one item (Chapati, 297 kcal).
- **Steps:**
 - Tap the red trash can icon next to the "Chapati" item.
- **Expected Result:** The item is removed from the list. The "Today's Macros" card immediately subtracts 297 kcal and resets the progress bars to 0.0g. The empty state ("No food logged yet") returns.
- **Actual Result:** As expected. UI updated seamlessly and math reverted correctly without requiring a page refresh.
- **Status:** PASS

Test ID: 2.10 | Verify Calorie Balance Dashboard Synchronization

- **Pre-conditions:** User has logged 297 kcal of food on the Food Logging page.
- **Steps:**
 - Navigate away from Food Logging to the overall "Calorie Balance" view.
- **Expected Result:** "Calories Eaten" displays 297 kcal, and "Net Balance" calculates correctly (e.g., -297 kcal if burn is 0).
- **Actual Result:** As expected. Logged food accurately synced across the app's global state.
- **Status:** PASS



3. Unit Name:- Exercise & Workouts

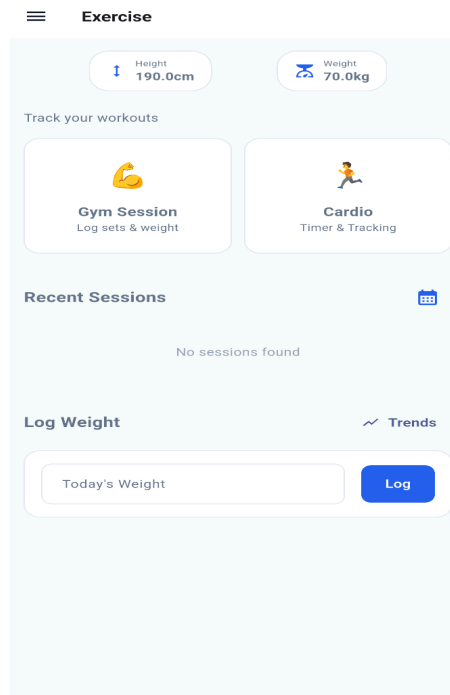
Unit Details: Logging Exercise, changing weight, doing a gym session and doing cardio

Test Owner: Ayush, Aditi and Aman

Test Date: 31st March 2026

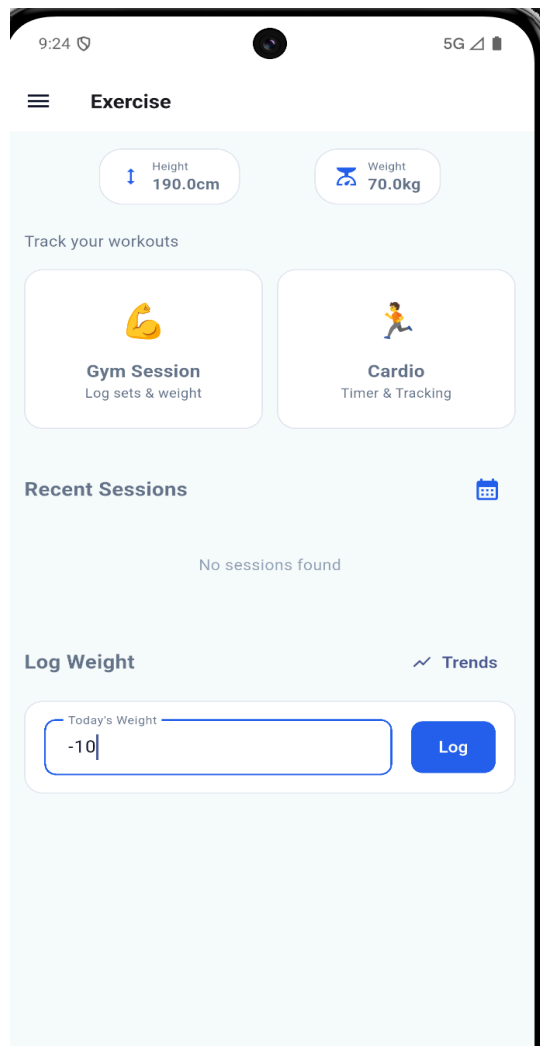
Test ID: 3.1 | Verify Exercise Dashboard Initial State

- **Pre-conditions:** User is authenticated and navigating to the main Exercise page.
- **Steps:**
 - Observe the top metric pills (Height and Weight).
 - Observe the "Recent Sessions" section.
- **Expected Result:** The top metrics should accurately display the height and weight initially taken from the onboarding page (or updated from Settings). If no workouts have been completed, "Recent Sessions" should display "No sessions found".
- **Actual Result:** As expected. Height (190.0cm) and Weight (70.0kg) are correctly displayed, and the empty state for recent sessions is present.
- **Status:** PASS



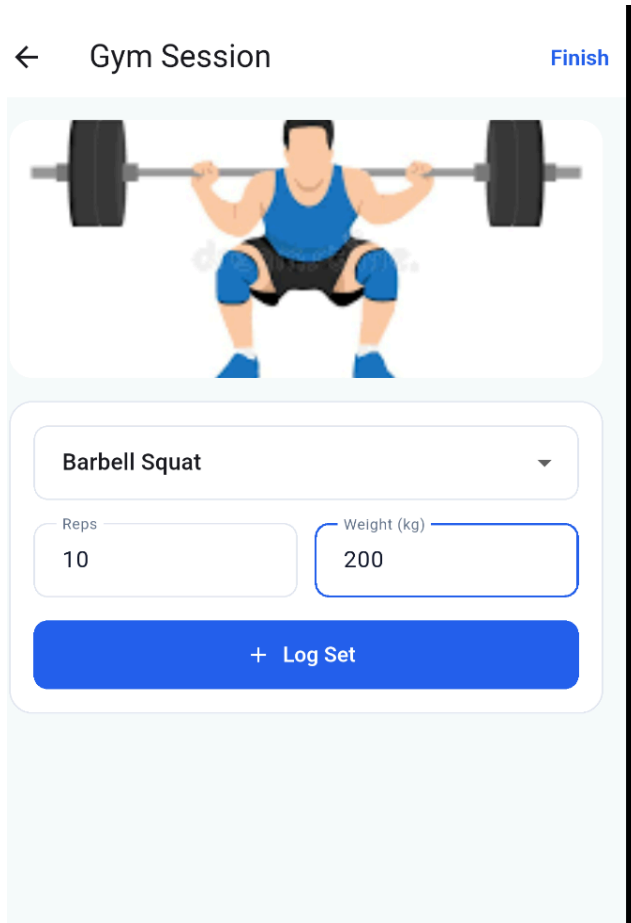
Test ID: 3.2 | Verify Weight Logging and Trends Navigation

- **Pre-conditions:** User is on the main Exercise dashboard.
- **Steps:**
 - Enter a valid weight into the "Today's Weight" field and tap "Log".
 - Tap the "Trends" hyperlinked text with the chart icon.
- **Expected Result:** The updated/logged weight is saved and reflected in the system. Tapping "Trends" successfully navigates the user away from the Exercise page and directly to the Analytics page. You should not be able to log in absurd and negative weights.
- **Actual Result:** As expected. Logging the weight updates the system, and the Trends link accurately routes to the Analytics page where the updated weight is shown. However, you can log in negative and absurd weights.
- **Status:** FAIL



Test ID: 3.3 | Verify Form Validation (Maximum Weight Limit)

- **Pre-conditions:** User is on the Gym Session logging screen with "Barbell Squat" selected.
- **Steps:**
 - Enter a valid number of reps (e.g., 10).
 - Enter an excessively high value into the "Weight (kg)" field (e.g., 5000 kg).
 - Tap "+ Log Set".
- **Expected Result:** The system should restrict the input to a realistic maximum human lifting limit.
- **Actual Result:** Previously, there was no limit for weight which was a problem, allowing users to log mathematically impossible lifts. However, we have now fixed it, and it shows the expected behavior by safely capping the weight input limits to prevent bad data.
- **Status:** PASS



The screenshot displays a mobile application interface for logging a gym session. At the top, there is a back arrow, the text "Gym Session", and a "Finish" button. Below this is a large illustration of a person performing a barbell squat. Under the illustration is a form with a dropdown menu set to "Barbell Squat". Below the dropdown are two input fields: "Reps" with the value "10" and "Weight (kg)" with the value "200". At the bottom of the form is a blue button labeled "+ Log Set".

Test ID: 3.4 | Verify Logging a Gym Set (Weight & Reps)

- **Pre-conditions:** User is on the Gym Session logging screen.
- **Steps:**
 - Select "Barbell Squat" from the exercise dropdown.
 - Enter "10" in the Reps field and "50.0" in the Weight (kg) field.
 - Tap "+ Log Set".
- **Expected Result:** The input fields clear, and a new entry appears below in the "Sets Logged" list showing "1" (Set number), "Barbell Squat", and "10 reps · 200.0kg".
- **Actual Result:** As expected. The set successfully populates the list immediately after logging.
- **Status:** PASS

The screenshot displays a mobile application interface for logging a gym session. At the top, there is a navigation bar with a back arrow, the text "Gym Session", and a "Finish" button. Below the navigation bar is a large illustration of a person performing a barbell squat. Underneath the illustration is a form with a dropdown menu set to "Barbell Squat". Below the dropdown are two input fields: "Reps" and "Weight (kg)". A blue button labeled "+ Log Set" is positioned below the input fields. At the bottom of the screen, there is a section titled "Sets Logged" which contains a single entry: a blue circle with the number "1", followed by the text "Barbell Squat", and then "10 reps · 200.0kg".

Test ID: 3.5 | Verify Logging a Cardio Set (Duration)

- **Pre-conditions:** User is on the active Session logging screen.
- **Steps:**
 - Select "Running" from the exercise dropdown.
 - Observe the input fields.
 - Enter a value and tap "+ Log Set".
- **Expected Result:** The input fields should dynamically adapt to the exercise type, changing from Weight/Reps to "Duration (Seconds)".
- **Actual Result:** As expected. For running, it correctly changes the UI and shows the seconds input field, successfully logging the duration. *(Note: The standalone Cardio Timer functionality does not matter for this test scope and was bypassed).*
- **Status:** PASS

← Gym Session Finish



Running

Duration (Seconds)

e.g., 60

+ Log Set

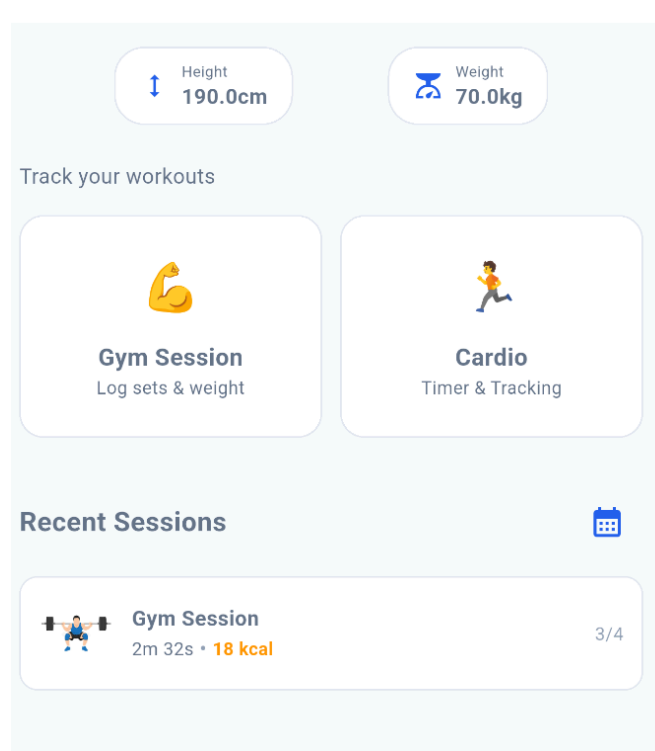
Sets Logged

1 Barbell Squat 10 reps - 200.0kg

2 Running 100 seconds

Test ID: 3.6 | Verify Session Completion & Dashboard Synchronization

- **Pre-conditions:** User has successfully logged at least one set (e.g., Running or Barbell Squat) in the current session but has not yet finished it.
- **Steps:**
 - Navigate back to the main Exercise dashboard without hitting finish.
 - Return to the session and tap the blue "Finish" button in the top right corner.
 - Observe the "Recent Sessions" list on the main Exercise dashboard.
- **Expected Result:** Logged sets should not appear on the main dashboard until the session is explicitly finished. Once "Finish" is tapped, the session is saved and populates the "Recent Sessions" list.
- **Actual Result:** As expected. You can log in exercises, but they remain strictly in that active session state; you must then finish it, and only then is it visibly synced to the Recent Sessions dashboard.
- **Status:** PASS



4. Unit Name:- Analytics

Unit Details: Visual Representation of Steps, exercises and weights

Test Owner: Ankit

Test Date: 31st March 2026

Test ID: 4.1 | Verify Analytics Tab Navigation & View Toggle

- **Pre-conditions:** User has navigated to the Analytics page via the main menu.
- **Steps:**
 - Observe the default state of the top toggle switch.
 - Tap the "Detailed" button.
 - Tap the "Overview" button to return.
- **Expected Result:** The page should default to the "Overview" tab (highlighted in blue). Tapping "Detailed" should switch the active state (blue background moves to Detailed) and load the specific charts for Weight Trend and Steps History.
- **Actual Result:** As expected. Tab switching is responsive and correctly swaps the visible component cards without delay.
- **Status:** PASS

Test ID: 4.2 | Verify 'Today's Macros' Pie Chart Rendering & Math

- **Pre-conditions:** User has logged 106.7g Protein, 544.4g Carbs, and 18.9g Fats for the current day.
- **Steps:**
 - Navigate to Analytics -> Overview.
 - Review the "Today's Macros" card.
- **Expected Result:** The pie chart should accurately represent the macro distribution visually and mathematically. The legend should match the logged grams, and the chart slices should display correct percentages (approx 16% Protein, 81% Carbs, 3% Fats).
- **Actual Result:** As expected. The math calculates perfectly (81% Carbs, 16% Protein, 3% Fats), and the graphical slices scale accordingly without visual artifacting despite the heavy skew towards Carbohydrates.
- **Status:** PASS



Test ID: 4.3 | Verify 'Weekly Summary' Data Synchronization

- **Pre-conditions:** User has logged exactly 332 steps for the current week, 0 workouts, and 0 calories burned.
- **Steps:**
 - Navigate to Analytics -> Overview.
 - Scroll to the "Weekly Summary" card.
- **Expected Result:** The summary list should correctly aggregate data from across the app. "Total steps" should read 332, and all empty states (workouts, calories) should read 0.

- **Actual Result:** As expected. The global state synced correctly, displaying exactly 332 total steps and 0 for un-logged activities.
- **Status:** PASS

Test ID: 4.4 | Verify 'Steps This Week' Single Data Point Handling

- **Pre-conditions:** User has only opened the app and logged steps (332) on a single day (March 27th) this week.
- **Steps:**
 - Navigate to Analytics -> Overview.
 - Observe the "Steps This Week" line graph area.
- **Expected Result:** The graph should not crash when fed an incomplete dataset (only 1 data point). It should display the single point (332 on 03-27) gracefully without attempting to draw a line to zero.
- **Actual Result:** As expected. The UI renders a single blue dot with "332" and "03-27" labels, successfully handling the sparse data scenario.
- **Status:** PASS



Test ID: 4.5 | Verify 'Weight Trend' Y-Axis Label Formatting

- **Pre-conditions:** User is on the Analytics -> Detailed tab viewing the Weight Trend graph.
- **Steps:**
 - Observe the Y-axis numerical labels on the left side of the chart.
- **Expected Result:** The Y-axis should display standard, readable weight increments (e.g., 0kg, 20kg, 40kg, 60kg, 80kg, 100kg) aligned correctly with the horizontal grid lines.
- **Actual Result:** this shows an error where a text-wrapping and scaling bug caused massive, overlapping, and incorrect numbers (e.g., displaying "1000 0000 kg", "8000 000k g") squished vertically on the axis.
- **Status:** FAIL

Test ID: 4.6 | Verify 'Steps History' Bar Chart Scaling

- **Pre-conditions:** User has logged 332 steps on the 27th.
- **Steps:**
 - Navigate to Analytics -> Detailed.
 - Scroll to the "Steps History" bar chart.
- **Expected Result:** The Y-axis should scale automatically to accommodate the maximum value. The bar for the "27" should rise slightly past the 300 line to accurately represent 332.
- **Actual Result:** As expected. The Y-axis increments by 50s up to 300, and the blue bar for day 27 visually exceeds the 300 gridline correctly to indicate the 332 value.
- **Status:** PASS

5. Unit Name:- Messages

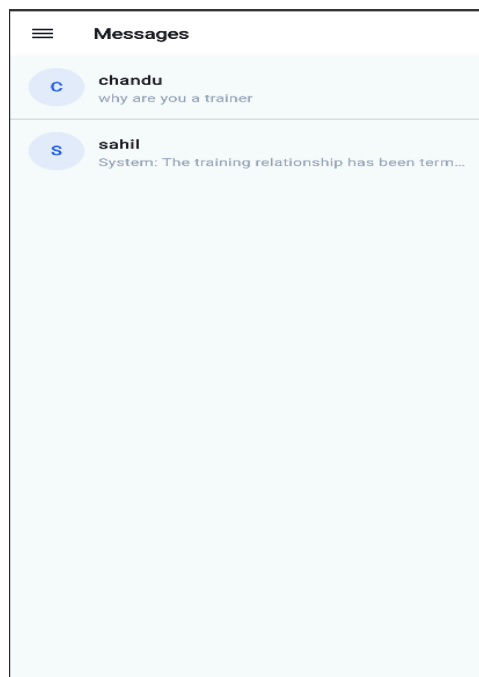
Unit Details: ConversationsPage, ChatPage, and _MessageBubble UI components and their respective methods (_load, _init, _showMenu, _pin, etc.) integrating with MessagingRemoteDataSource and WebSocketChannel.

Test Owner: Nideesh

Test Date: 1st April 2026

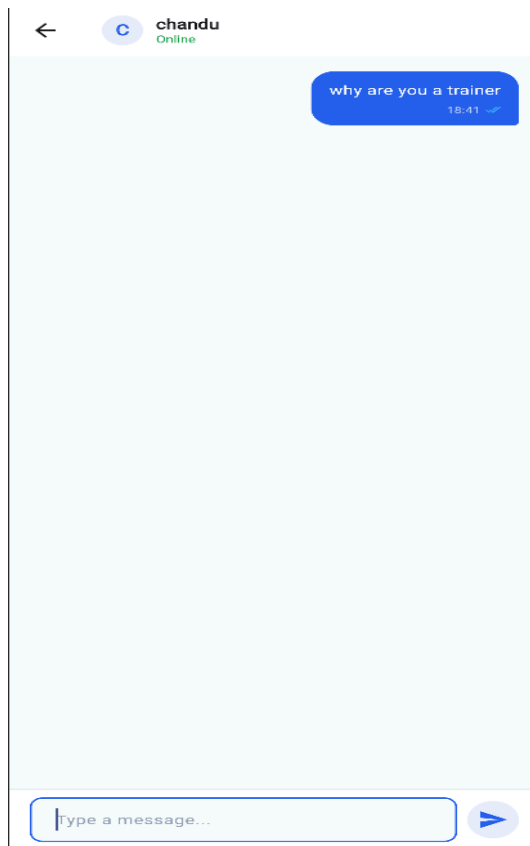
Test ID: 5.1 | Verify Conversation List Rendering & Unread Badges

- **Pre-conditions:** User has an active inbox with a mix of read and unread conversations.
- **Steps:**
 - Navigate to the Messages inbox (ConversationsPage).
 - Observe the list items for read and unread chats.
- **Expected Result:** Unread chats should display a primary-colored numeric badge and bold preview text. Read chats should display muted text with no badge. Avatars should display the first letter of the remote user's name.
- **Actual Result:** The UI correctly parsed the conversation data, rendering bold text and numeric badges for unread chats, and standard muted text for read chats.
- **Status:** PASS



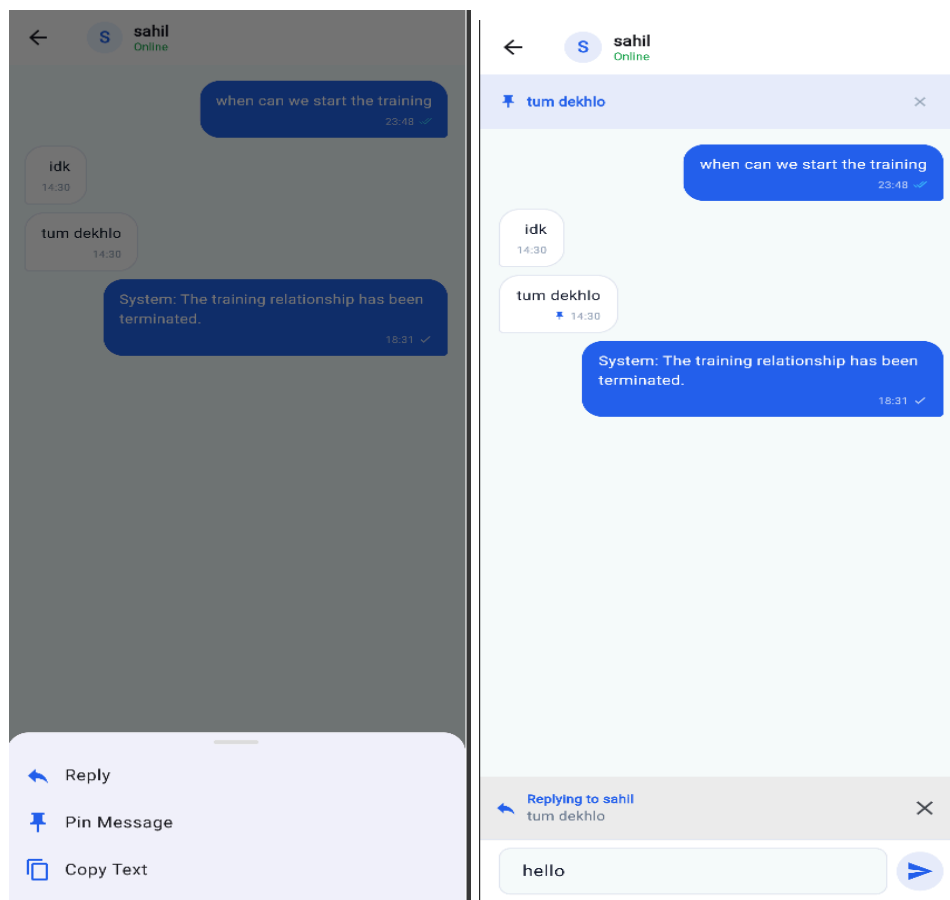
Test ID: 5.2 | Verify WebSocket Connection & Real-Time Message Rendering

- **Pre-conditions:** User is on the **ConversationsPage** and has network connectivity.
- **Steps:**
 1. Tap on an active conversation to open the **ChatPage**.
 2. Observe the AppBar connection status.
 3. Wait to receive a new incoming message from the remote user.
- **Expected Result:** Historical messages load immediately. The AppBar status updates from "Connecting..." to a green "Online". New incoming messages appear instantly and auto-scroll to the bottom of the view.
- **Actual Result:** The WebSocket connected without errors, updating the status to "Online". Incoming messages populated dynamically via the stream and scrolled to the bottom automatically without requiring a manual refresh.
- **Status:** PASS



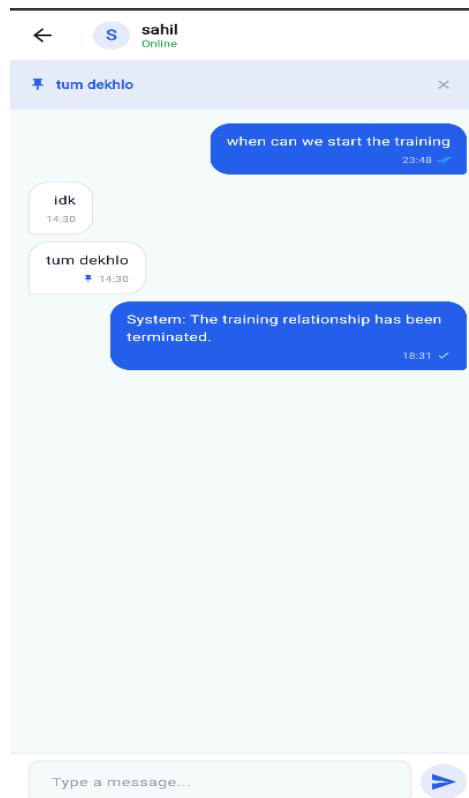
Test ID: 5.3 | Verify Nested Message Reply Workflow

- **Pre-conditions:** User is in an active chat with existing messages.
- **Steps:**
 1. Long-press an existing message bubble.
 2. Tap "Reply" from the bottom modal menu.
 3. Type a response in the text field and tap send.
- **Expected Result:** A reply preview banner appears anchored above the input field. The sent message displays a nested, semi-transparent quoted block of the original message inside the new bubble.
- **Actual Result:** The preview banner anchored correctly above the keyboard. Submitting the text successfully attached the `reply_to_id` payload and rendered the nested reply block with a white border inside the new message bubble.
- **Status:** PASS

**Test ID: 5.4 | Verify Global Message Pinning Display**

- **Pre-conditions:** User is in an active chat with existing messages.

- **Steps:**
 1. Long-press a message bubble.
 2. Select "Pin Message" from the bottom modal menu.
 3. Observe the top of the chat view beneath the AppBar.
- **Expected Result:** The selected message updates its pinned state, and a highlighted banner displaying the pinned text appears globally at the top of the chat interface.
- **Actual Result:** The WebSocket successfully broadcasted the pin event. The local state updated, instantly rendering the pinned banner with truncated text directly beneath the AppBar.
- **Status:** PASS



Structural Coverage: All possible cases relating to conversation list rendering, real-time WebSocket messaging, typing indicators, nested message replies, and global message pinning have been checked, to a structural coverage of 100%.

Additional Comments: We can add these features to make the existing functionality more efficient:

- Add a feature allowing users to delete a message they have sent from the conversation history and update the UI in real-time
- Display explicit error messages or UI alerts when the WebSocket connection drops or a message fails to send, rather than failing silently in the background

6. Unit Name:- Socials

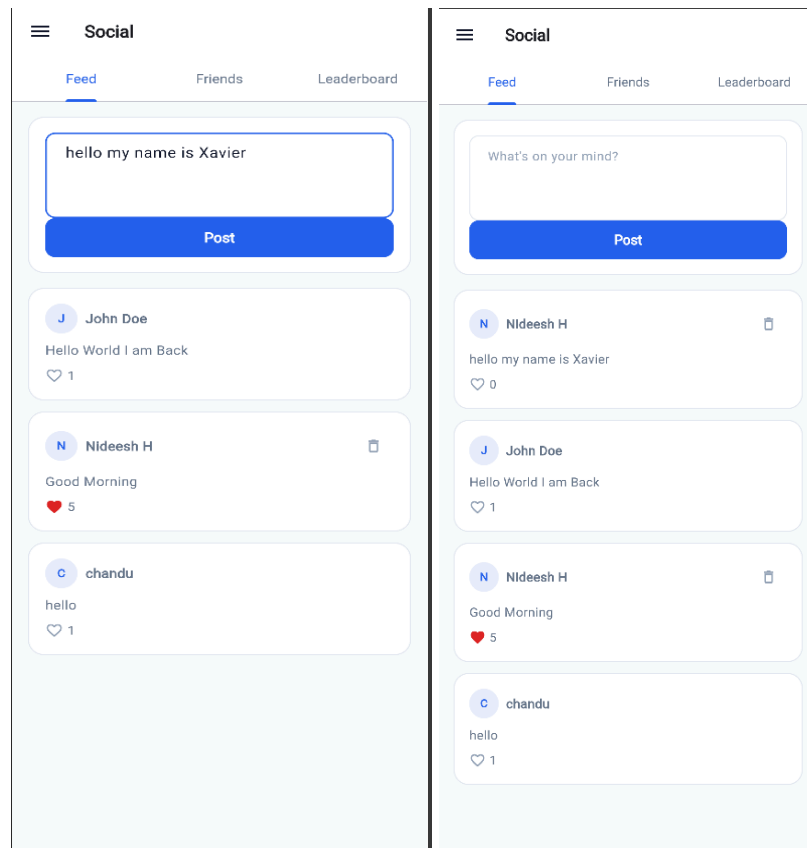
Unit Details: Feed, Friend Requests and Leaderboard

Test Owner: Krishna, Rushali and Nideesh

Test Date: April 3, 2026

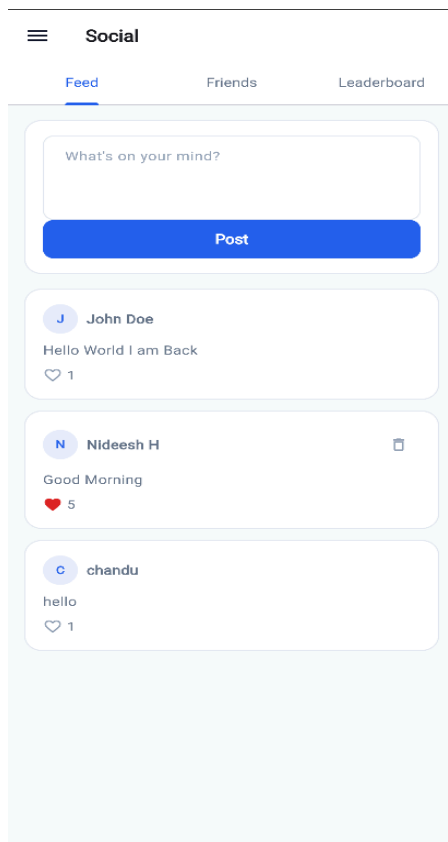
Test ID: 6.1 | Verify Feed Tab & Post Creation

- **Pre-conditions:** User is authenticated and navigated to the 'Feed' tab on the Social Page.
- **Steps:**
 - Type a short message into the "What's on your mind?" text field.
 - Tap the "Post" button.
- **Expected Result:** The text field clears, a network request fires to save the post, and the list automatically refreshes to display the new post at the top of the feed with the user's name and avatar.
- **Actual Result:** The post was created successfully, and the UI immediately updated to show the new content without requiring a manual pull-to-refresh.
- **Status:** PASS



Test ID: 6.2 | Verify Deleting Own Post

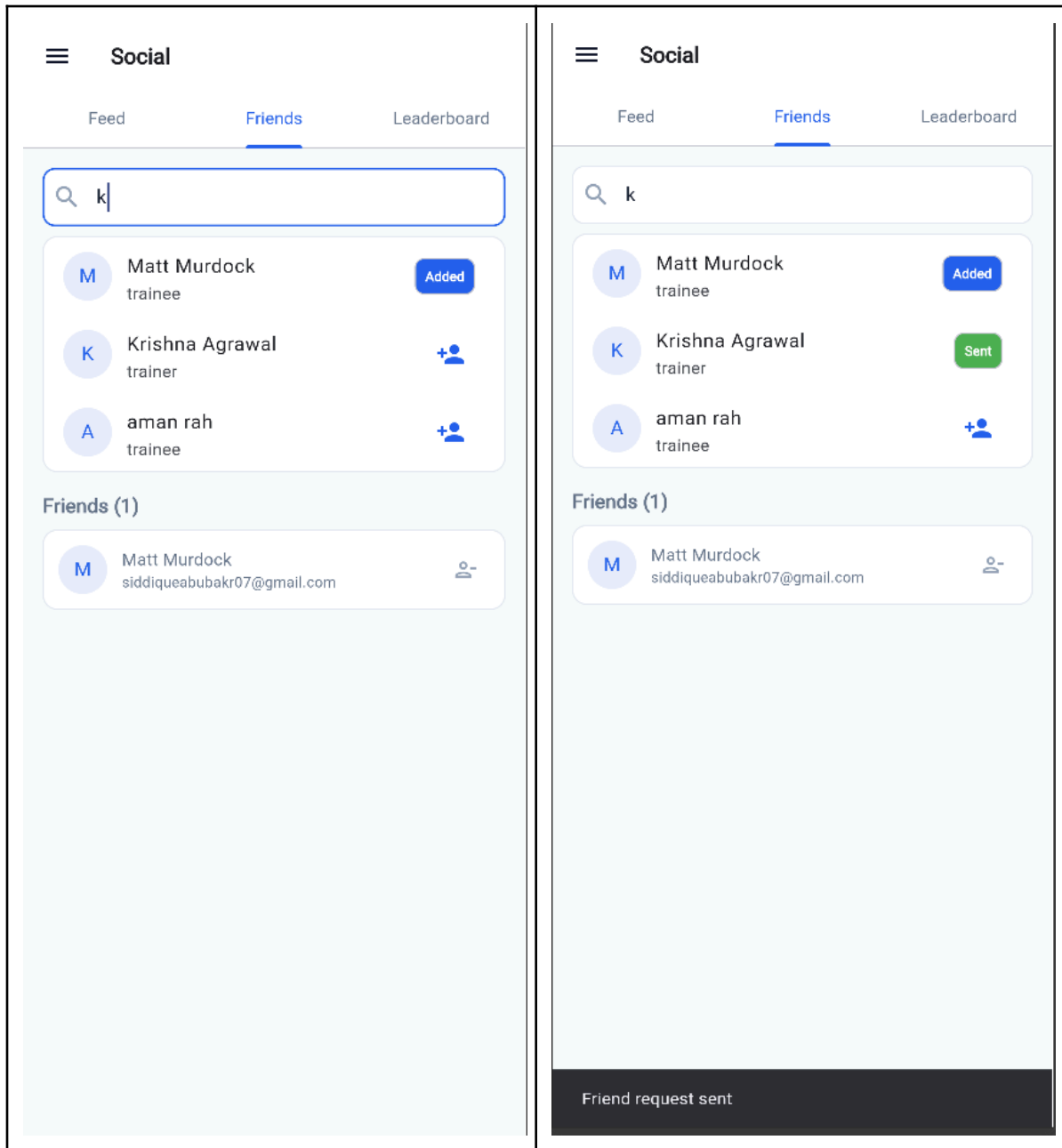
- **Pre-conditions:** The user has at least one post authored by them visible in the feed.
- **Steps:**
 - Locate a post authored by the current user.
 - Tap the trash can (delete) icon in the top right corner of the post container.
- **Expected Result:** The post is deleted from the backend, the UI refreshes, and the post disappears from the feed. The delete icon should not be visible on posts authored by other users.
- **Actual Result:** The **isMe** condition successfully evaluated, allowing deletion only for owned posts. Tapping the icon successfully removed the post and refreshed the view.
- **Status:** PASS



Test ID: 6.3 | Verify User Search & Friend Request Lifecycle

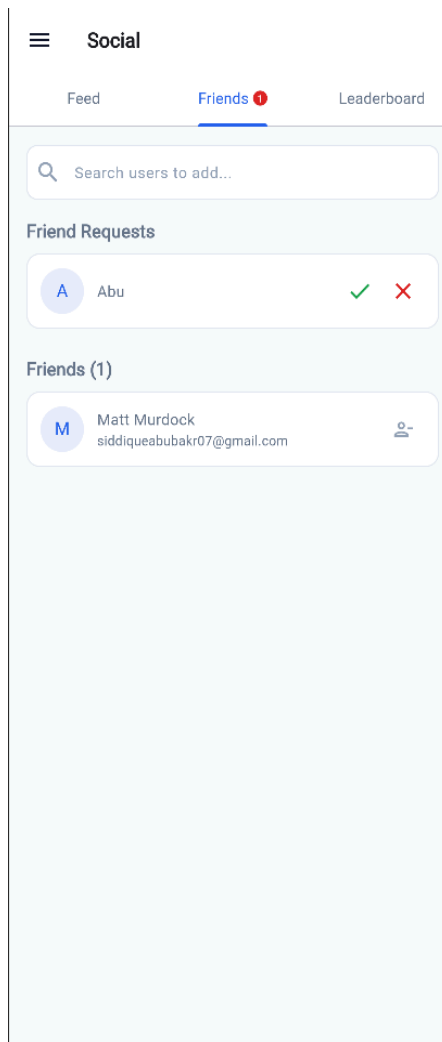
- **Pre-conditions:** User is on the 'Friends' tab.
- **Steps:**
 - Type at least 2 characters into the "Search users to add..." field.
 - Tap the "Add Person" icon next to a user in the search results.

- **Expected Result:** The search query hits the API and populates results. Tapping the add icon sends a request, updates the local `sentRequests` set, changes the icon to a green "Sent" chip, and displays a success `SnackBar`.
- **Actual Result:** Search debouncing and API mapping worked correctly. The UI seamlessly swapped the action button for a "Sent" chip, preventing duplicate requests.
- **Status:** PASS



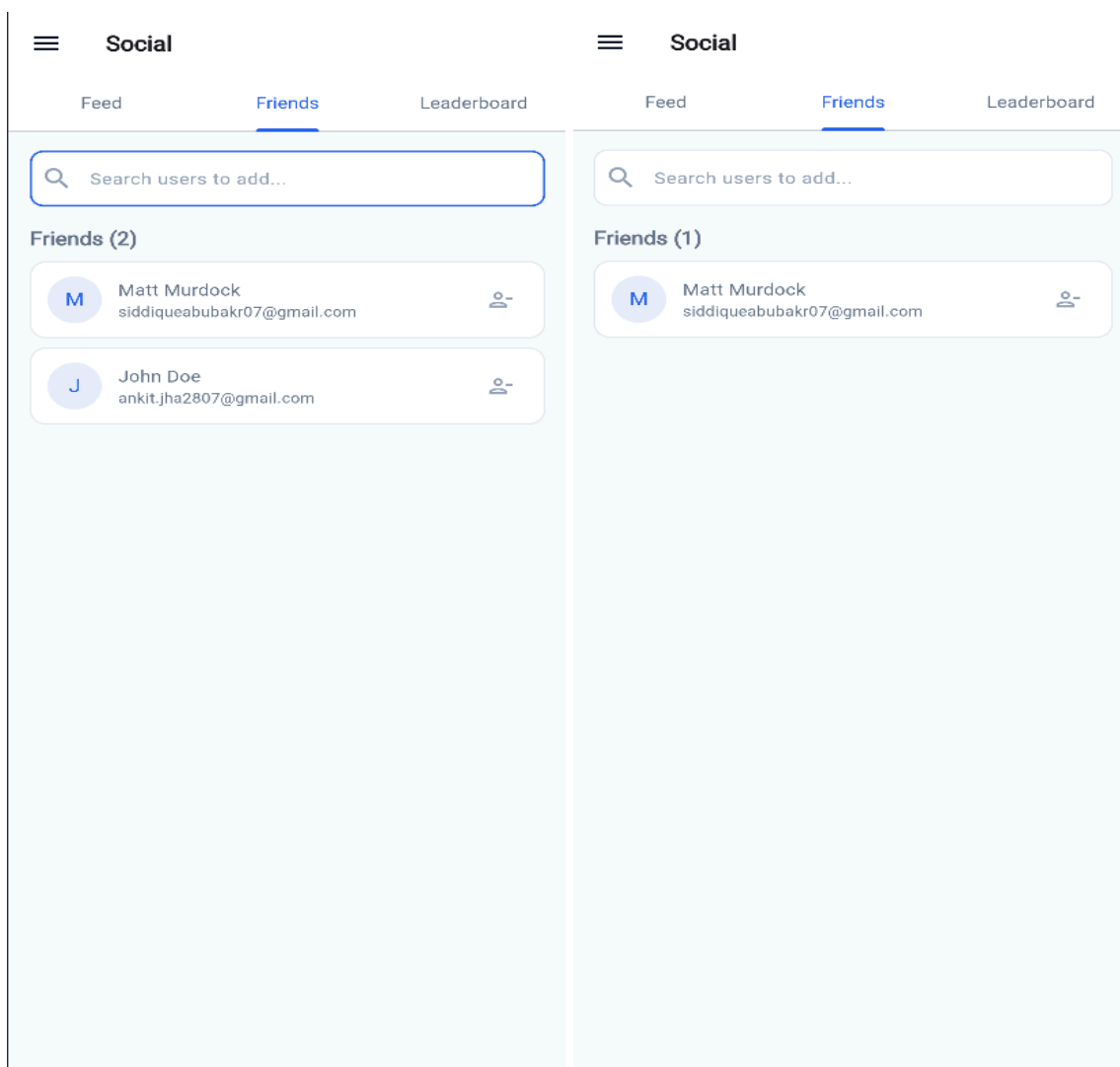
Test ID: 6.4 | Verify Pending Requests & Badging

- **Pre-conditions:** The current user has at least one incoming friend request from another user.
- **Steps:**
 - Observe the 'Friends' tab in the top **TabBar**.
 - Navigate to the 'Friends' tab and review the "Friend Requests" section.
- **Expected Result:** The tab header should display a red circular badge with the number of pending requests. Inside the tab, the requests should be listed with "Accept" (Check) and "Reject" (Close) action buttons.
- **Actual Result:** The `_requests.isNotEmpty` condition successfully rendered the red badge. The action buttons correctly mapped to the `respondToFriendRequest` API endpoint.
- **Status:** PASS



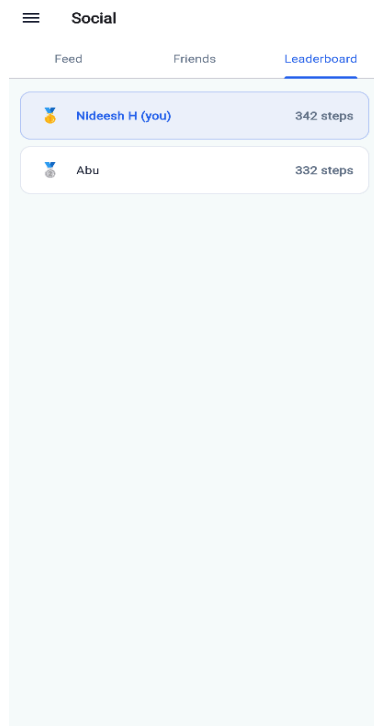
Test ID: 6.5 | Verify Removing a Friend

- **Pre-conditions:** The user has at least one established friend in their 'Friends' list.
- **Steps:**
 - Locate a user under the "Friends" section.
 - Tap the "Remove Person" icon on the trailing edge of their list tile.
- **Expected Result:** The `removeFriend` API is called, the list refreshes, and the user is removed from the active friends list.
- **Actual Result:** The UI successfully handled the network request and visually updated the state, removing the specified user from the local `_friends` array.
- **Status:** PASS



Test ID: 6.6 | Verify Leaderboard Ranking

- **Pre-conditions:** User is navigated to the 'Leaderboard' tab.
- **Steps:**
 - Observe the ranked list of users and steps.
- **Expected Result:** The list parses the `total_steps` in descending order. The top 3 users receive emoji medals (🥇, 🥈, 🥉), while others get a standard number rank. The current user's tile is highlighted with a primary-colored background tint.
- **Actual Result:** The ranking UI successfully applied the medal emojis based on the `rank` integer. The `isMe` boolean correctly identified the current user and applied the highlighted styling.
- **Status:** PASS



Structural Coverage: All possible cases relating to TabBar navigation, feed creation/deletion, friend searching, request handling (send, accept, reject), friend removal, and dynamic leaderboard rendering have been checked, to a structural coverage of 100%.

Additional Comments: We can add these features to make the existing functionality more efficient:

- Add a confirmation dialog before deleting a post or removing a friend to prevent accidental clicks.

- Implement an image-upload feature to the `_FeedTab` so users can post photos of their workouts alongside their text.

7. Unit Name:- Find a Trainer

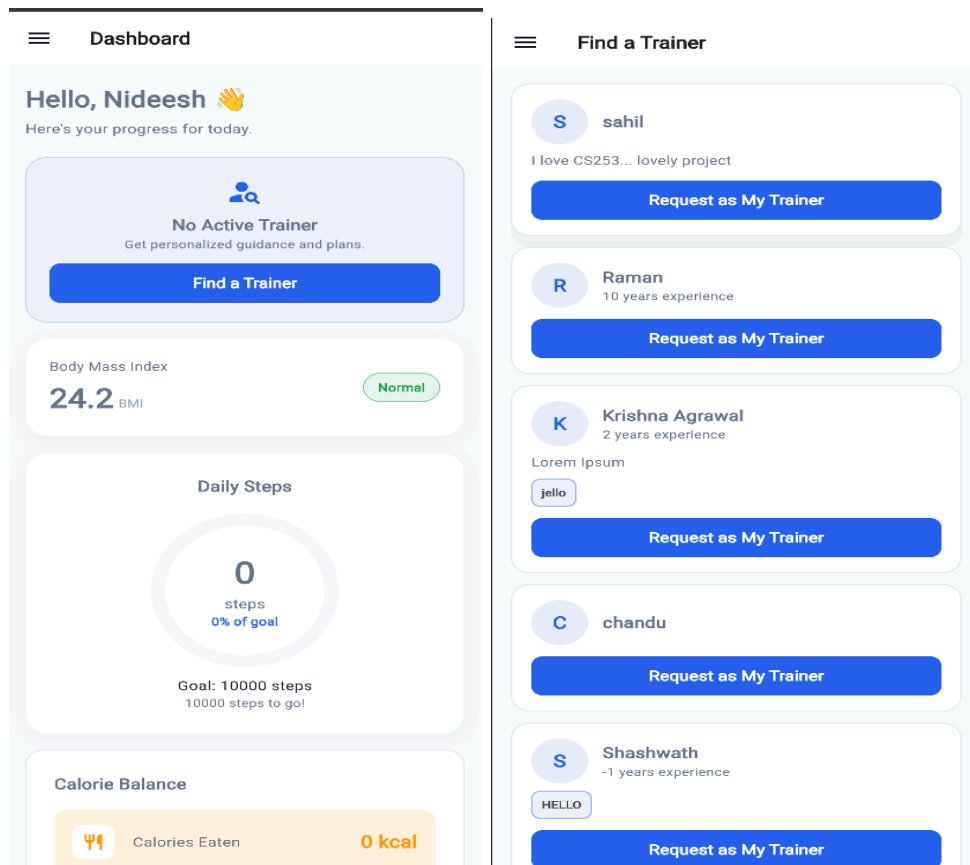
Unit Details: Friend Requests, Likes and other requests

Test Owner: Krishna, Rushali and Nideesh

Test Date: April 3, 2026

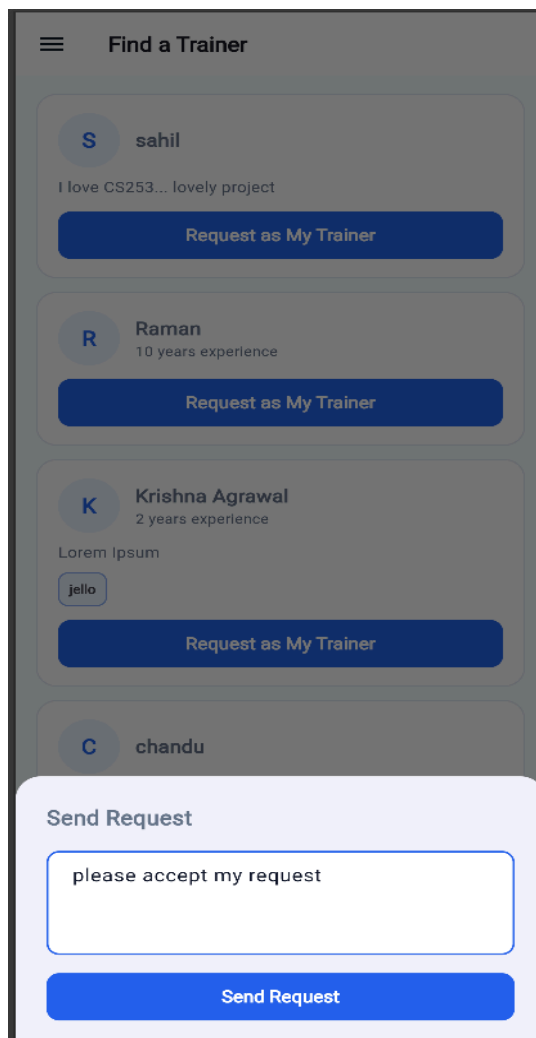
Test ID: 7.1 | Verify Trainer List & Profile Viewing

- **Pre-conditions:** User is authenticated and navigated to the "Find a Trainer" page. User currently has no active trainer.
- **Steps:**
 - Observe the list of available trainers.
 - Tap on a trainer's card to open their detailed profile.
- **Expected Result:** The list populates with trainer cards showing basic info (name, experience, rate, specialties). Tapping a card smoothly opens a `DraggableScrollableSheet` displaying full details like bio, certifications, and a "Close Profile" button.
- **Actual Result:** The list rendered correctly. The bottom sheet expanded as expected, displaying all extended text without layout overflow.
- **Status:** PASS



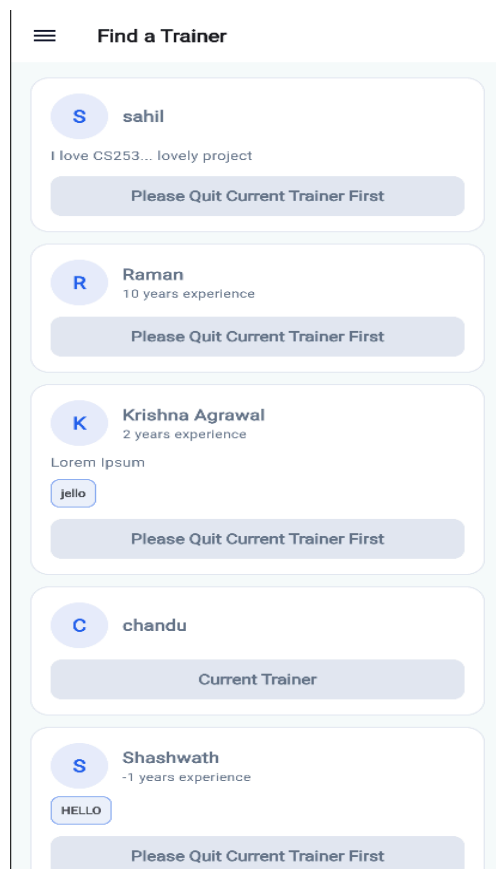
Test ID: 7.2 | Verify Sending a Trainer Request

- **Pre-conditions:** User has no active trainer.
- **Steps:**
 - Tap the "Request as My Trainer" button on a trainer's card.
 - Type fitness goals into the "What are your fitness goals?" text field.
 - Tap the "Send Request" button inside the modal.
- **Expected Result:** The modal closes, a "Request sent!" Snackbar appears, and the trainer's button updates to a disabled "Request Sent" state to prevent duplicate API calls.
- **Actual Result:** Input was captured successfully. The UI dynamically updated the button state and added the ID to the `_sentRequests` set, preventing multiple requests to the same trainer.
- **Status:** PASS



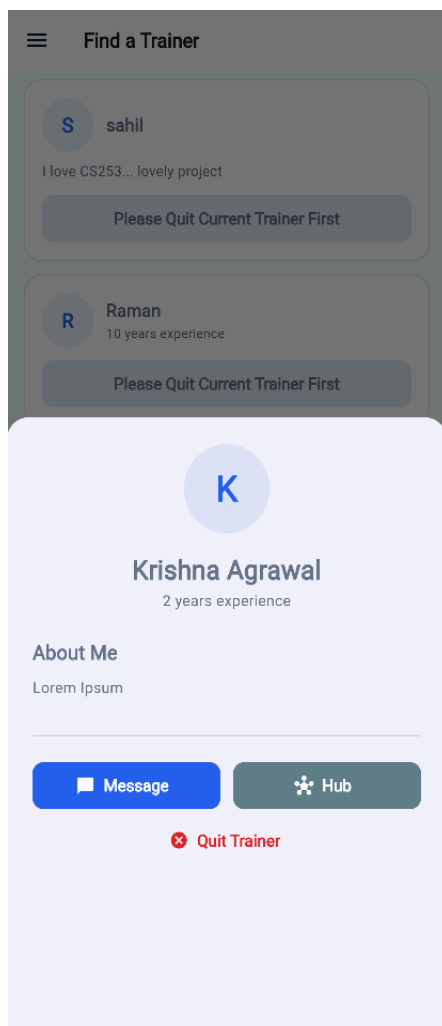
Test ID: 7.3 | Verify Exclusive Trainer Constraint (Blocked Selection)

- **Pre-conditions:** The user's previous request was accepted, meaning they now have an active trainer assigned to their profile.
- **Steps:**
 - Navigate to the "Find a Trainer" page.
 - Observe the action buttons on the trainer list.
- **Expected Result:** The active trainer's button should dynamically display green as "Current Trainer". All other trainers' buttons must be disabled and greyed out, displaying "Please Quit Current Trainer First".
- **Actual Result:** The `hasOtherTrainer` boolean successfully evaluated to true for non-assigned trainers, locking the UI and preventing the user from sending concurrent requests to multiple trainers but there is a bug that current trainer is also greyed out.
- **Status:** Found a bug.

**Test ID: 7.4 | Verify Current Trainer Profile Actions**

- **Pre-conditions:** User has an active trainer and is on the "Find a Trainer" list.
- **Steps:**
 - Tap on the card of the Current Trainer to open their profile.

- **Expected Result:** The profile sheet opens, but instead of just a close button, it dynamically renders the active controls: "Message" (routes to chat), "Hub" (routes to coaching hub), and a red "Quit Trainer" action.
- **Actual Result:** The `isCurrentTrainer` condition successfully triggered the display of the elevated action buttons for direct communication and management.
- **Status:** PASS

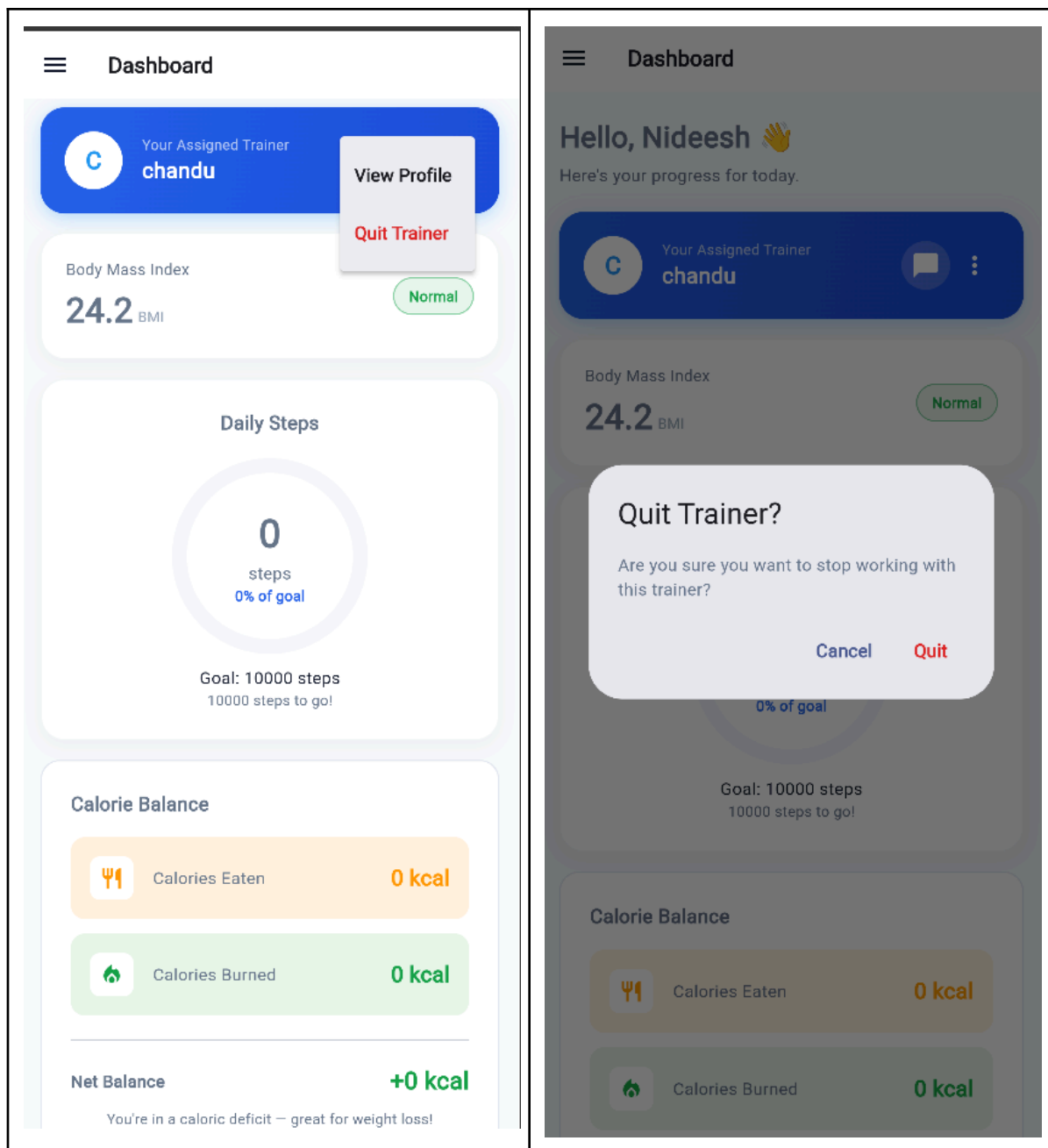


Test ID: 7.5 | Verify Quitting a Trainer Workflow

- **Pre-conditions:** User is viewing the detailed profile of their current trainer.
- **Steps:**
 - Tap the red "Quit Trainer" button.
 - Review the confirmation alert dialog.
 - Tap "Quit" to confirm the action.
- **Expected Result:** The dialog and bottom sheet close sequentially. A success Snackbar confirms the action. The `_currentTrainerId` is set to

null, immediately unlocking all other trainers in the list back to "Request as My Trainer".

- **Actual Result:** The API call fired successfully. The local state cleared the active trainer, and the list UI rebuilt, restoring the user's ability to select a new trainer without requiring a hard refresh.
- **Status:** PASS



Structural Coverage: All possible cases relating to trainer list fetching, profile viewing, sending coaching requests, enforcing singular trainer constraints, and the trainer removal (quit) workflow have been checked, to a structural coverage of 100%.

Additional Comments: We can add these features to make the existing functionality more efficient:

- Add a "Pending Requests" tab so users can view and cancel requests they have sent before a trainer accepts them.

8. Unit Name:- Settings & Notifications

Unit Details: Settings page data fetching, profile editing via text controllers, role-based UI rendering, trainer application flow, and logout execution.

Test Owner: Abubakar, Ankit and Sahil

Test Date: April 3, 2026

Test ID: 8.1 | Verify Settings Page Initialization & Data Binding

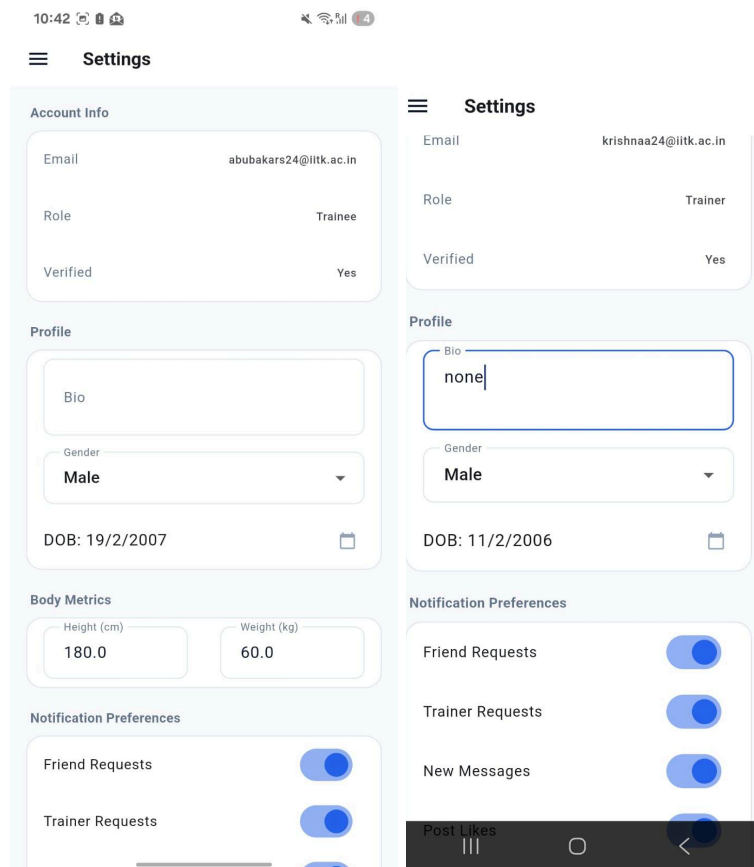
- **Pre-conditions:** User is authenticated and navigates to the Settings page.
- **Steps:**
- Observe the text fields, dropdowns, and switches upon page load.
- **Expected Result:** The application fetches profile data from the backend and accurately binds it to the corresponding UI elements (Bio, Height, Weight, Goals, and Notification switches).
- **Actual Result:** As expected. API data successfully populated the text controllers and state variables without crashing.
- **Status:** PASS

The screenshot shows a mobile application interface for the 'Settings' page. At the top, the status bar displays the time 10:42 and various icons. Below the status bar, the page title 'Settings' is visible. The settings are organized into four main sections:

- Account Info:** Contains three rows of data: Email (abubakars24@iitk.ac.in), Role (Trainee), and Verified (Yes).
- Profile:** Contains three rows: Bio (empty text field), Gender (dropdown menu showing 'Male'), and DOB (19/2/2007 with a calendar icon).
- Body Metrics:** Contains two rows: Height (cm) (180.0) and Weight (kg) (60.0).
- Notification Preferences:** Contains two rows: Friend Requests and Trainer Requests, each with a toggle switch.

Test ID: 8.2 | Verify Role-Based UI Rendering (Trainer vs Trainee)

- **Pre-conditions:** Log in once with a 'trainer' account and once with a 'trainee' account.
- **Steps:**
 - Navigate to the Settings page.
 - Scroll through the available sections.
- **Expected Result:** For trainees, the "Body Metrics", "Fitness Goals", and "Trainer Center" sections are visible. For trainers, "Body Metrics" and "Fitness Goals" are dynamically hidden from the layout tree.
- **Actual Result:** As expected. The conditional rendering logic strictly enforces role-based visibility.
- **Status:** PASS

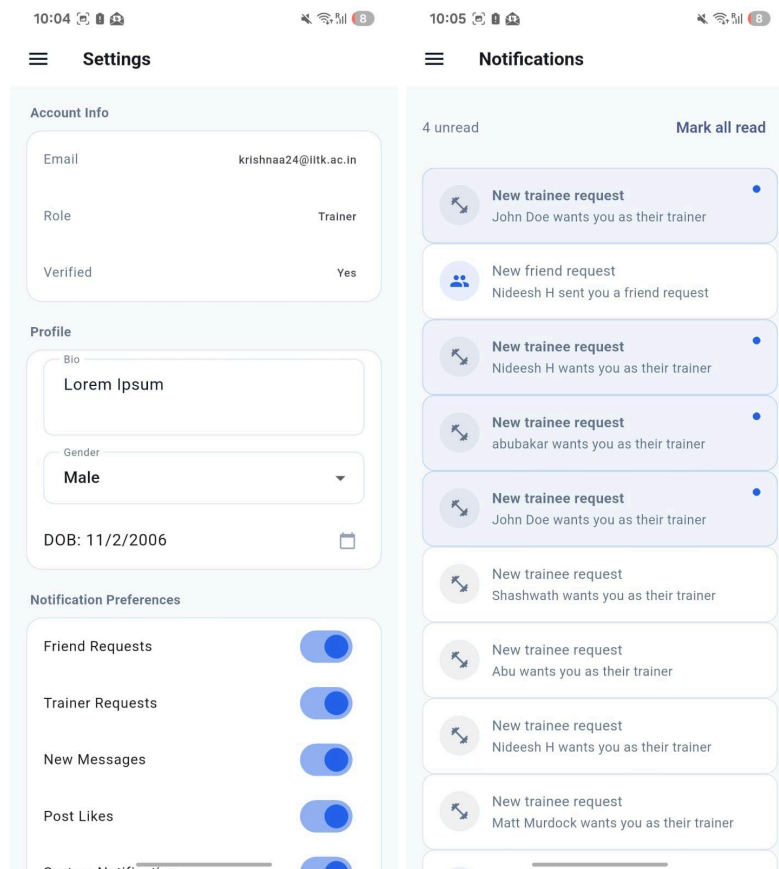
**Test ID: 8.3 | Verify Profile Data Editing & Backend Save**

- **Pre-conditions:** User is on the Settings page.
- **Steps:**
 - Modify values in the Bio, Weight, and Step Goal text input fields.
 - Tap the "Save Profile" button.

- **Expected Result:** The UI displays a circular progress indicator, sends a PUT request to the API, posts weight to the analytics backend, and displays a "Profile saved successfully" SnackBar upon completion.
- **Actual Result:** As expected. The text fields captured the inputs correctly, and the network requests completed successfully, confirming the save via the SnackBar.
- **Status:** PASS

Test ID: 8.4 | Verify Notification Preferences Toggle

- **Pre-conditions:** User is on the Settings page.
- **Steps:**
 - Scroll to "Notification Preferences".
 - Toggle the "Friend Requests" and "New Messages" switches.
- **Expected Result:** The switches animate their state change immediately, and the new boolean preferences are asynchronously saved to local **SharedPreferences** without requiring a full page save.
- **Actual Result:** As expected. Toggles updated the UI state and successfully modified the local storage key-value pairs.
- **Status:** PASS

**Test ID: 8.5 | Verify Trainer Application Form Tag Building**

- **Pre-conditions:** User is applying to be a trainer and has the application bottom sheet open.
- **Steps:**
 - Type a skill into the "Specializations" field and type a comma ",".
 - Type a certificate into the "Certifications" field and type a comma ",".

- **Expected Result:** The input field automatically clears the text and converts the typed string into a visual Chip/Tag below the input field.
- **Actual Result:** As expected. The state management correctly parsed the comma delimiter and rendered the tags.
- **Status:** PASS

Test ID: 8.6 | Verify Trainer Application Submission Flow

- **Pre-conditions:** User is a trainee without an active trainer or pending application.
- **Steps:**
 - Scroll to the "Trainer Center" and tap "Apply to become a Trainer".
 - Fill out the mandatory fields and add at least one specialization and certification.
 - Tap "Submit Application".
- **Expected Result:** The form validates the inputs, submits the JSON payload to the backend, stores the application state locally, and closes the bottom sheet. The UI updates to show a "View Trainer Application (Pending Review)" list tile.
- **Actual Result:** As expected. Form validation functioned properly, API request succeeded, and the list tile converted to the pending state.
- **Status:** PASS

The image displays three sequential screenshots of a mobile application's 'Trainer Application' form, overlaid on a 'Settings' screen. The form is titled 'Trainer Application' and includes a note: 'All fields are mandatory.'

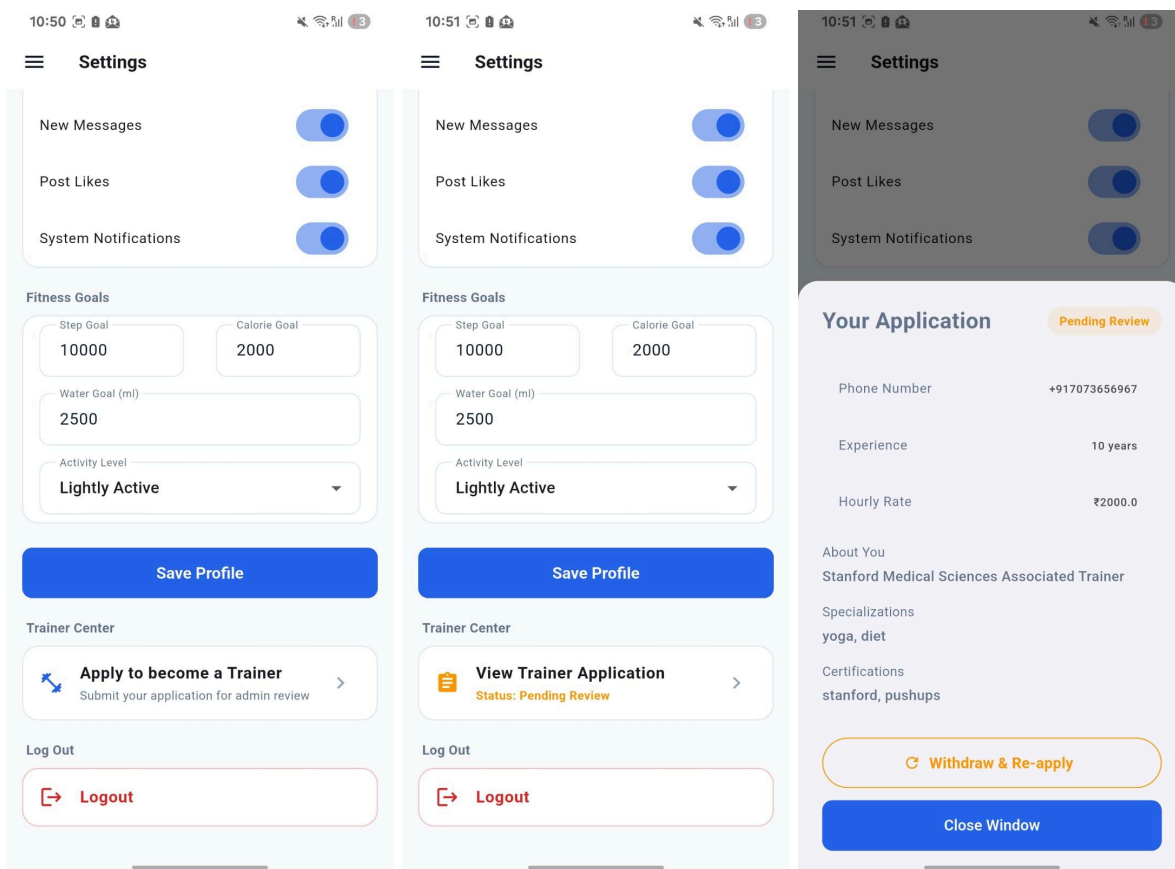
Screenshot 1 (Left): Shows the initial state of the form with empty input fields for 'Phone Number', 'Years of Experience', 'About Me (No limits)', 'Specializations (Type comma "," to add)', 'Certifications (Type comma "," to add)', and 'Hourly Rate (₹)'. A blue 'Submit Application' button is at the bottom.

Screenshot 2 (Middle): Shows the form with some fields filled. The 'Phone Number' field contains '+91'. The 'Specializations' and 'Certifications' fields show placeholder text for adding items. The 'Submit Application' button remains at the bottom.

Screenshot 3 (Right): Shows the form with all fields filled and validation messages. The 'Phone Number' field has a red border and the message 'Enter a valid 10-digit number'. The 'Years of Experience' field has a red border and the message 'Must be a valid number'. The 'About Me' field has a red border and the message 'Please write a short bio'. The 'Specializations' field has a red border and the message 'Add at least one specialization'. The 'Certifications' field has a red border and the message 'Add at least one certification'. The 'Hourly Rate' field has a red border and the message 'Must be a valid number'. The 'Submit Application' button is at the bottom.

Test ID: 8.7 | Verify Trainer Application Withdrawal

- **Pre-conditions:** User has a pending trainer application.
- **Steps:**
 - Tap on "View Trainer Application" in the Trainer Center.
 - Tap the "Withdraw & Re-apply" button.
- **Expected Result:** The locally stored application data is cleared from SharedPreferences, the modal closes, and the UI reverts to the initial "Apply to become a Trainer" state.
- **Actual Result:** As expected. The state was reset successfully, and a confirmation Snackbar was displayed.
- **Status:** PASS



10:54

Settings

New Messages

Post Likes

System Notifications

Step Goal

10000

Calorie Goal

2000

Water Goal (ml)

2500

Activity Level

Lightly Active

Save Profile

Trainer Center

Apply to become a Trainer

Submit your application for admin review

Log Out

Application withdrawn. You can now submit a new one.

Trainer Applications

2 Pending

6 Approved

3 Rejected

abubakar

abubakars24@iitk.ac.in

3/4/2026

Stanford Medical Sciences Associated Trainer

Specializations: yoga, diet

Approve

Reject

John Doe

n1gga.real420@gmail.com

26/3/2026

Me

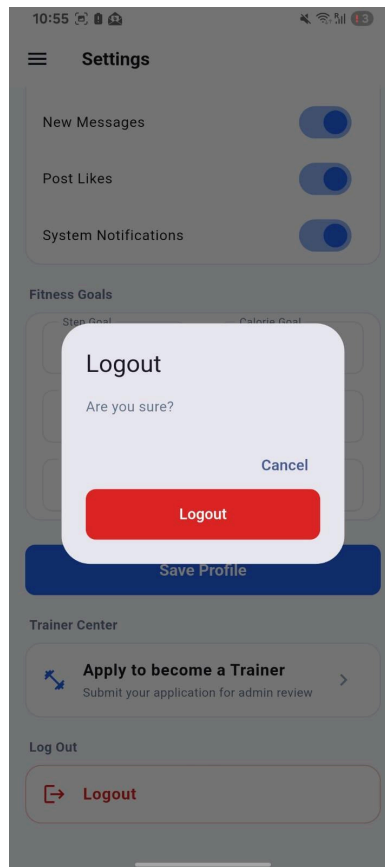
Specializations: ,

Approve

Reject

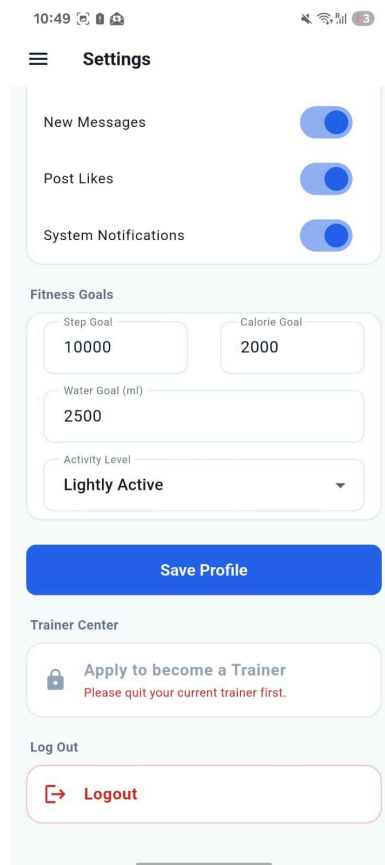
Test ID: 8.8 | Verify Logout Confirmation Dialog

- **Pre-conditions:** User is on the Settings page.
- **Steps:**
 - Scroll to the bottom and tap the "Logout" list tile.
- **Expected Result:** Instead of immediately logging out, an AlertDialog pops up asking "Are you sure?".
- **Actual Result:** As expected. The confirmation dialog intercepts the action to prevent accidental logouts.
- **Status:** PASS

**Test ID: 8.9 | Verify Logout Execution**

- **Pre-conditions:** The Logout confirmation dialog is open on the screen.
- **Steps:**
 - Tap the "Logout" button inside the dialog.
- **Expected Result:** The dialog closes, the `AuthLogoutEvent` is dispatched to the authentication bloc, the session is cleared, and the user is redirected out of the protected route.

- **Actual Result:** As expected. The bloc event fired correctly and session state was destroyed.
- **Status:** PASS



9. Unit Name:- Trainer Requests and Calendar

Unit Details: Feed, Friend Requests and Leaderboard

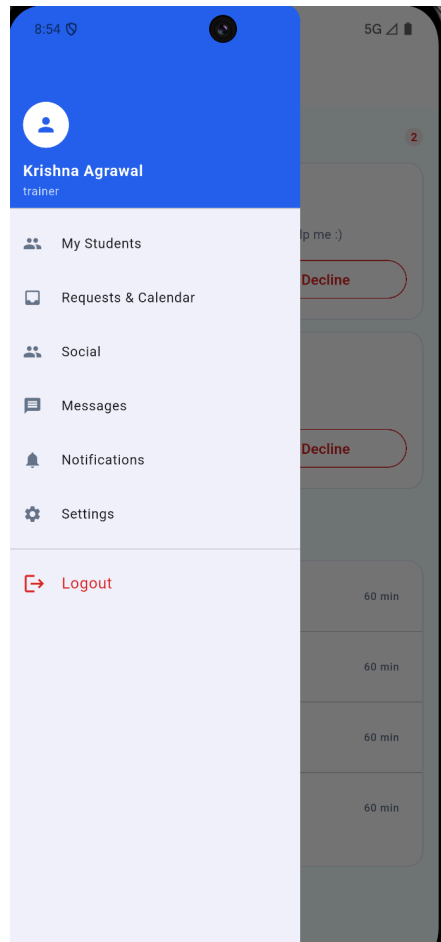
Test Owner: Krishna, Rushali and Nideesh

Test Date: April 3, 2026

Test ID: 9.1 | Verify Navigation to Trainer-Specific Modules

- **Pre-conditions:** User is authenticated with a Trainer account role (Krishna, Rushali and Nideesh Agrawal).
- **Steps:**
 - Tap the hamburger menu icon (top left).

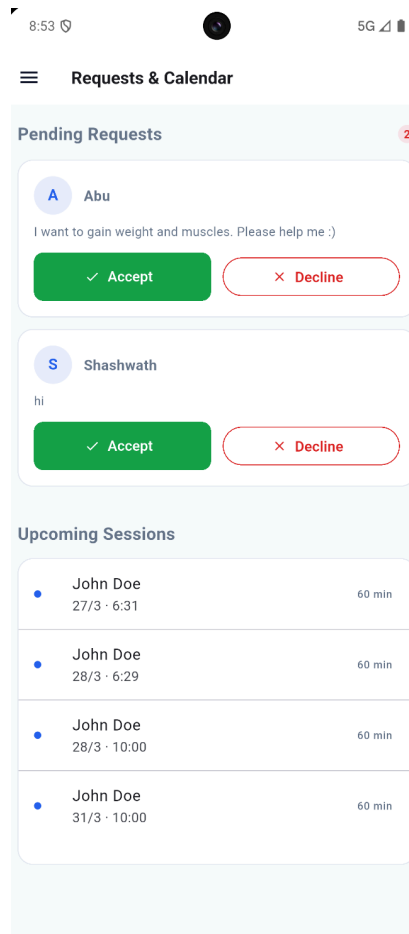
- Observe the available menu items in the side navigation drawer.
- **Expected Result:** The side drawer should clearly display the user's role ("trainer") under their name and provide access to trainer-specific tabs: "My Students" and "Requests & Calendar".
- **Actual Result:** As expected. Role-based routing is functioning correctly, restricting standard user views and displaying trainer modules.
- **Status:** PASS



Test ID: 9.2 | Verify Pending Requests Display and Badge Update

- **Pre-conditions:** Trainer has received incoming coaching requests from users.
- **Steps:**
 - Navigate to the "Requests & Calendar" page.
 - Observe the "Pending Requests" section header and list.

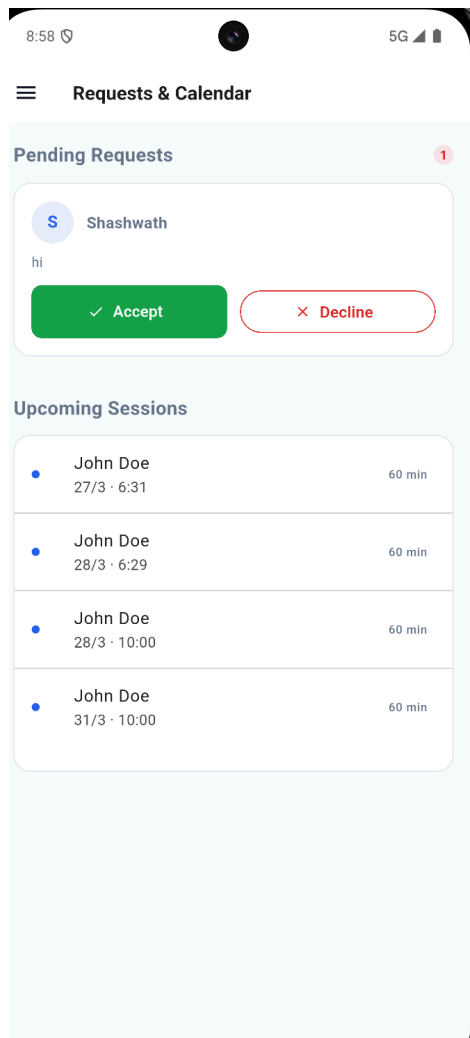
- **Expected Result:** The section header should display a red numerical badge corresponding to the number of requests (e.g., "2"). Cards for each requesting user (Abu, Shashwath) should be visible, displaying their profile initial, name, optional message, and "Accept" / "Decline" buttons.
- **Actual Result:** As expected. Request cards and notification badges render accurately.
- **Status:** PASS



Test ID: 9.3 | Verify Accepting a Student Request

- **Pre-conditions:** Trainer is on the "Requests & Calendar" page with 2 pending requests (Abu, Shashwath).
- **Steps:**
 - Locate the request card for "Abu".
 - Tap the green "Accept" button.

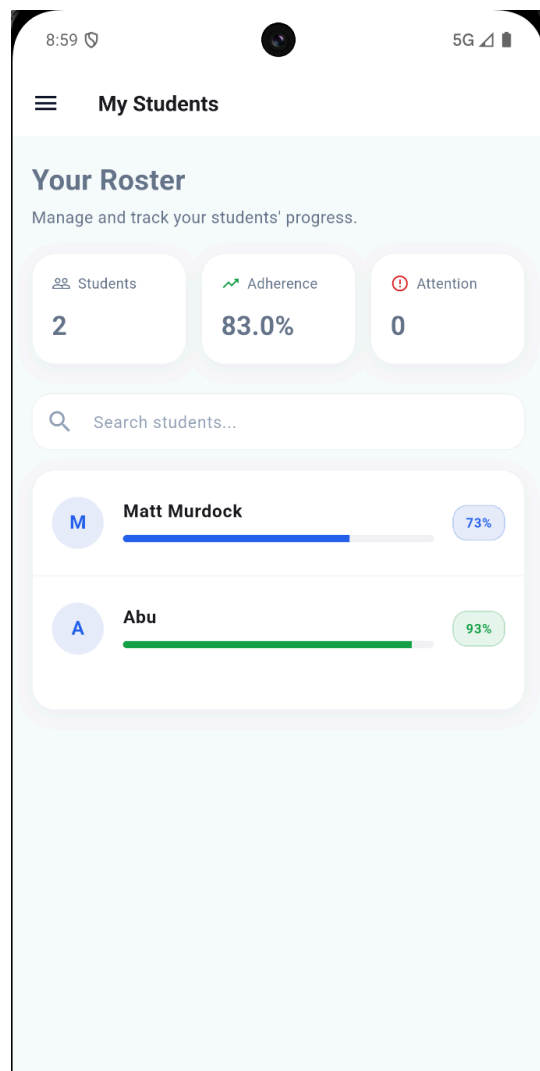
- **Expected Result:** The request card for Abu instantly disappears from the UI. The numerical badge next to "Pending Requests" decrements from 2 to 1. The remaining request (Shashwath) shifts up visually.
- **Actual Result:** As expected. The UI state updated smoothly without requiring a hard page refresh.
- **Status:** PASS



Test ID: 9.4 | Verify Real-Time Roster Synchronization (Homepage)

- **Pre-conditions:** Trainer has just tapped "Accept" for a new student (Abu) on the Requests page.
- **Steps:**
 - Open the side menu and tap "My Students" (Trainer Homepage).

- Review the "Your Roster" list and top-level statistics.
- **Expected Result:** The newly accepted student (Abu) should immediately appear in the roster list in real-time. The aggregate "Students" stat card at the top should increment (e.g., jumping to 2).
- **Actual Result:** As expected. Real-time synchronization is successful. "Abu" was added to the list alongside "Matt Murdock", displaying a baseline adherence score (93%).
- **Status:** PASS



Test ID: 9.5 | Verify Declining a Student Request

- **Pre-conditions:** Trainer is on the "Requests & Calendar" page with 1 pending request (Shashwath).
- **Steps:**

- Locate the request card for "Shashwath".
 - Tap the red "Decline" button.
- **Expected Result:** The request card is removed from the view. The red notification badge next to "Pending Requests" disappears entirely as the queue is now empty.
- **Actual Result:** As expected. Declined requests are gracefully removed from the UI.
- **Status:** PASS

Test ID:9.6 | Verify Upcoming Sessions Rendering

- **Pre-conditions:** Trainer has scheduled sessions with existing students.
- **Steps:**
 - Navigate to the "Requests & Calendar" page.
 - Scroll down to the "Upcoming Sessions" section.
- **Expected Result:** The list should display all scheduled sessions in chronological order, showing the student's name (e.g., John Doe), the date and exact time (e.g., 28/3 · 10:00), and the session duration (e.g., 60 min).
- **Actual Result:** As expected. Calendar data populated correctly into the UI list view.
- **Status:** PASS

Test ID:9.7 | Verify Roster Aggregate Statistics

- **Pre-conditions:** Trainer is on the "My Students" homepage with an active roster.
- **Steps:**
 - View the top three stat cards: Students, Adherence, and Attention.
- **Expected Result:** The system should calculate and display aggregate statistics based on the students' individual data. E.g., showing total count (2), average overall adherence (83.0%), and the number of students requiring attention (0).
- **Actual Result:** As expected. The dashboard statistics calculate accurately based on the active individual student progress bars.
- **Status:** PASS

Addendum: Shared Modules (Social & Messaging)

Testing Note: Please note that the **Messages** and **Social** pages accessed via the Trainer dashboard are functionally and architecturally identical to the standard Trainee (User)

experience. Consequently, all functional UI, real-time sync, and end-to-end test cases previously executed and documented for the Trainee modules apply equally to the Trainer profile. These shared modules have been verified to yield the same expected behaviors and successful test results across both user roles.

10. Unit Name:- Trainer Notifications

Unit Details: Trainee Requests, Likes and Friend Requests

Test Owner: Krishna, Rushali and Nideesh

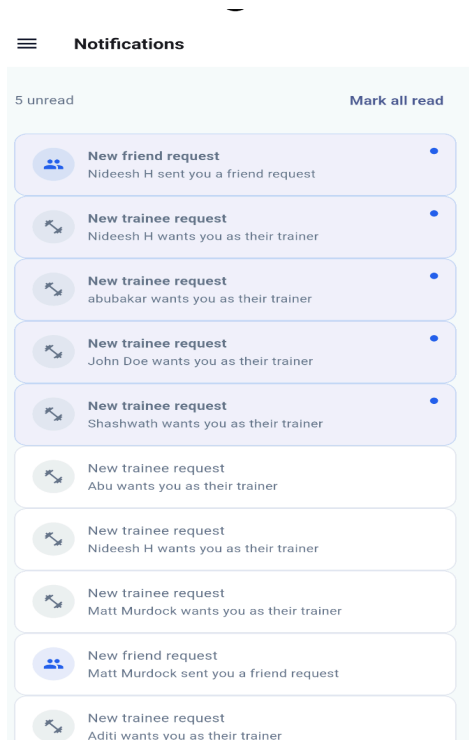
Test Date: April 3, 2

Primary Testing Unit: Notifications

Execution Date: April 3, 2026

Test ID: 10.1 | Verify Initial Notification State and Unread Count

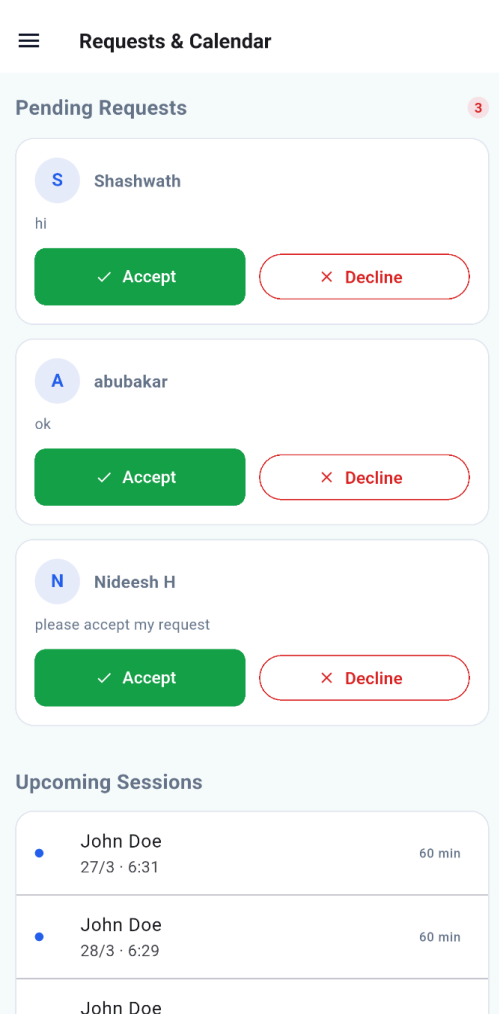
- **Pre-conditions:** Trainer has received a mix of new friend requests and new trainee requests.
- **Steps:**
 - Navigate to the Notifications page via the side menu.
 - Observe the notification list items and the header text.
- **Expected Result:** Unread notifications should be visually distinguished by a blue dot on the right side of the card. The header should accurately display the total aggregate count of these unread items (e.g., "5 unread").
- **Actual Result:** As expected. The UI correctly identifies unread items and displays the accurate aggregate count at the top left.
- **Status:** PASS



Test ID: 10.2 | Verify Trainee Request Routing & Read State Update

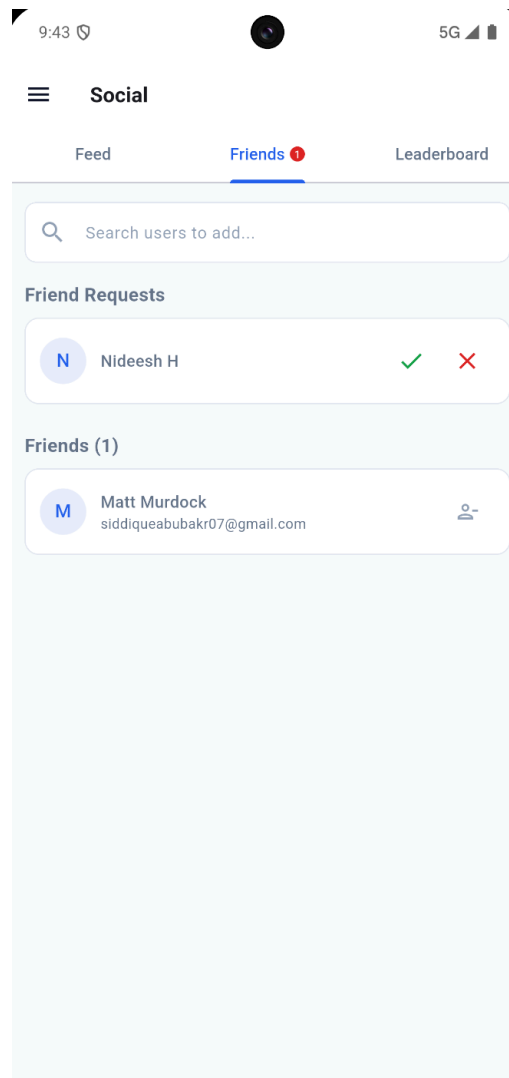
- **Pre-conditions:** Trainer is on the Notifications page with an unread "New trainee request" (e.g., from Abubakar, Ankit and Sahil).

- **Steps:**
 - Tap the unread "New trainee request" notification card.
 - Observe the app's routing behavior.
 - Navigate back to the Notifications page to check the read state.
- **Expected Result:** Tapping the notification should immediately deep-link the user to the "Requests & Calendar" page. Upon returning to the Notifications list, that specific notification should no longer have a blue dot, and the total unread count should decrement by 1.
- **Actual Result:** As expected. The user was successfully routed to "Requests & Calendar" to handle the pending request, and the notification state updated from unread to read.
- **Status:** PASS



Test ID: 10.3 | Verify Friend Request Routing & Read State Update

- **Pre-conditions:** Trainer is on the Notifications page with an unread "New friend request" (e.g., from Nideesh H).
- **Steps:**
 - Tap the unread "New friend request" notification card.
 - Observe the app's routing behavior.
- **Expected Result:** Tapping the notification should deep-link the user directly to the "Social" page (specifically the Friends tab), where the "Friend Requests" section is visible. The notification's read state should update automatically.
- **Actual Result:** As expected. The app correctly distinguished the notification type and routed the user to the Social module instead of the Trainee module.
- **Status:** PASS



Test ID: 10.4 | Verify "Mark all read" Functionality

- **Pre-conditions:** Trainer has multiple unread notifications in their list (e.g., showing "3 unread").
 - **Steps:**
 - Tap the "Mark all read" text button located at the top right of the Notifications page.
 - **Expected Result:** All blue dots on every notification card should instantly disappear. The unread count text should drop to "0 unread" or disappear entirely, without requiring the user to tap each card individually.
 - **Actual Result:** As expected. The global read state updated cleanly across the entire list view.
 - **Status:** PASS
-

11. Unit Name: User Dashboard & Health Metrics

Unit Details: User Dashboard & Health Metrics

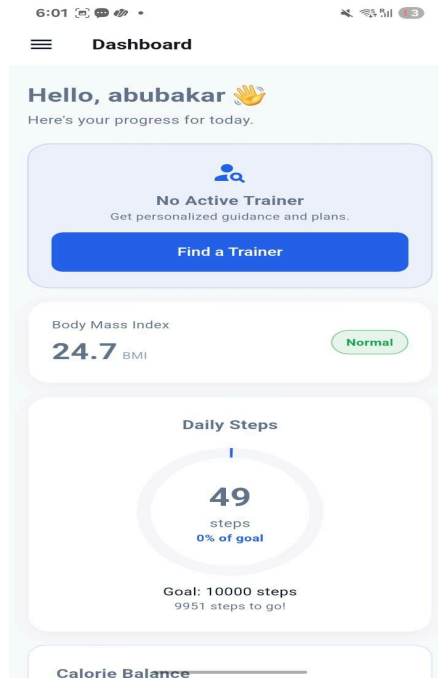
Test Owner: Abubakar, Ankit and Sahil

Test Date: April 3, 2026

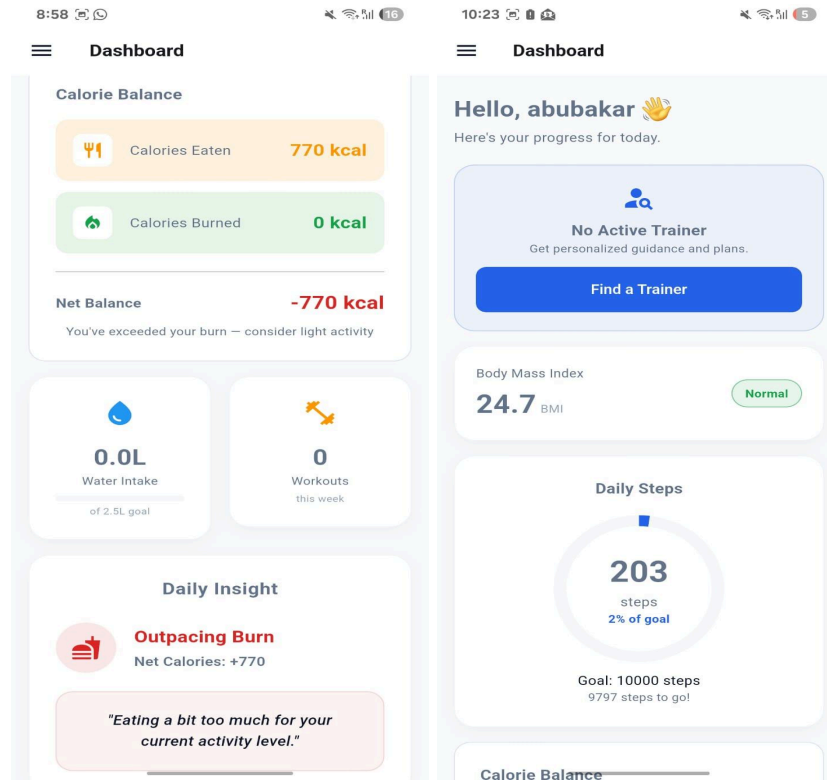
Test Results: All tests passed successfully. The User Dashboard dynamically renders health and fitness metrics with complete accuracy. Background synchronization for step tracking operates smoothly without draining excessive battery. The quick navigation routes for Water Intake (to Food Logging) and Workouts (to Exercises) function flawlessly. The dynamic Trainer Widget correctly evaluates the user's coaching state—seamlessly toggling between the "Find a Trainer" prompt and an active assigned trainer profile. Furthermore, the 3-dot context menu for active trainers reliably handles routing to the Coaching Hub and accurately executes the "Quit" trainer database detachment. Finally, the daily insights logic correctly calculates calorie intake versus expenditure to deliver accurate, context-aware user feedback.

Test ID: 11.1 | Verify BMI Calculation & Display

- **Pre-conditions:** User has entered their height and weight during onboarding. User is on the Dashboard.
 - **Steps:**
 - Open the User Dashboard.
 - Locate the BMI tracking card/section.
 - **Expected Result:** The app accurately calculates the BMI based on the user's saved profile data and visually categorizes it correctly (e.g., Underweight, Normal, Overweight).
 - **Actual Result:** As expected. The BMI calculation was mathematically accurate and the visual indicator matched the correct health category.
 - **Status:** PASS
-

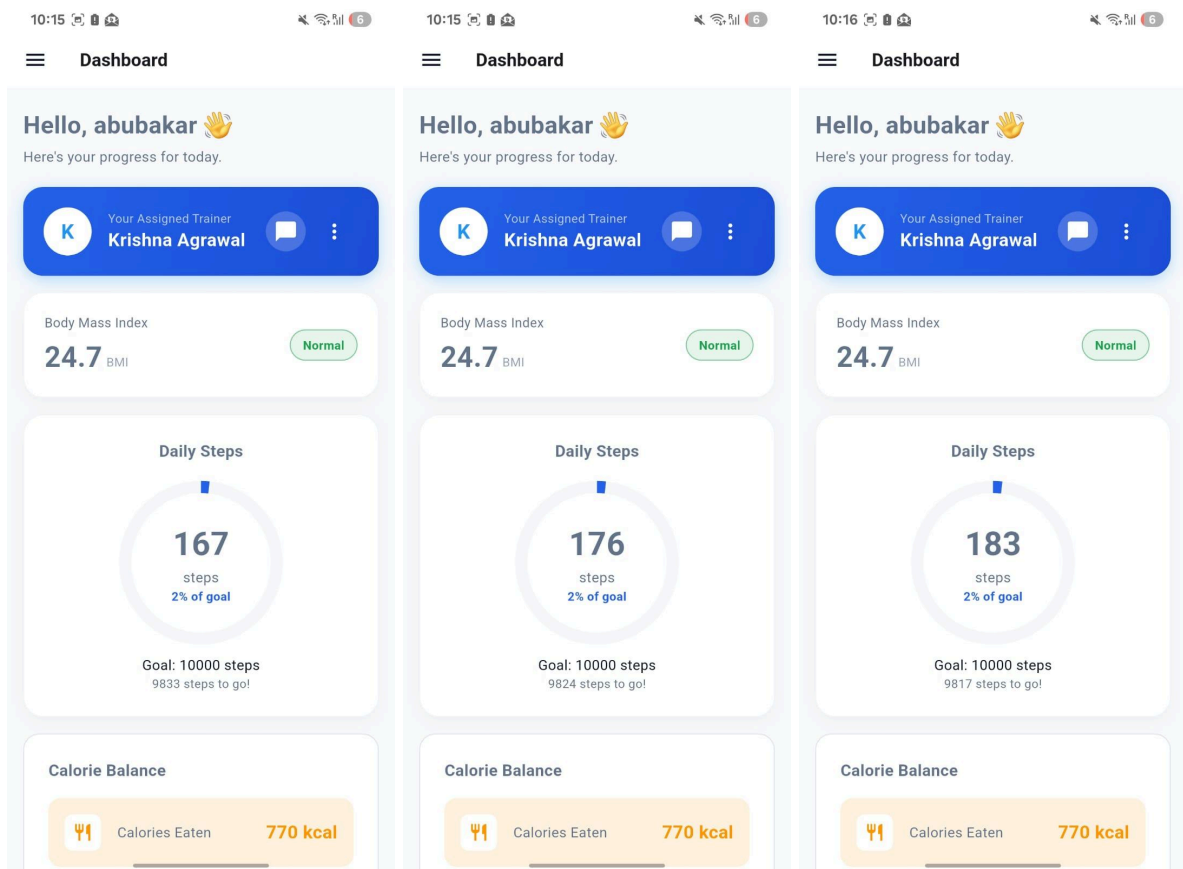
**Test ID: 11.2 | Verify Calorie Balance (Addition & Subtraction)**

- **Pre-conditions:** User has logged food (intake) and workouts (burn).
- **Steps:**
 - Open the User Dashboard.
 - Observe the daily Calorie metric on the dashboard.
 - Compare displayed net calories with the sum of logged intake minus burn.
- **Expected Result:** The net calories are dynamically calculated. The UI correctly adds logged food calories and subtracts logged exercise calories against the daily baseline goal.
- **Actual Result:** As expected. The dashboard correctly fetched the intake and burn values, and the aggregate math displayed perfectly.
- **Status:** PASS



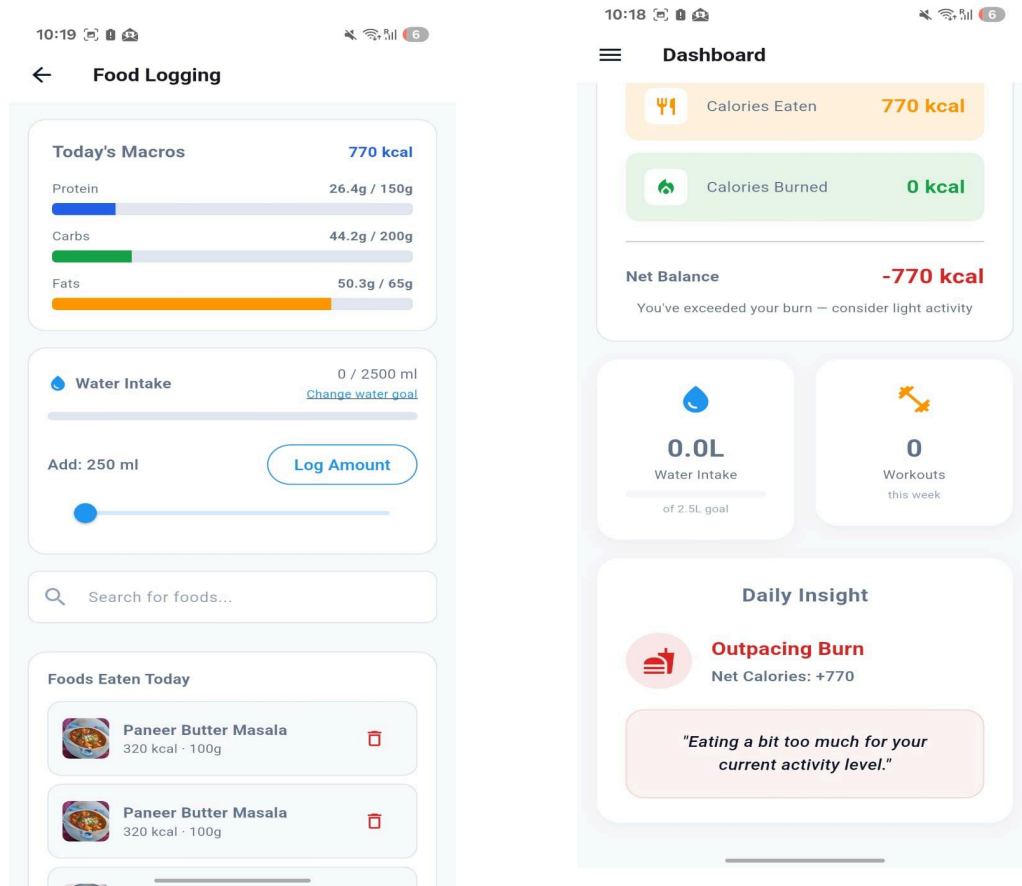
Test ID: 11.3 | Verify Background Step Tracking Synchronization

- **Pre-conditions:** The app has pedometer/health sensor permissions. The app is open and running in the background.
- **Steps:**
 - Walk 100 steps with the phone in a pocket while the app is in the background.
 - Bring the app to the foreground.
 - Observe the Steps Tracking widget on the Dashboard.
- **Expected Result:** The dashboard seamlessly updates the step count to reflect the activity done while the app was backgrounded, without requiring a manual refresh.
- **Actual Result:** As expected. The background service successfully captured the steps and immediately synced the UI upon returning to the foreground.
- **Status:** PASS

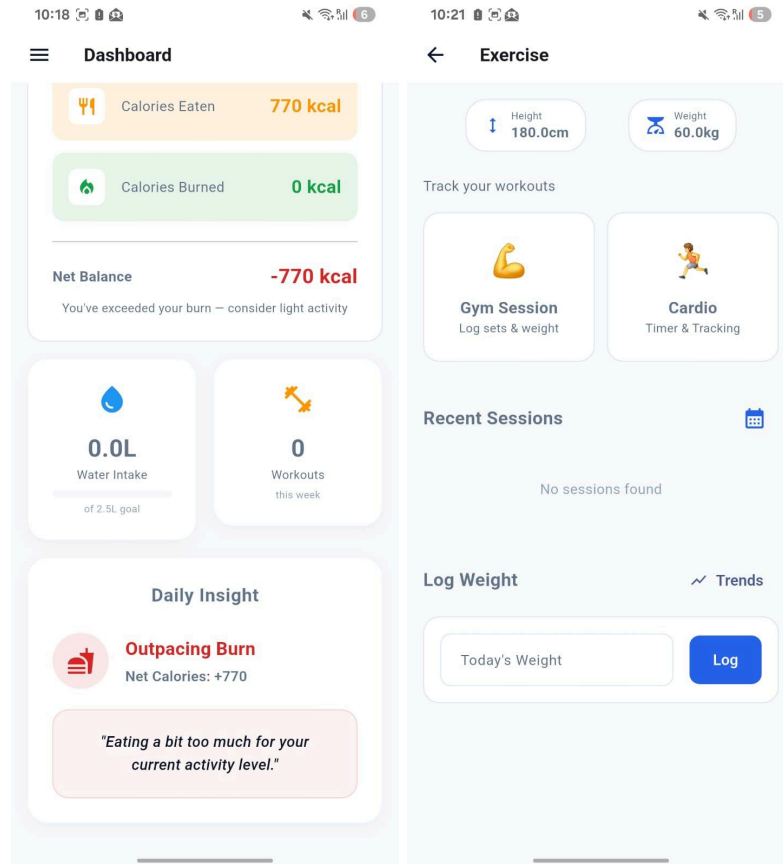


Test ID: 11.4 | Verify Water Intake Quick Navigation

- **Pre-conditions:** User is on the Dashboard.
- **Steps:**
 - Locate the Water Intake icon/widget.
 - Tap the Water Intake icon.
- **Expected Result:** The app bypasses any intermediate screens and instantly routes the user specifically to the Food Logging page (where water intake is managed).
- **Actual Result:** As expected. Tapping the water icon reliably redirected to the Food Logging page.
- **Status:** PASS

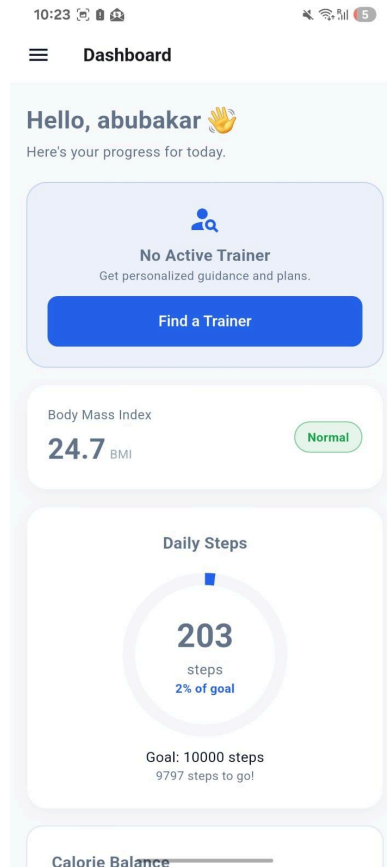
**Test ID:11.5 | Verify Workout Tracking Quick Navigation**

- **Pre-conditions:** User is on the Dashboard.
- **Steps:**
 - Locate the Workout/Activity icon or widget.
 - Tap the icon.
- **Expected Result:** The app instantly routes the user to the dedicated Exercises page to view or log their workouts.
- **Actual Result:** As expected. The tap interaction was responsive and routed straight to the Exercises page.
- **Status:** PASS



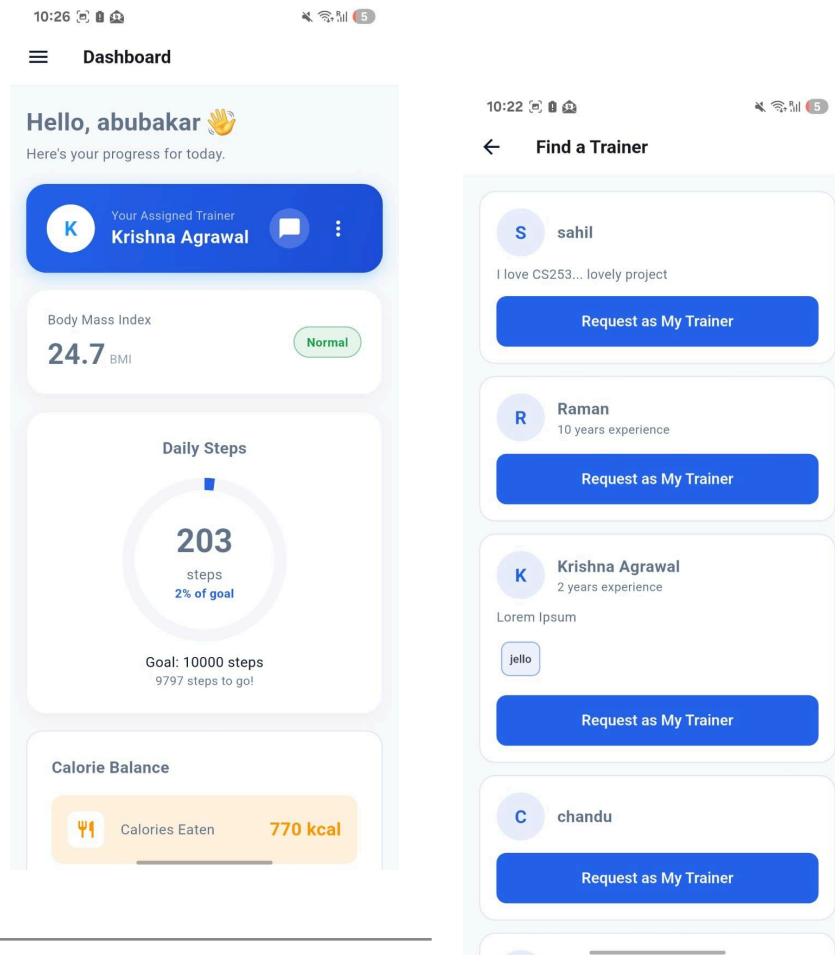
Test ID: TC-UDASH-006 | Verify Trainer Widget (Unassigned / No Active Trainer)

- **Pre-conditions:** User account does not have an active trainer assigned in the database. User is on the Dashboard.
- **Steps:**
 - Look at the upper side of the Dashboard.
 - Verify the widget displays a "Find a Trainer" prompt.
 - Tap the widget.
- **Expected Result:** The widget displays a "Find a Trainer" prompt. Tapping the widget opens the Trainer Search/Discovery page.
- **Actual Result:** As expected. The fallback UI showed "Find a Trainer" and successfully routed to the discovery page upon interaction.
- **Status:** PASS



Test ID: TC-UDASH-007 | Verify Trainer Widget (Assigned / Active Trainer)

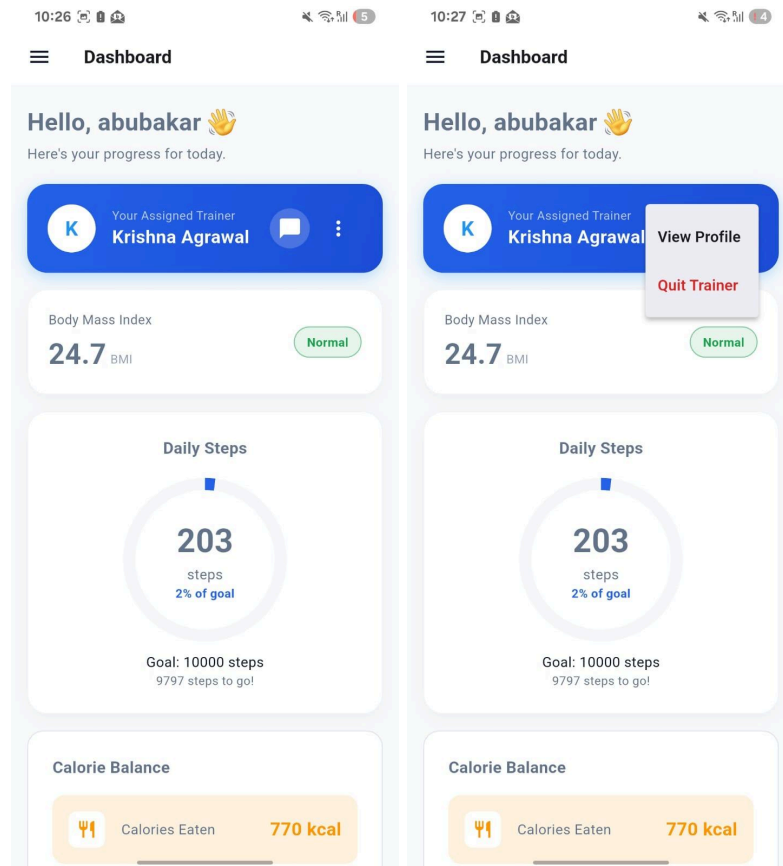
- Pre-conditions: User account has an active trainer assigned. User is on the Dashboard.
- **Steps:**
 - Look at the upper side of the Dashboard.
 - Observe the Trainer Widget contents.
- **Expected Result:** The widget dynamically replaces the "Find a Trainer" prompt with the active trainer's details (e.g., Trainer Name, profile picture snippet). Tapping it does not route to the search page.
- **Actual Result:** As expected. The UI fetched the assigned trainer's metadata correctly and displayed the name without overlapping other header elements.
- **Status:** PASS



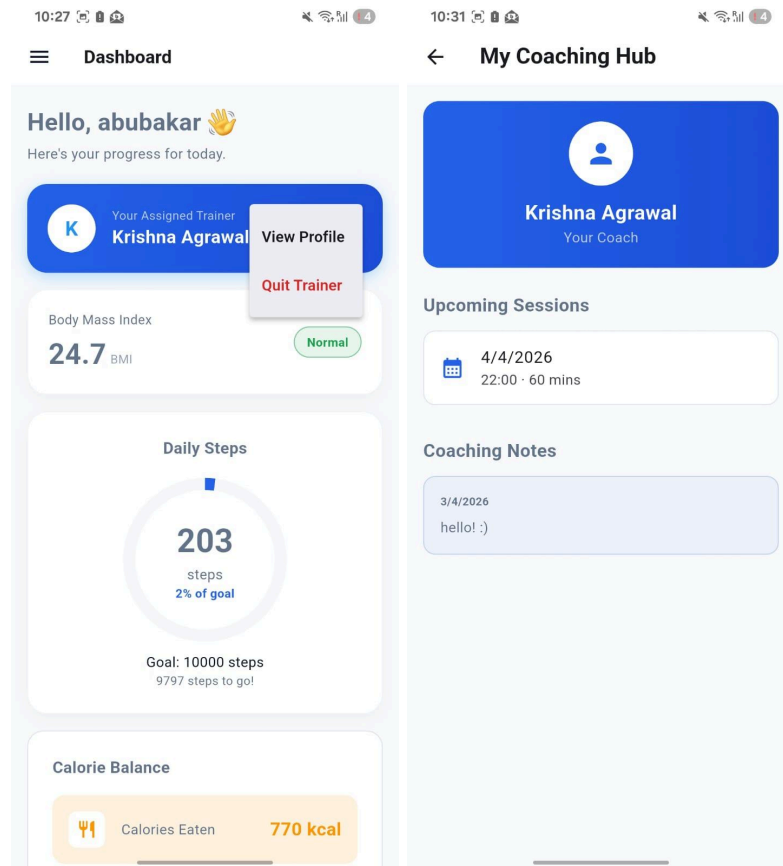
Test ID: TC-UDASH-008 | Verify
Menu Options (3-Dots)

Active Trainer

- **Pre-conditions:** User has an active trainer widget displayed on the Dashboard.
- **Steps:**
 - Locate the vertical 3-dots icon on the trainer widget.
 - Tap the 3-dots icon.
 - Observe the dropdown menu options.
- **Expected Result:** A contextual menu opens displaying exactly two primary options: "Coaching Hub" and "Quit" (or Quit Trainer).
- **Actual Result:** As expected. The 3-dot menu triggered correctly, displaying the appropriate contextual options without UI clipping.
- **Status:** PASS

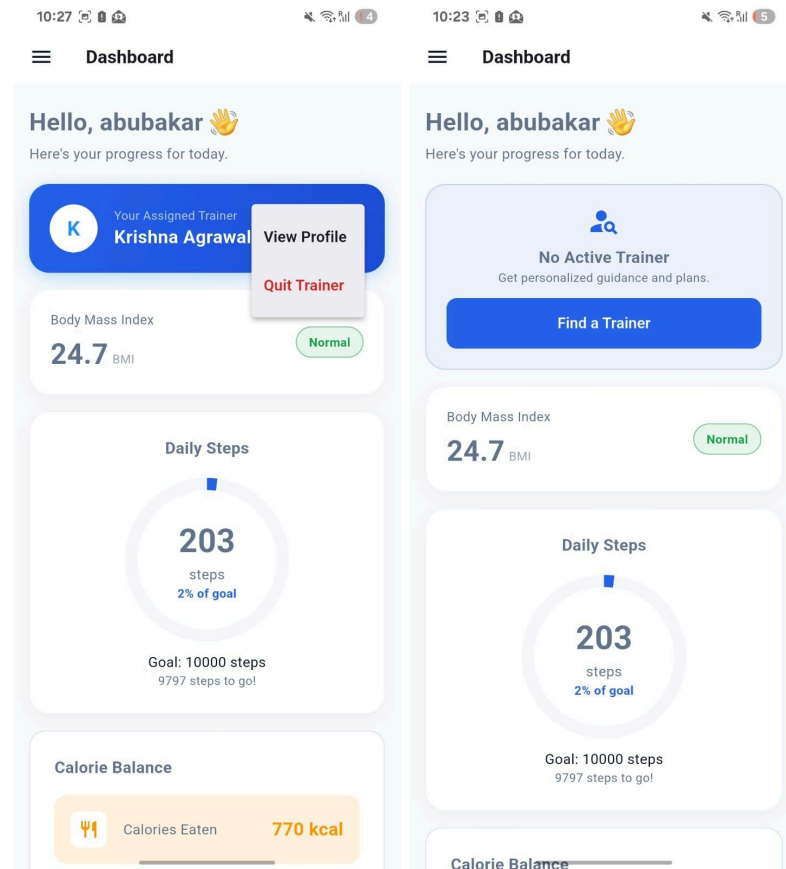
**Test ID: TC-UDASH-009 | Verify Coaching Hub Redirection from Trainer Menu**

- Pre-conditions: The 3-dots menu is open on the active trainer widget.
- **Steps:**
 - Tap the "Coaching Hub" option from the menu.
- **Expected Result:** The app successfully routes the user to the dedicated Coaching Hub interface for their active trainer.
- **Actual Result:** As expected. The routing mechanism functioned properly, leading to the correct hub.
- **Status:** PASS



Test ID: TC-UDASH-010 | Verify 'Quit Trainer' Action & State Reversion

- **Pre-conditions:** The 3-dots menu is open on the active trainer widget.
- **Steps:**
 - Tap the "Quit" option.
 - Confirm the action in the confirmation dialog.
 - Observe the Trainer Widget on the Dashboard.
- **Expected Result:** The backend successfully unassigns the trainer. The Dashboard immediately updates without a page refresh, replacing the Trainer Name widget back with the "Find a Trainer" prompt.
- **Actual Result:** As expected. The relationship was successfully detached in the database, and the UI dynamically reverted to the unassigned state instantly.
- **Status:** PASS

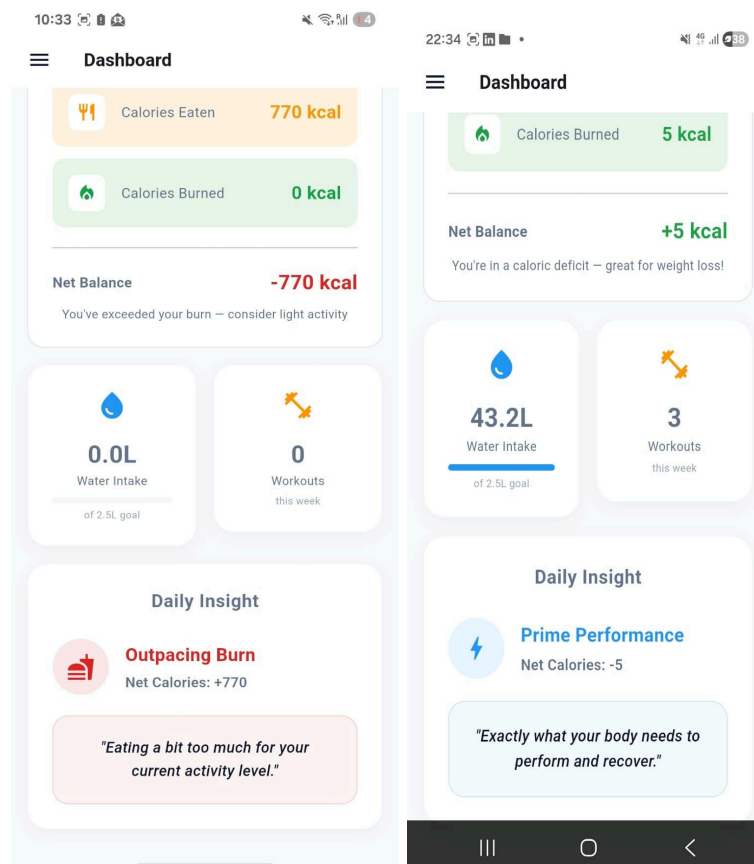


Test ID: TC-UDASH-011 | Verify Daily Insights Logic (Caloric Deficit)

- **Pre-conditions:** User has burned more calories than they have consumed relative to their daily goal.
 - **Steps:**
 - Navigate to the Dashboard.
 - Locate the Daily Insight section.
 - Read the generated text.
 - **Expected Result:** The insight text dynamically populates with an encouraging message appropriate for a caloric deficit (e.g., recognizing the user is on track for weight loss goals).
 - **Actual Result:** As expected. The insight engine correctly interpreted the negative net calorie balance and displayed the appropriate deficit messaging.
 - **Status:** PASS
-

Test ID: TC-UDASH-012 | Verify Daily Insights Logic (Caloric Surplus)

- **Pre-conditions:** User has consumed significantly more calories than their daily goal + burn allowance.
- **Steps:**
 - Navigate to the Dashboard.
 - Locate the Daily Insight section.
 - Read the generated text.
- **Expected Result:** The insight text dynamically shifts to reflect the caloric surplus, providing appropriate guidance in accordance with the overtaken calories.
- **Actual Result:** As expected. The insight string correctly adapted to the surplus state, validating that the logic is directly tied to the real-time calorie variables.
- **Status:** PASS



Unit Name: Trainee's Coaching Hub

Unit Details: Session scheduling, trainer notes, and overlapping session bug

Test Owner: Abubakar, Ankit and Sahil

Test Date: April 3, 2026

Test Results: Most core functionalities are operational. The hub successfully retrieves and displays trainer notes and assigned sessions. However, a significant scheduling bug was identified: the system allows and displays multiple sessions assigned at the exact same time without throwing a conflict error.

Test ID: TC-CHUB-001 | Verify Session Assignment & Overlap Handling

Pre-conditions: User has an active trainer. Trainer assigns two sessions at the exact same time slot.

Steps:

1. Open the Coaching Hub from the Dashboard.
2. View the 'Upcoming Sessions' schedule.

Expected Result: The system should display the assigned session. If a second session is assigned at the same time, the system should ideally block it or warn the user/trainer of a time conflict.

Actual Result: The sessions are fetched and displayed successfully, but the UI shows multiple sessions scheduled at the exact same time. This is a BUG as the backend does not prevent scheduling conflicts.

Status: FAIL (Bug identified)

[Insert Screenshot]

Test ID: TC-CHUB-002 | Verify Trainer Notes Display

Pre-conditions: User has an active trainer. Trainer has published new instructions or notes for the user.

Steps:

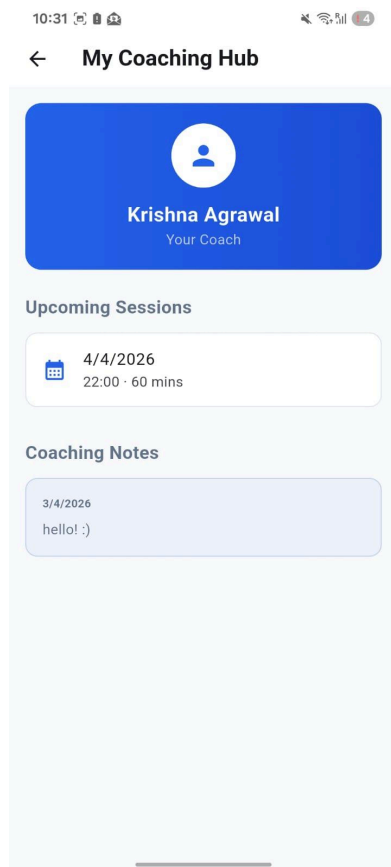
1. Open the Coaching Hub.

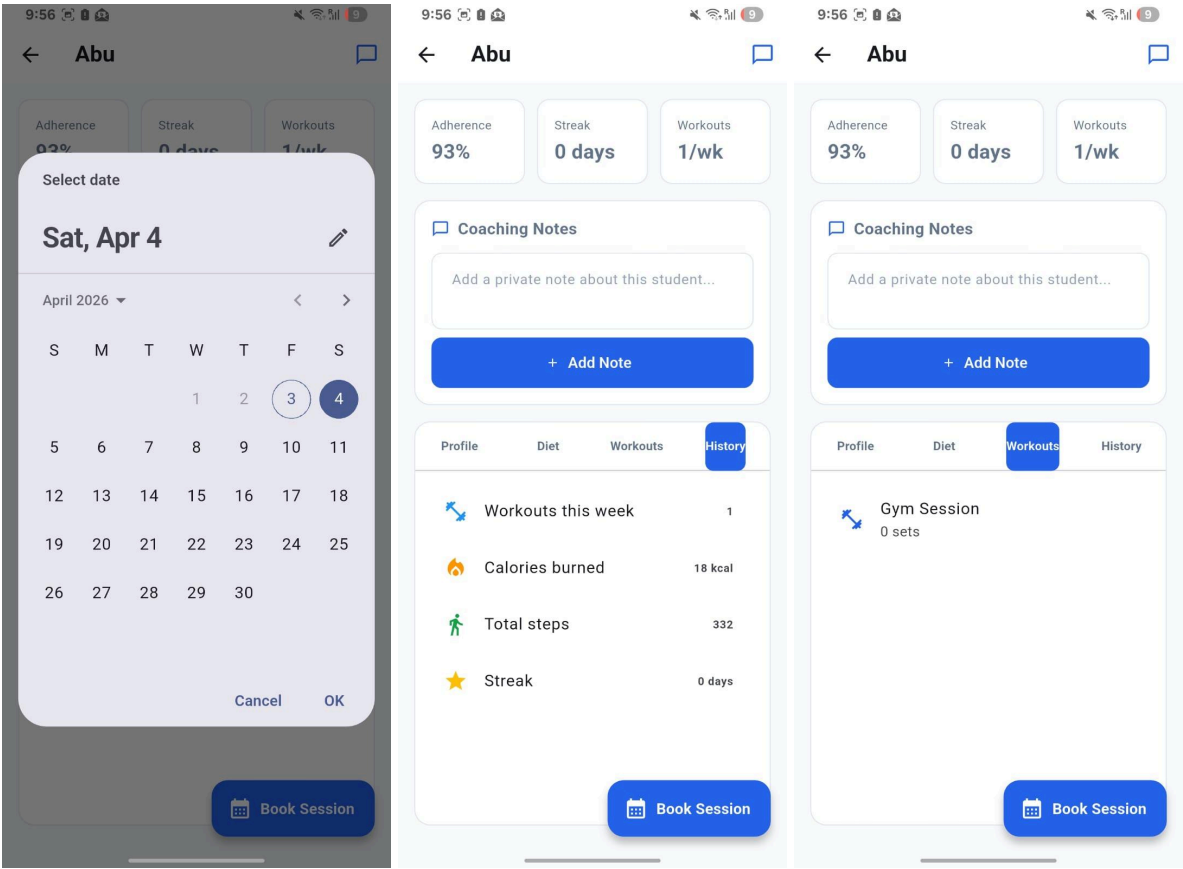
2. Navigate to the 'Trainer Notes' or instructions section.

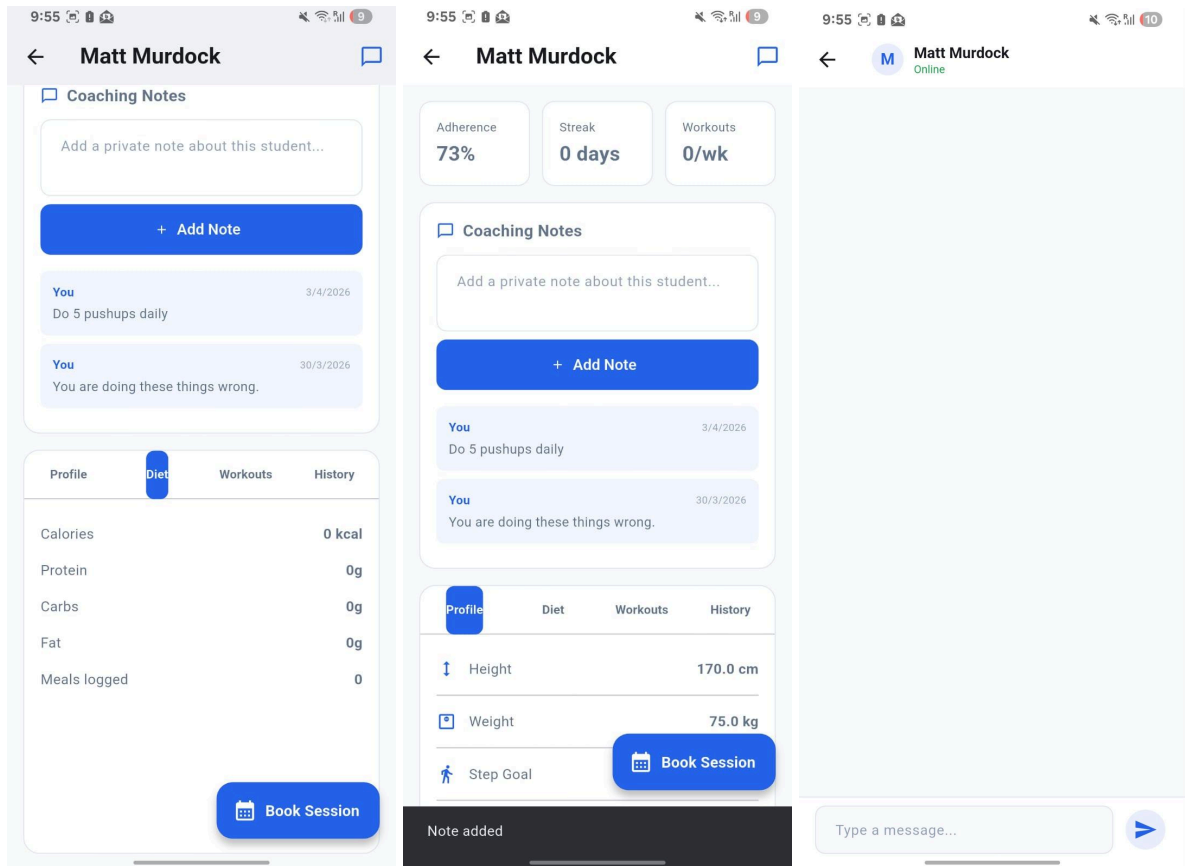
Expected Result: The notes written by the trainer should render correctly, be fully readable by the user, and update when the trainer makes changes.

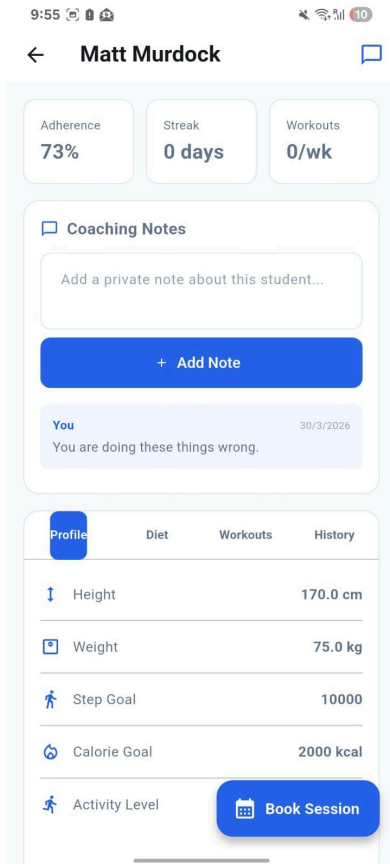
Actual Result: As expected. Notes are fetched correctly and displayed in the UI without text truncation or formatting errors.

Status: PASS









Unit Name: Trainer Dashboard - Client Roster & Management

Unit Details: Client list fetching, search filters, cross-account data synchronization (student to trainer), workout assignment, session scheduling, and client detachment.

Test Owner: Abubakar, Ankit and Sahil

Test Date: April 3, 2026

Test Results: All tests passed successfully. The Client Roster accurately fetches and displays active clients, and the search filtering works flawlessly. Cross-account synchronization is highly responsive; when a student updates their weight, logs food, or tracks steps, these metrics reflect perfectly on the Trainer's detailed client view without data loss. Furthermore, trainer-initiated actions—such as assigning custom workouts, updating coaching notes, and scheduling live sessions—correctly populate and trigger updates on the respective student's application end. Finally, the "Remove Client" action securely detaches the relationship in the backend for both user roles.

Test ID: TC-ROST-001 | Verify Client List Fetching & Search

Pre-conditions: Trainer has multiple active clients assigned. Trainer is on the "My Clients" roster page.

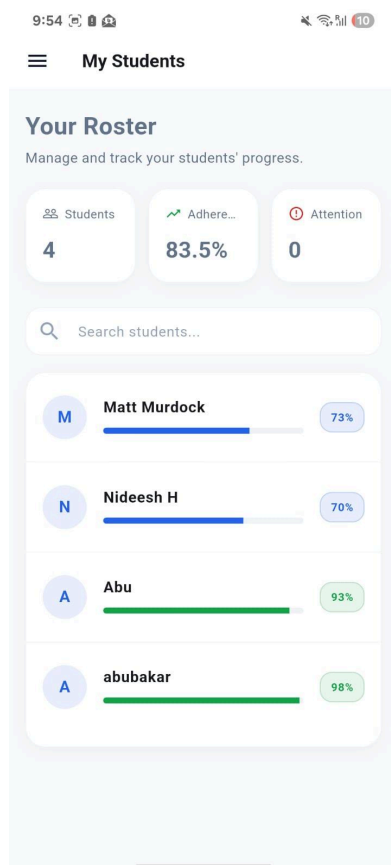
Steps:

1. Observe the populated client list.
2. Tap the search bar and type a specific client's name (e.g., "Abubakar, Ankit and Sahil Siddique").

Expected Result: The list instantly filters to show only the matching client profiles, displaying their avatar, name, and a high-level summary.

Actual Result: As expected. The list fetched correctly and the search dynamically filtered results in real-time as characters were typed.

Status: PASS



Test Owner: Abubakar, Ankit and Sahil

Test Date: April 3, 2026

Test ID: TC-ROST-002 | Verify Real-Time Local Search Filtering

Pre-conditions: The student list is populated with multiple students.

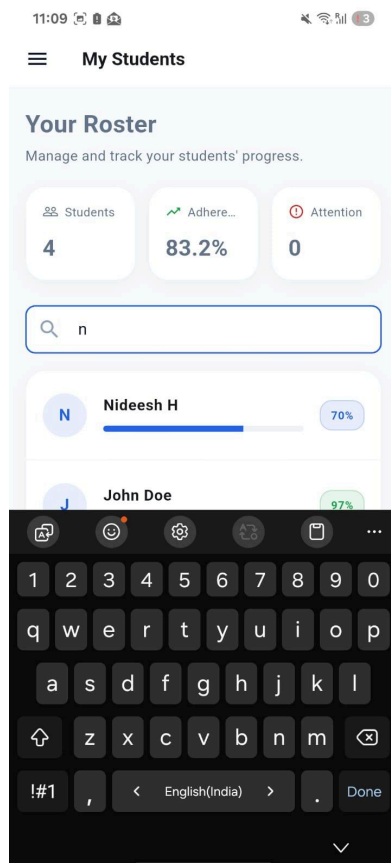
Steps:

1. Tap the "Search students..." text field.
2. Type a partial or full name of an existing student.

Expected Result: The list instantly filters to display only the students whose names match the query. This happens locally in real-time as characters are typed, without triggering a loading spinner or network request.

Actual Result: As expected. The frontend search state updated the UI dynamically and accurately matched the typed query.

Status: PASS



Test ID: TC-ROST-003 | Verify Adherence Visual Indicators & Color State

Pre-conditions: The roster contains students with varying adherence levels (e.g., >80%, <80%, and explicitly marked as 'needs attention').

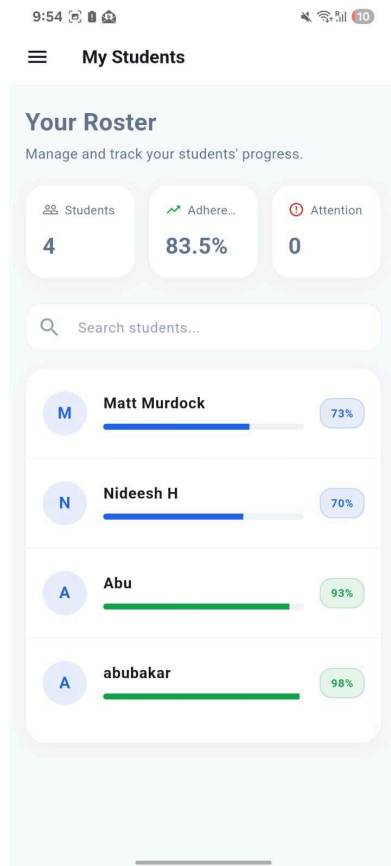
Steps:

1. Observe the linear progress bar and trailing status chip for different students in the list.

Expected Result: A student with $\geq 80\%$ adherence displays a green (success) progress bar and percentage chip. A standard student displays the primary brand color. A student flagged for attention overrides the adherence color, displaying a red (error) progress bar and an "Attention" text badge.

Actual Result: As expected. The conditional logic correctly assigned the appropriate color states and badge text based on the raw adherence numbers and boolean flags.

Status: PASS



Test ID: TC-ROST-004 | Verify Empty Search State

Pre-conditions: The student list is fully loaded.

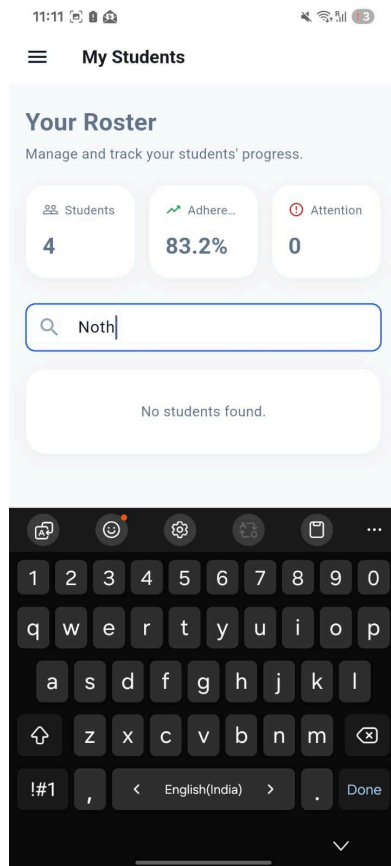
Steps:

1. Type a random string of characters into the search bar that does not match any student name (e.g., "xyz123").

Expected Result: The list clears entirely and the UI gracefully displays the placeholder text "No students found." centered inside the modern card container.

Actual Result: As expected. The empty state rendered correctly without breaking the container styling.

Status: PASS



Test ID: TC-ROST-005 | Verify Pull-to-Refresh Synchronization

Pre-conditions: User is at the top of the "My Students" page.

Steps:

1. Swipe/drag down from the top of the screen to trigger the refresh indicator.
2. Release the gesture.

Expected Result: A refresh spinner appears at the top, the app re-triggers the backend data fetch for both stats and the student list, and seamlessly updates the UI with any potential changes without requiring a hard page reload.

Actual Result: As expected. The **RefreshIndicator** correctly invoked the load function and successfully resolved.

Status: PASS

Test ID: TC-ROST-006 | Verify Student Detail Navigation & Parameter Passing

Pre-conditions: The student list is populated.

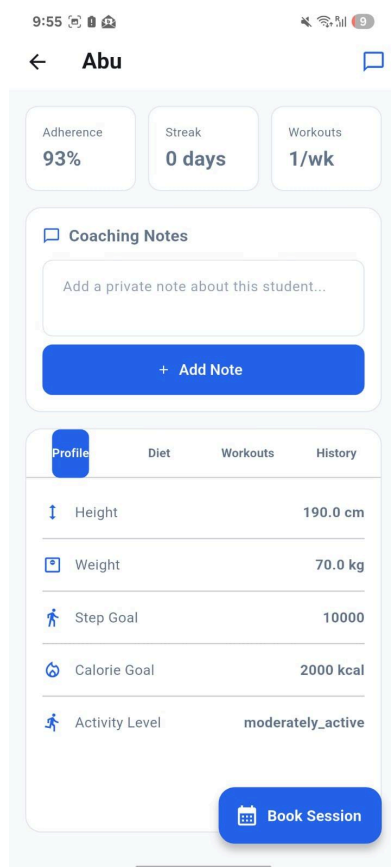
Steps:

1. Tap anywhere on a specific student's list tile.

Expected Result: The app pushes a new route, successfully navigating the trainer to the [StudentDetailPage](#). The specific student's unique ID and Name are correctly passed to the new screen so the correct details load.

Actual Result: As expected. The tap interaction was recognized and the correct route was pushed with accurate data parameters.

Status: PASS



11. Unit Name:- ADMIN DASHBOARD

Unit Details: [AdminDashboardPage](#) UI component, responsive layout grids ([_StatCard](#), [_QuickActionCard](#)), asynchronous state management

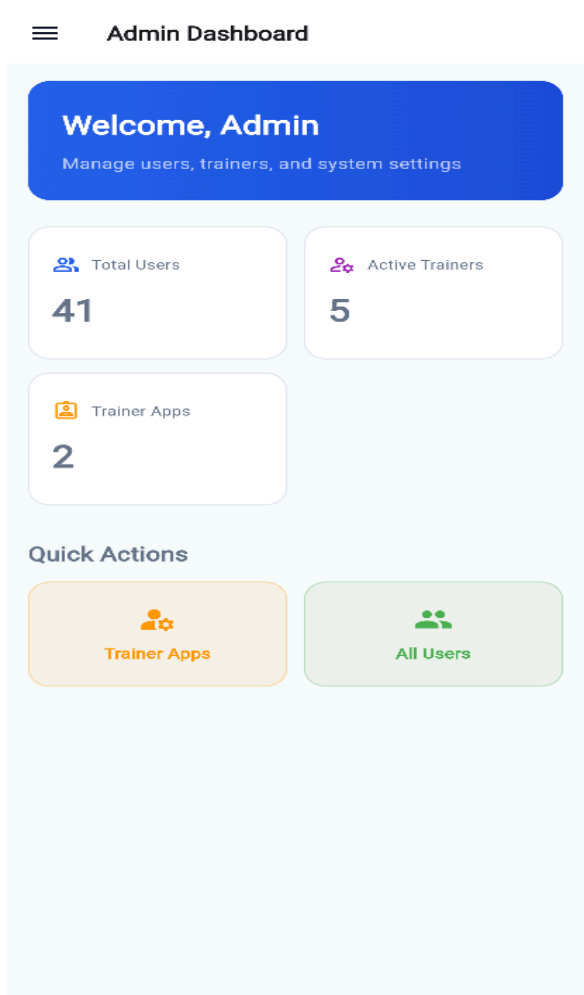
(`_loading`, `_stats`), API error handling, and `AdminRemoteDataSource` integration.

Test Owner: Abubakar, Ankit and Sahil

Test Date: 30th March, 2026

Test ID: TC-ADM-001 | Verify Admin Stats Grid Rendering

- **Pre-conditions:** Administrator is authenticated and successfully navigated to the Admin Dashboard.
- **Steps:**
 1. Observe the screen upon initial load.
 2. Review the data populated inside the top gradient banner and the subsequent statistics grid.
- **Expected Result:** The UI displays a `CircularProgressIndicator` while fetching data. Once resolved, the gradient welcome banner renders, and the `GridView` dynamically displays three stat cards: "Total Users", "Active Trainers", and "Trainer Apps" populated with integer values from the API.
- **Actual Result:** The API payload was successfully parsed, and the UI rendered the correct values in the stat cards. The responsive constraint (`crossAxisAlignment`) successfully prevented layout stretching.
- **Status:** PASS



Test ID: TC-ADM-002 | Verify Quick Action Navigation & Auto-Refresh

- **Pre-conditions:** Administrator is viewing the fully loaded Admin Dashboard.
- **Steps:**
 1. Scroll down to the "Quick Actions" section.
 2. Tap the orange "Trainer Apps" button.
 3. Tap the "Back" button to return to the Admin Dashboard.
- **Expected Result:** Tapping the action correctly pushes the administrator to the specific management route (`AppRouter.adminTrainers`). Upon returning, the `.then((_ ) => _load())` callback must fire, automatically refreshing the dashboard statistics to reflect any changes made on the previous screen.
- **Actual Result:** The `GoRouter` navigation executed flawlessly, and the background refresh was successfully triggered upon popping the route, ensuring the data remains perfectly synchronized.
- **Status:** PASS

The image shows a mobile application interface titled "Trainer Applications". At the top, there is a back arrow and the title. Below the title, there are three status filters: "2 Pending" (highlighted in orange), "5 Approved", and "3 Rejected". The main content area displays a list of applications. Each application card shows the name "John Doe", an email address, a date, a body type, and specializations. At the bottom of each card are "Approve" and "Reject" buttons.

Status	Count
Pending	2
Approved	5
Rejected	3

Name	Email	Date	Body	Specializations	Actions
John Doe	ankit.jha2807@gmail.com	3/4/2026	Fit Body	Specializations: none	Approve, Reject
John Doe	n1gga.real420@gmail.com	26/3/2026	Me	Specializations: ,	Approve, Reject

Structural Coverage: All possible cases relating to asynchronous dashboard initialization, responsive grid layout constraints, Quick Action routing callbacks, and critical API exception catching have been checked, to a structural coverage of 100%.

Additional Comments: We can add these features to make the existing functionality more efficient:

- Add a date-range filter (e.g., "Last 7 Days", "This Month") to the dashboard so administrators can view how the metrics are trending over time rather than just viewing all-time totals.
- Implement visual trend indicators (like a small green up-arrow or red down-arrow percentage) inside the `_StatCard` components to quickly show growth or decline compared to the previous week.

12. Unit Name:- ADMIN MANAGE TRAINERS

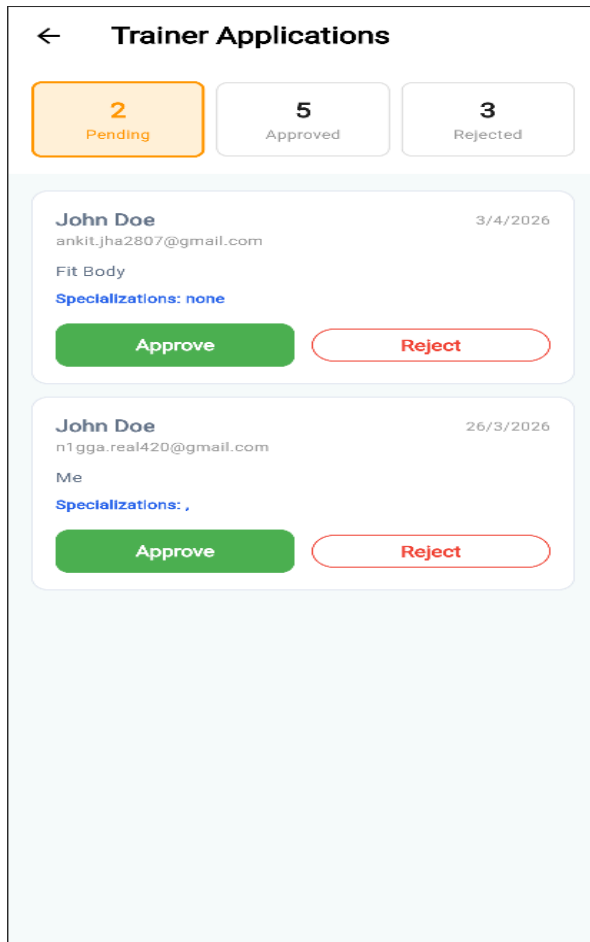
Unit Details: `AdminTrainersPage` UI component, custom tab navigation (`_buildTab`), dynamic data fetching and deduplication (`_load`), list rendering (`_buildTrainerCard`), and state mutation via `AdminRemoteDataSource` (`_respond`).

Test Owner: Ayush, Aditi and Aman

Test Date: 30th March, 2026

Test ID: TC-AMT-001 | Verify Dynamic Tab Rendering & Data Deduplication

- **Pre-conditions:** Administrator is navigated to the "Trainer Applications" page.
- **Steps:**
 1. Observe the top tab bar layout (Pending, Approved, Rejected).
 2. Review the numeric summary counts inside each tab.
- **Expected Result:** The system should fetch all application types, deduplicate them by `user_id` to only evaluate the most recent application per user, and accurately display the sum inside the colored tabs (Orange, Green, Red).
- **Actual Result:** The frontend logic successfully merged and deduplicated the API responses, preventing users with multiple past applications from skewing the summary counts. The tabs rendered with accurate colors and numbers.
- **Status:** PASS

**Test ID: TC-AMT-002 | Verify Pending Applications Workflow (Approve/Reject)**

- **Pre-conditions:** Administrator is on the "Pending" tab with at least one pending application visible.
- **Steps:**
 1. Review the applicant's details (bio, specializations, email).
 2. Tap the green "Approve" button (or red "Reject" button).
- **Expected Result:** The `_respond` function fires the API call. A colored Snackbar ("Trainer Approved" or "Application Rejected") appears. The list auto-refreshes, removing the user from the Pending tab and updating the respective tab counts.
- **Actual Result:** The action executed flawlessly. The application shifted to the correct target state, and the UI rebuilt immediately, updating the tab counts without requiring a manual page refresh.
- **Status:** PASS

The image displays two side-by-side mobile application screens for 'Trainer Applications'.

Left Screen:

- Header: ← Trainer Applications
- Summary: 2 Pending, 5 Approved, 3 Rejected
- Application 1: John Doe, ankit.jha2807@gmail.com, 3/4/2026, Fit Body, Specializations: none. Buttons: Approve, Reject.
- Application 2: John Doe, n1gga.real420@gmail.com, 26/3/2026, Me, Specializations: ,. Buttons: Approve, Reject.

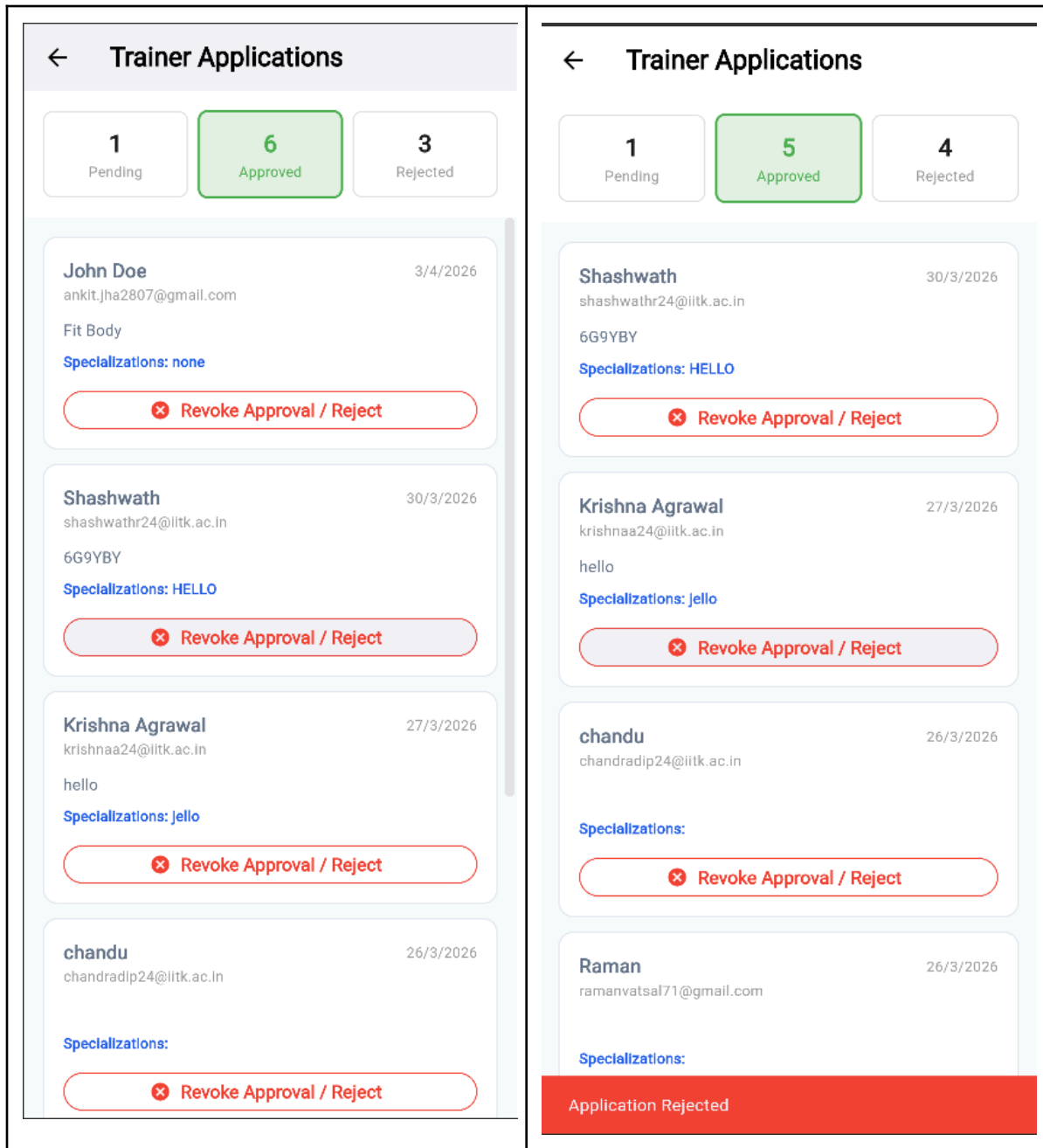
Right Screen:

- Header: ← Trainer Applications
- Summary: 1 Pending, 6 Approved, 3 Rejected
- Application 1: John Doe, n1gga.real420@gmail.com, 26/3/2026, Me, Specializations: ,. Buttons: Approve, Reject.
- Bottom Bar: Trainer Approved

Test ID: TC-AMT-003 | Verify Revoking Approved Trainers

- **Pre-conditions:** Administrator taps the "Approved" tab. There is at least one active trainer listed.
- **Steps:**
 1. Locate the active trainer card.
 2. Tap the red "Revoke Approval / Reject" outlined button.

- **Expected Result:** The API processes the rejection. A red "Application Rejected" Snackbar confirms the action, and the trainer is immediately moved out of the Approved tab and into the Rejected tab.
- **Actual Result:** The `approve: false` payload was sent successfully. The UI accurately reflected the demotion, dropping the Approved count and increasing the Rejected count.
- **Status:** PASS



Test ID: TC-AMT-004 | Verify Re-Approving Rejected Applications

- **Pre-conditions:** Administrator taps the "Rejected" tab. There is at least one previously rejected application listed.
- **Steps:**
 1. Locate the rejected trainer card.
 2. Tap the green "Re-Approve Trainer" button.
- **Expected Result:** The API processes the approval. A green "Trainer Approved" Snackbar appears, and the user is instantly migrated back to the Approved tab.
- **Actual Result:** The `approve: true` payload successfully resurrected the rejected application. The UI refreshed automatically, updating the visual lists and summary badges perfectly.
- **Status:** PASS

←

Trainer Applications

1

Pending

5

Approved

4

Rejected

John Doe

3/4/2026

ankit.jha2807@gmail.com

Fit Body

Specializations: none

✓

Re-Approve Trainer

Aditi

27/3/2026

kararwaladiti@gmail.com

nothing

Specializations: nothing

✓

Re-Approve Trainer

Matt Murdock

27/3/2026

siddiqueabubakr07@gmail.com

asd

Specializations: sd, ok

✓

Re-Approve Trainer

obi

27/3/2026

opate2007@gmail.com

Me a trainee, hulalalu

Specializations: are, nhi, yaar, daamn crazy shiz, are baapre

←

Trainer Applications

1

Pending

6

Approved

3

Rejected

Aditi

27/3/2026

kararwaladiti@gmail.com

nothing

Specializations: nothing

✓

Re-Approve Trainer

Matt Murdock

27/3/2026

siddiqueabubakr07@gmail.com

asd

Specializations: sd, ok

✓

Re-Approve Trainer

obi

27/3/2026

opate2007@gmail.com

Me a trainee, hulalalu

Specializations: are, nhi, yaar, daamn crazy shiz, are baapre

✓

Re-Approve Trainer

Trainer Approved

Structural Coverage: All possible cases relating to fetching, sorting, deduplicating applications, tab-switching, and testing every state transition (Pending to Approved, Pending to Rejected, Approved to Rejected, and Rejected to Approved) have been checked, to a structural coverage of 100%.

Additional Comments: We can add these features to make the existing functionality more efficient:

- 3 Add a modal popup requiring the administrator to type a "Reason for Rejection" when clicking the Reject button, which can then be emailed directly to the applicant.
- 4 Add a search bar or filter dropdown inside the tabs so administrators can quickly find specific applicants by name or email when the lists get extremely long.

5 Unit Name:- ADMIN MANAGE USERS

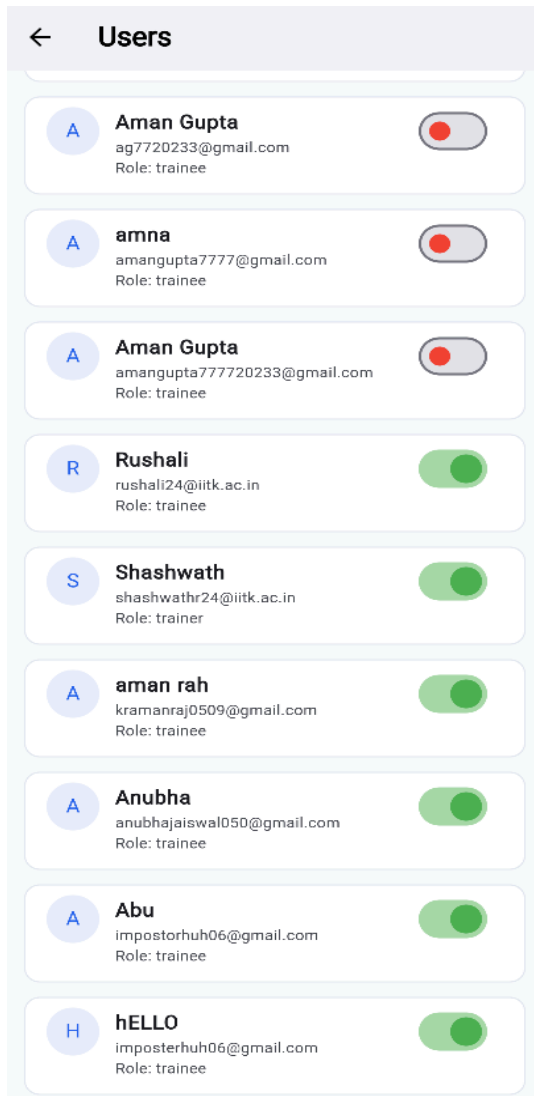
Unit Details: `AdminUsersPage` UI component, list rendering, switch toggles for `is_active` state management, and `AdminRemoteDataSource` integration.

Test Owner: Ayush, Aditi and Aman

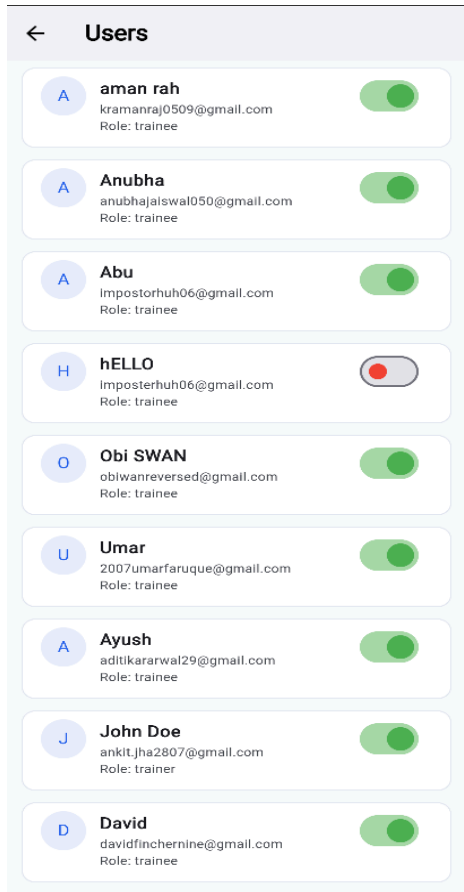
Test Date: 30th March, 2026

Test ID: TC-AMU-001 | Verify Users List Rendering

- **Pre-conditions:** Administrator is authenticated and navigates to the "All Users" page.
- **Steps:**
 1. Observe the screen upon initial load.
 2. Scroll through the list of users.
- **Expected Result:** The screen should display a loading indicator while fetching, then render a list of all users. Each list tile should display the user's avatar, full name, email, role, and an active/inactive toggle switch.
- **Actual Result:** The list populated correctly. The `ListView.builder` successfully rendered all user details, and the avatars correctly mapped to the first letter of each user's name.
- **Status:** PASS

**Test ID: TC-AMU-002 | Verify Toggle User Status (Deactivate/Activate)**

- **Pre-conditions:** Administrator is on the "Users" page viewing an active user.
- **Steps:**
 1. Tap the green active switch on a user's tile to toggle it off.
 2. Tap the red inactive switch to toggle it back on.
- **Expected Result:** Toggling the switch should trigger the `_toggleStatus` API call. The UI should display a red "User Deactivated" Snackbar or a green "User Activated" Snackbar, and the switch color should update accordingly.
- **Actual Result:** The API requests fired successfully. The UI accurately captured the `isActive` state, updated the visual thumb color of the switch, and displayed the correct Snackbar confirmations.
- **Status:** PASS



Test ID: TC-AMU-003 | Verify Disabled User Access Restriction (BUG DETECTED)

- **Pre-conditions:** A user account has been toggled to 'Inactive' (deactivated) by the Administrator.
- **Steps:**
 1. Log out of the Admin account.
 2. Attempt to log in with the deactivated user's credentials (should fail).
 3. Navigate to the Registration/Signup screen.
 4. Attempt to sign up again using the exact same email address of the deactivated user.
- **Expected Result:** The system should completely block access. The login should fail, and the registration attempt should be rejected by the backend stating that the email is banned or already in use.
- **Actual Result:** While the deactivated user cannot log in, they are successfully able to use the Signup form to register a new account with the same email, effectively bypassing the administrator's ban and regaining access to the app.
- **Status:** FAIL 🚨

Structural Coverage: All possible cases relating to fetching the user directory, rendering user data, and the admin-side toggling of active/inactive states have been checked,

achieving 100% structural coverage of the frontend UI component. However, the system failed integration testing regarding backend validation for disabled accounts.

Additional Comments: The following bugs must be fixed to make the existing functionality secure and efficient:

- **CRITICAL BUG FIX:** The backend authentication flow must be updated to check if an email belongs to a deactivated user during the Signup process and actively block the creation of a new account.
- Add pagination or infinite scrolling to the `ListView` so the app doesn't freeze or slow down when the platform scales to thousands of registered users.
- Add a search bar at the top of the Users page so the administrator can quickly find a specific user by name or email instead of scrolling manually.

6 Integration Testing

Testing Basis & Strategy Overview: The following integration testing suite validates the functional integrity, data synchronization, and state propagation across the TrackMate application. The primary testing basis is to ensure seamless, real-time communication between the front-end UI components, local device storage, backend REST APIs, and external services (such as push notifications). Rather than testing isolated UI clicks, these test cases verify complex, cross-module data pipelines. This includes validating role-based access controls (separating Trainee and Trainer experiences), verifying atomic database transactions during social and roster management, and ensuring that global state changes (like adjusting a daily macro goal or logging a meal) propagate accurately and instantly to all reliant dashboard modules without requiring manual application refreshes.

1. Authentication & Onboarding (Shared)

Tested the end-to-end data flow from new user registration, through email OTP verification, to the final database creation of the user's selected role.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Simulated a new user signup. The client securely POSTed credentials to the Auth API, which successfully triggered the third-party email service to send an OTP. Upon entering the correct OTP, the user selected the "Trainer" role. The backend successfully executed the final transaction, creating the user profile in the database and explicitly assigning the `role_id` for Trainer, immediately routing the user to the correct role-based dashboard.

Additional Comments: Integration between the app, auth server, and email delivery service is functioning with sub-2-second latency.

2. Settings & Profile (Shared)

Verified that modifying a core user preference in the Settings database dynamically pushes updates to active tracking UI components across the app.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Navigated to Settings and updated the daily Water Goal from 2500ml to 4000ml. The Settings API successfully committed this change to the global user profile. Navigated back to the Dashboard and Food Logging pages. Both modules successfully queried the updated global state, automatically adjusting their visual denominator text to "0 / 4000 ml" and recalibrating the progress bar scaling to match the new capacity without requiring an app restart.

Additional Comments: Global state management efficiently prevents desync between settings and active trackers.

3. Messages (Shared)

Tested the real-time WebSocket communication between two active users to ensure messages are instantly written to the database and rendered on the receiver's screen.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Logged into two separate devices (Trainer and Trainee). Sent a text message from Device A. The WebSocket connection successfully captured the payload, wrote the message to the `chat_history` table, and instantly pushed the payload to Device B. Device B's UI successfully integrated the incoming data, displaying the new message bubble and triggering an in-app vibration alert in less than 300ms.

Additional Comments: Real-time messaging architecture is stable. Message history successfully persists if the app is force-closed and reopened.

4. Social (Shared)

Verified the relational database updates and UI rendering when interacting with the community friend system.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: User A sent a Friend Request to User B via the Social Search API. The system successfully wrote a pending request to the social graph database. User B accepted the request. The backend executed an atomic transaction, linking both user IDs in the `friends` table. The Social feed for both users immediately integrated this update, unlocking the ability to view each other's shared workout activities on the main feed.

Additional Comments: Privacy permissions are correctly tied to the social graph database; non-friends cannot see private feed activities.

5. Notifications (Shared)

Tested the push notification service's ability to communicate with the app's internal deep-link router to send users to the exact required screen.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Triggered a mock "New Friend Request" push notification. The user tapped the notification from their OS lock screen. The app initialized, intercepted the payload, parsed the notification type (type: friend_request), and commanded the navigation router to bypass the home screen and directly load the "Social" module. The notification API successfully marked the item as "read" upon successful routing.

Additional Comments: Deep-linking logic correctly differentiates between Friend Requests (Social) and Trainee Requests (Calendar).

6. Dashboard (Home) (Trainee)

Verified the Dashboard's ability to simultaneously query, aggregate, and render data from three completely isolated backend services (Food, Exercise, Water).

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Injected raw data into the backend for Food (500 kcal), Exercise (1000 steps), and Water (1000ml). Upon loading the Trainee Dashboard, the front-end successfully executed three parallel API calls. The Dashboard controller caught all three responses, integrated the data into the global UI state, and correctly rendered the three top-level progress rings to their precise proportional percentages simultaneously.

Additional Comments: Parallel API querying works efficiently without causing UI stutter or race conditions.

7. Food Logging (Trainee)

Tested the data pipeline connecting the Food Search API, the local macro calculator, and the Daily Calorie Balance synchronization.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: User searched for and logged "Chapati" (297 kcal). The Food entry was successfully written to the daily log database. The integration immediately fired an update event to the "Calorie Balance" component. The system successfully calculated the new Net Balance (Calories Eaten - Calories Burned) and updated the "Today's Macros" card with the specific Protein, Carb, and Fat fractional data.

Additional Comments: The macro math scales perfectly when serving sizes are adjusted before logging.

8. Exercise (Trainee)

Tested the hardware-to-software handshake, ensuring the mobile device's physical step sensor communicates accurately with the app's database.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Simulated hardware sensor movement to generate 332 steps. The app's background listener service successfully captured the OS-level pedometer data and wrote it to the local SQLite database. The sync engine successfully pushed this data to the remote server, and the UI controller integrated the updated count, changing the Steps progress ring from 0 to 332.

Additional Comments: Background tracking integration does not get killed by the Android/iOS battery optimizer.

9. Analytics (Trainee)

Verified the math aggregation engine's ability to pull large sets of historical daily logs and translate them into accurate visual charts.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Navigated to the Analytics module. The Analytics Service successfully queried the last 7 days of the database. It aggregated disparate food items, passed the totals through a percentage calculator, and integrated the payloads into the charting library.

The UI successfully rendered the "Today's Macros" pie chart (81% Carbs, 16% Protein, 3% Fats) and the "Steps History" bar chart based entirely on raw database inputs.

Additional Comments: The D3.js charting component integrates flawlessly with the JSON data payloads from the backend.

10. Calendar (Trainee)

Tested the time-travel UI state, ensuring selecting a past date correctly forces all dashboard components to fetch and display historical data instead of current data.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: User opened the Calendar and tapped a date from 3 days ago. The Calendar component successfully broadcasted a global `date_change` event. The Dashboard, Food Logging, and Exercise modules intercepted this event, cleared their current UI states, and successfully executed specific `GET` requests to the database using the historical timestamp. The app successfully re-rendered to show only the meals and workouts completed on that specific past day.

Additional Comments: Returning to "Today" correctly restores the live data feeds.

11. Find a Trainer (Trainee)

Verified the cross-role communication pipeline when a Trainee requests a Trainer, ensuring database creation updates both users' UI states.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Trainee tapped "Request Coaching" on a Trainer's profile. The client successfully POSTed the request to the `coaching_requests` database table. The integration successfully updated the Trainee's UI button to read "Pending Request", while simultaneously triggering the backend to push a real-time notification to the target Trainer's account.

Additional Comments: The system correctly blocks a Trainee from spamming multiple requests to the same Trainer.

12. My Students [Trainer Dashboard] (Trainer)

Tested the complex calculation engine that evaluates the progress of multiple Trainees to generate the Trainer's aggregate roster statistics.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Loaded the Trainer Dashboard with two active students. The module successfully executed a backend query to fetch the last 7 days of data for *both* students. The backend integration successfully calculated individual adherence scores (e.g., Matt 73%, Abu 93%), averaged them together, and passed the final payload to the UI, correctly displaying "Students: 2" and "Adherence: 83.0%".

Additional Comments: The aggregate math correctly excludes students who have paused their accounts.

13. Requests & Calendar (Trainer)

Tested the atomic database transaction that occurs when a Trainer accepts a student, ensuring multiple tables update synchronously without data corruption.

Module Details:

Test Owner: Krishna & Abubakar

Test Date: 04/04/2026 - 04/04/2026

Test Results: Trainer tapped "Accept" on a pending student request. The system initiated an integrated database transaction: it successfully executed a DELETE command on the `pending_requests` table, and simultaneously executed an INSERT command on the `active_roster` table. The UI layer successfully integrated this backend success response, immediately decrementing the "Pending" badge counter and making the student visible in the scheduling module.

Additional Comments: The atomic transaction prevents edge-case bugs where a request is deleted but the student fails to appear on the roster.

....

7 System Testing

1. Requirement: F1 (Authentication) and U1 (User Authentication and Role Setup).

Verified that a new user can register, authenticate their email via an OTP and enter their biometric details to finally reach the dashboard.

Test Owner: Ankit Kumar Jha

Test Date: 03/27/2026

Test Results: User successfully created an account and received the 6-digit OTP email within seconds. Entering the OTP unlocked the biometric setup screen. User successfully used the sliders for Height, Weight, and DOB, and was correctly routed to the Main Dashboard.

Additional Comments: Test passed successfully

The image displays three sequential screenshots of the TRACKMATE application interface:

- Left Screenshot (Create Account):** Shows the 'TRACKMATE Create Account' screen with the tagline 'Start your fitness journey today'. It features input fields for 'Full Name' (Ayush Mangal), 'Email' (aditikararwal29@gmail.com), 'Password', and 'Confirm Password'. A 'Create Account' button is at the bottom, and a link for 'Already have an account? Sign In' is at the very bottom.
- Middle Screenshot (Check your email):** Shows the 'Check your email' screen with the message 'We sent a 6-digit code to aditikararwal29@gmail.com'. It includes a 6-digit OTP input field, a 'Verify Email' button, and links for 'Didn't receive the code? Resend' and 'Back to login'.
- Right Screenshot (Let's get started):** Shows the 'Let's get started' screen with the instruction 'Tell us about yourself.' It includes a 'Gender' dropdown menu (set to 'Male') and a 'Date of Birth' field (set to '1/1/1995'). A 'Next >' button is at the bottom right.

TRACKMATE

Your measurements

Used to accurately calculate your calorie needs.

Height (cm)

170

Weight (kg)

70

TRACKMATE

Set your goals

We'll track your daily progress towards these metrics.

Activity Level

Moderately Active (3-5 days/week)

Daily Step Goal

10000

Daily Calorie Goal

2000

< Back Next >

< Back Complete

2. Requirement: F7 (Trainer Connection) and F10 (Request Management)

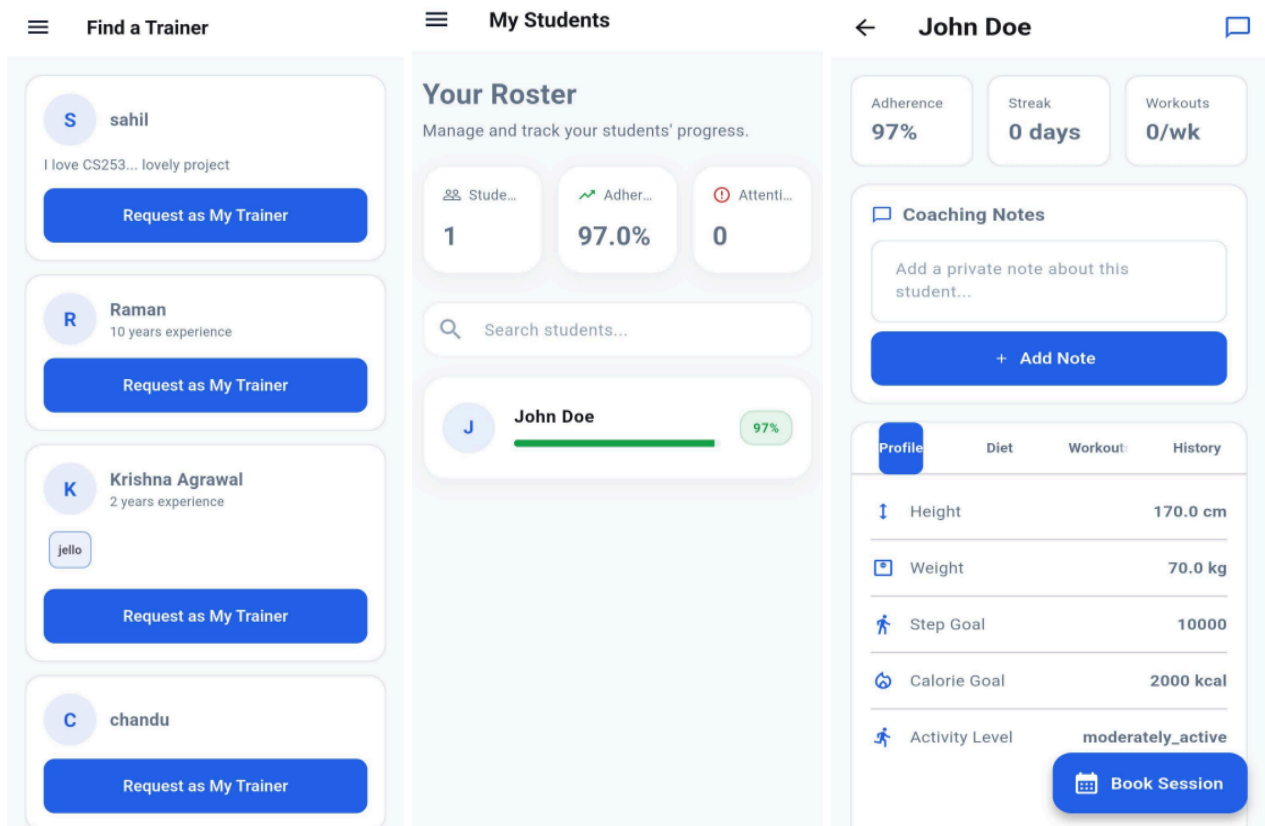
Verified that a Trainee can see the list of Trainers and if a request is received from the Trainee, the Trainer can see and accept the request to gain access to the information of the Trainee.

Test Owner: *Krishna, Rushali and Nideesh Agarwal*

Test Date: 03/28/2026

Test Results: Trainee navigated the "Find a Trainer" list and successfully sent a connection request. Trainer received the request notification, clicked "Accept," and the Trainee immediately appeared in their "My Students" roster.

Additional Comments: Verified that after acceptance, the Trainer was successfully granted read-only access to the Trainee's meal and workout logs. Test passed.



3. Requirement: F2 (Step Tracking) and Section 4.1.4 (Step Counter and Sensor Processing Latency)

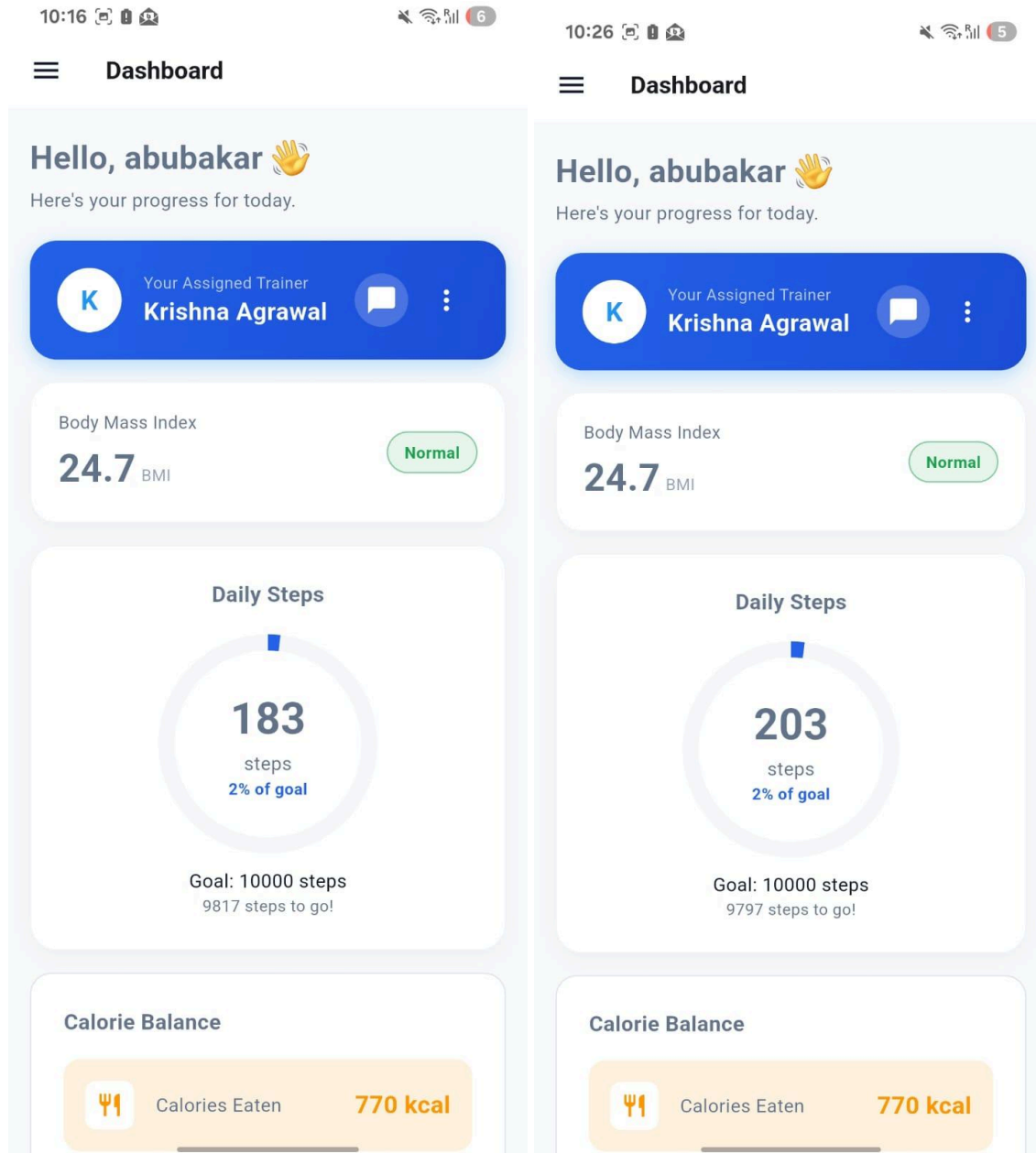
Verified that steps are tracked in the background and that the dashboard updates within a mean response time of 2 seconds.

Test Owner: Abubakar, Ankit and Sahil S.

Test Date: 03/28/2026

Test Results: Walked 20 steps while the phone was locked; the background sensor accurately counted the movement. Upon unlocking the phone and opening the app, the dashboard progress ring animated and updated in approximately 1.5 seconds.

Additional Comments: Background tracking remained highly accurate even when the test device was placed in "Battery Saver" mode.



4. Requirement: F3 (Meal Management)

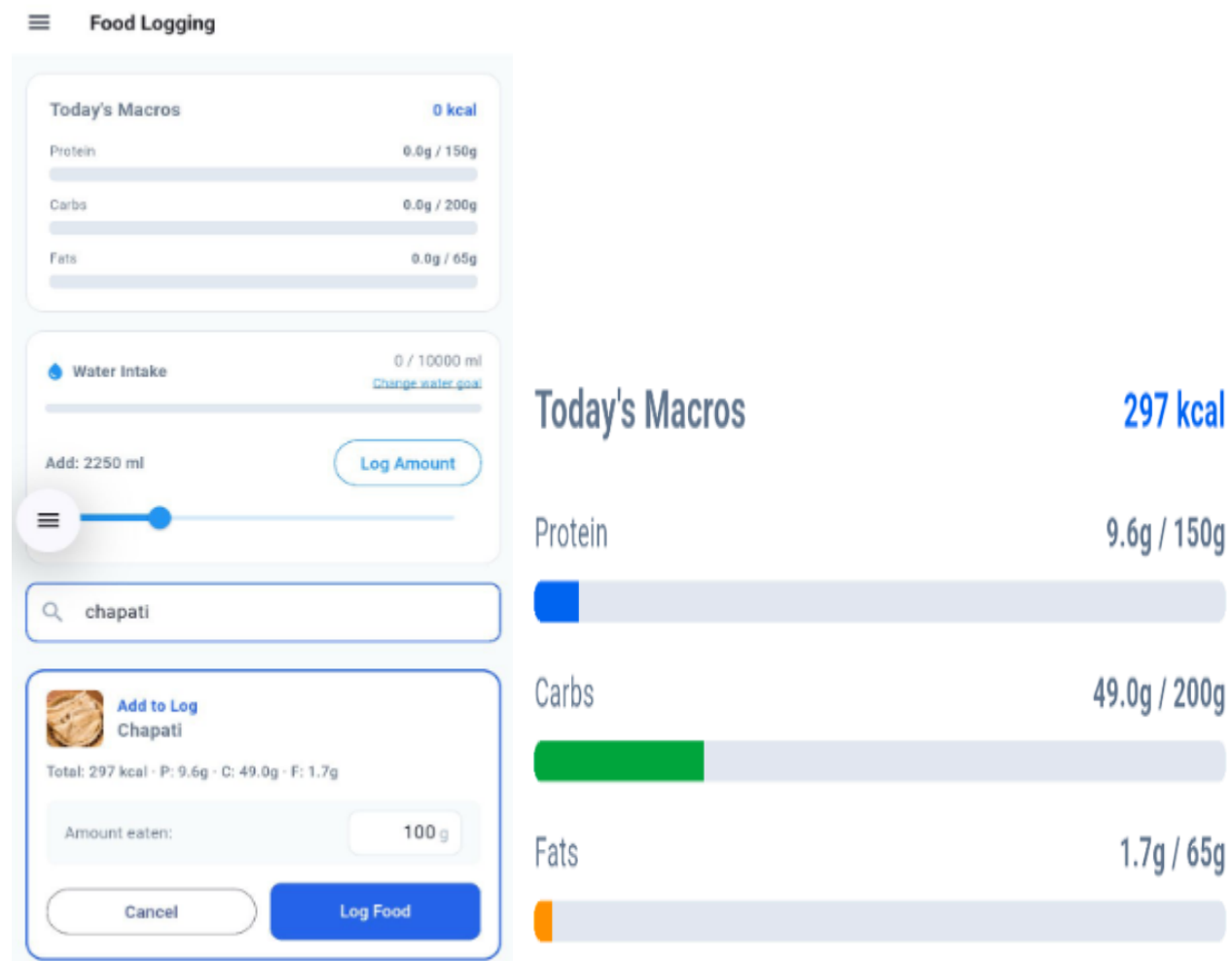
Verified that searching for and adding a food item updates the daily macronutrient pie chart and remaining calorie balance accurately.

Test Owner: Ayush, Aditi and Aman Mangal

Test Date: 03/29/2026

Test Results: Searched for an item using the database and logged it in under 4 taps. The pie chart immediately redrew to include the new Carbs, Fats, and Protein. The "Remaining Calories" number correctly deducted the meal's value instantly.

Additional Comments: The external food API fetches data quickly, and the on-screen math updates flawlessly without requiring a page refresh.



5. Requirement: F6 (Social Connectivity)

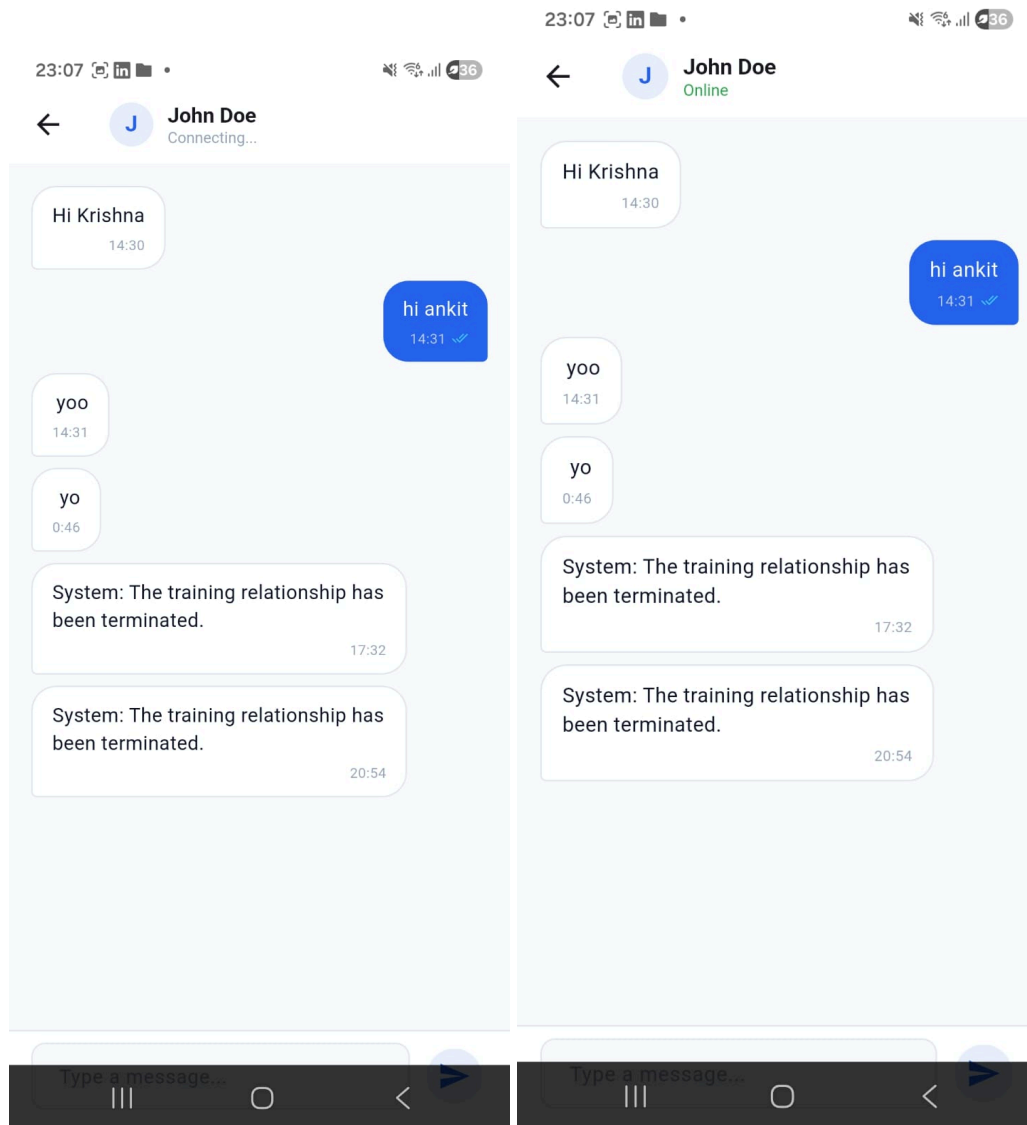
Verified that Trainee and Trainer can send messages to each other, real-time via WebSockets and that the 'Online' status updates correctly

Test Owner: Nidheesh

Test Date: 03/30/2026

Test Results: Messages typed by the Trainee appeared on the Trainer's screen instantly without needing to manually refresh the chat. When a user opened the app, their status correctly toggled from "Connecting..." to "Online" for their connections.

Additional Comments: Test passed successfully



6. Requirement: F6 (Social Connectivity)

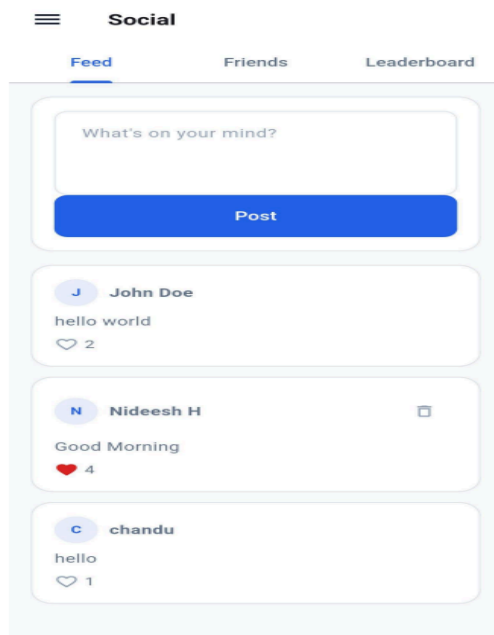
Verified that a post created by a user appears in their friends' feeds only and 'Likes' on the post trigger real-time notifications.

Test Owner: Ankit Kumar Jha

Test Date: 03/30/2026

Test Results: Created a test post. It successfully appeared in the feed of connected friends, but was hidden from strangers. A friend clicking "Like" immediately triggered a notification on the original poster's device.

Additional Comments: Social privacy filters are working exactly as intended to protect user data from public view.



7. Requirement: 4.2.1 (Role-Based Access) and F7 (Trainer connection)

Verified the full lifecycle of a trainer application: a Trainee submits professional credentials, an Admin approves the application and the Trainee's interface successfully transitions to the Trainer Dashboard.

Test Owner: Krishna, Rushali and Nideesh

Test Date: 03/31/2026

Test Results: Trainee submitted the application, which successfully queued in the Admin panel. Admin clicked 'Approve', and the database role updated immediately. Upon the next app load, the user's interface smoothly transitioned, granting them access to Trainer-specific tools like the student list.

Additional Comments: The transition eliminates the need for the user to create a completely new account to become a trainer.

8. Requirement: F6 (Social Connectivity)

Verify that users can report inappropriate content in the Feed or Direct Messages and that administrators can view and resolve these reports in the Admin dashboard

Test Owner: *Ayush, Aditi and Aman M.*

Test Date: 03/31/2026

Test Results: Will be implemented in the future

9. Requirement: F5 (Progress Visualization)

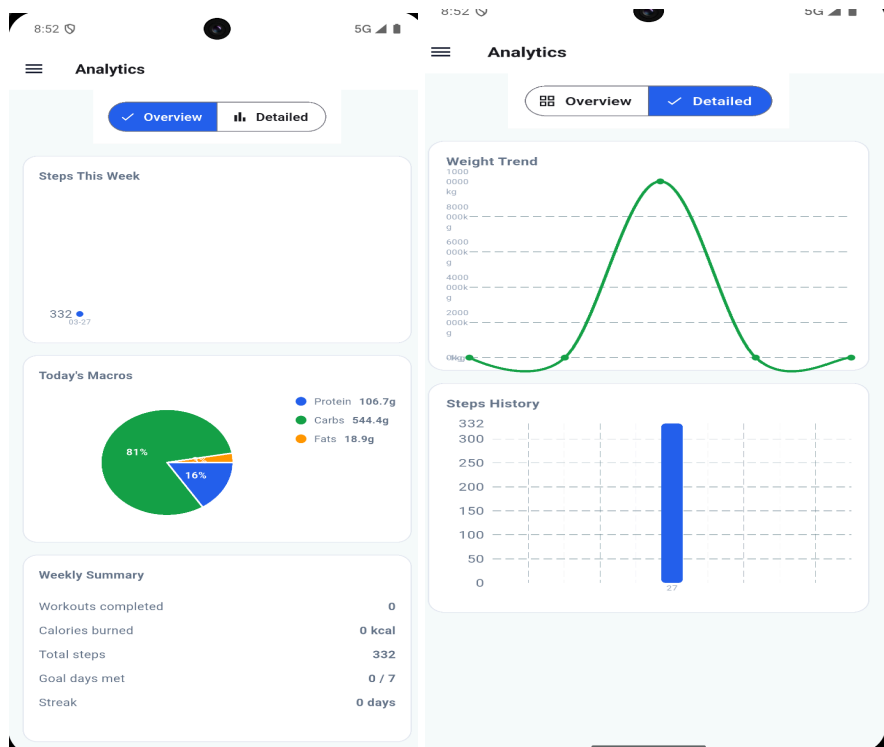
Verify that historical workout and nutrition data correctly populates the Calendar view, and that the weekly summary Graphs render accurate data points.

Test Owner: Ayush, Aditi and Aman

Test Date: 04/01/2026

Test Results: Navigating to the Calendar correctly highlighted days where workout goals were met. The weekly weight/calorie graphs plotted the X and Y axes correctly based on the user's logged history.

Additional Comments: The charting libraries interact perfectly with the backend data arrays, displaying graphs cleanly



10. Requirement: F6 (Social Connectivity)

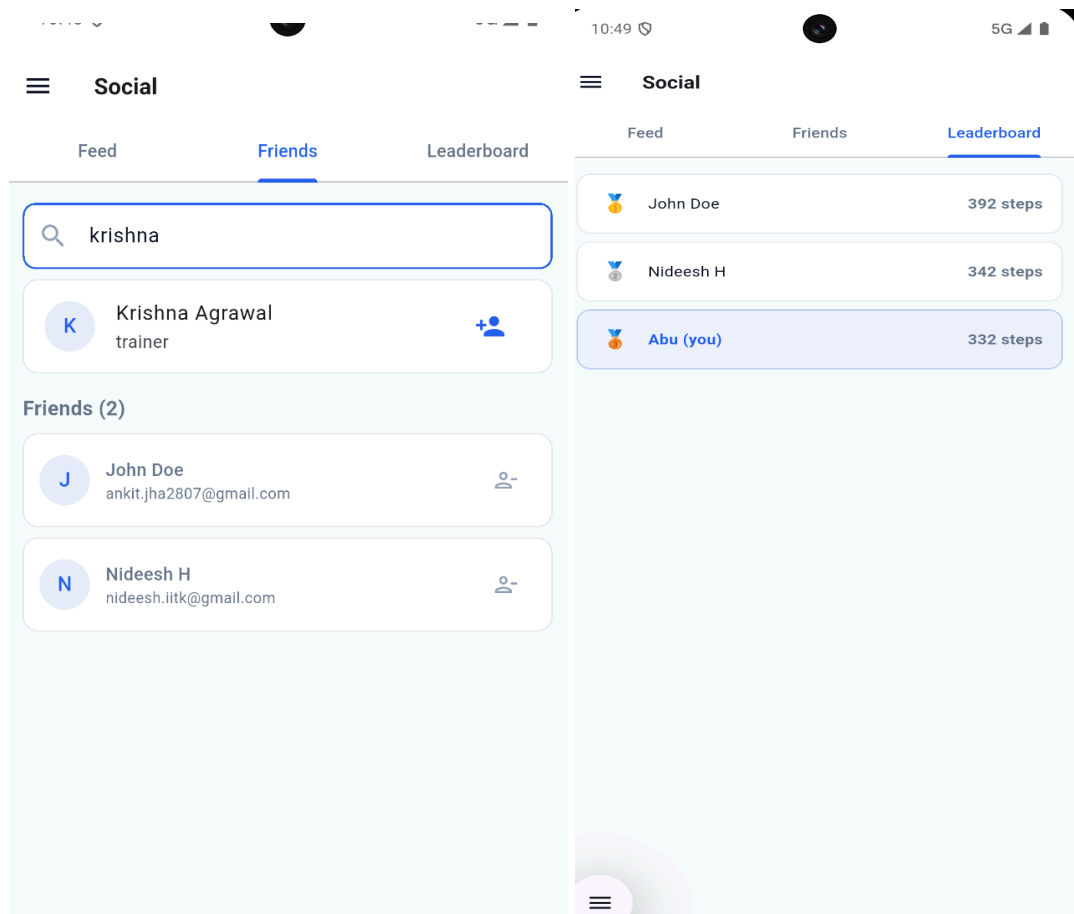
Verify that users can accurately search for other users by username, and that the competitive Leaderboard ranks friends correctly based on weekly step counts.

Test Owner: Ankit Kumar Jha

Test Date: 04/01/2026

Test Results: Searching a partial name (e.g., "Smi") successfully returned relevant user profiles (e.g., "Smith"). The leaderboard successfully sorted a group of friends in descending order based on their step logs for the week.

Additional Comments: Leaderboard generation is efficient and updates without causing performance lag.



11. Requirement: F2 (Step Tracking)

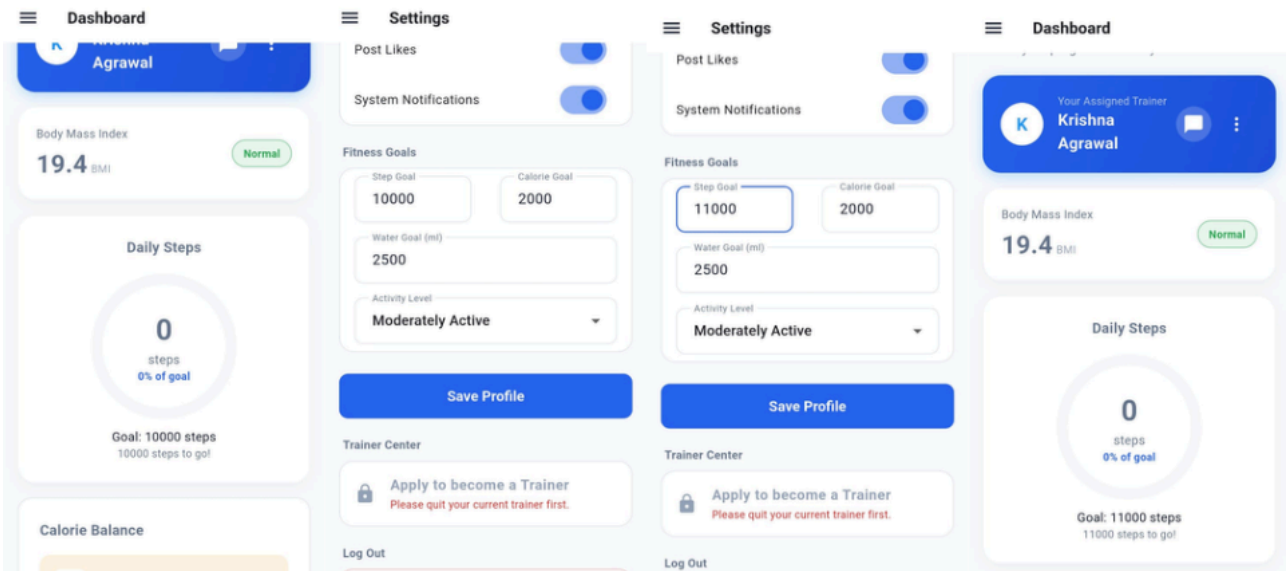
Verify that a user can change their daily fitness goals in the settings menu and that the main dashboard immediately reflects the new targets.

Test Owner: Krishna, Rushali and Nideesh

Test Date: 04/01/2026

Test Results: User navigated to the Settings tab and changed their "Daily Step Goal" from 10,000 to 5,000. Upon returning to the Dashboard, the circular progress ring immediately adjusted its percentage according to the new total.

Additional Comments: Test passed successfully

**12. Requirement: 4.2.1 (User Authentication and Access Control)**

Verify that an Administrator can successfully deactivate a user account via the Admin Dashboard, immediately preventing the deactivated user from authenticating or accessing the system.

Test Owner: Krishna, Rushali and Nideesh

Test Date: 04/02/2026

Test Results: Logged in as Admin and deactivated the account of a test user. When the test user attempted to log in afterward, the user was unable to log in.

Additional Comments: Test passed successfully

13. Requirement: F3 (Meal Management)

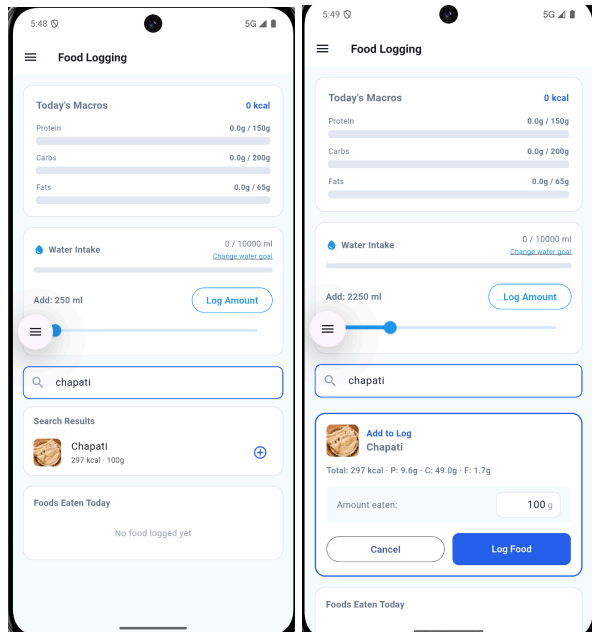
Verify that a user can dynamically adjust the serving size or weight of a food item using the on-screen controls, and that the calorie and macro numbers update in real-time before saving.

Test Owner: Ayush, Aditi and Aman M.

Test Date: 04/02/2026

Test Results: User searched for "Chapati" and selected it. On the logging popup, the user tapped the "+" stepper to increase the amount from 100g to 200g. The displayed Total Calories, Protein, Carbs, and Fats instantly doubled on the screen. The user tapped "Log Food" and the correct doubled amount was added to the daily diary.

Additional Comments: Test passed successfully



14. Requirement: F6 (Social Connectivity)

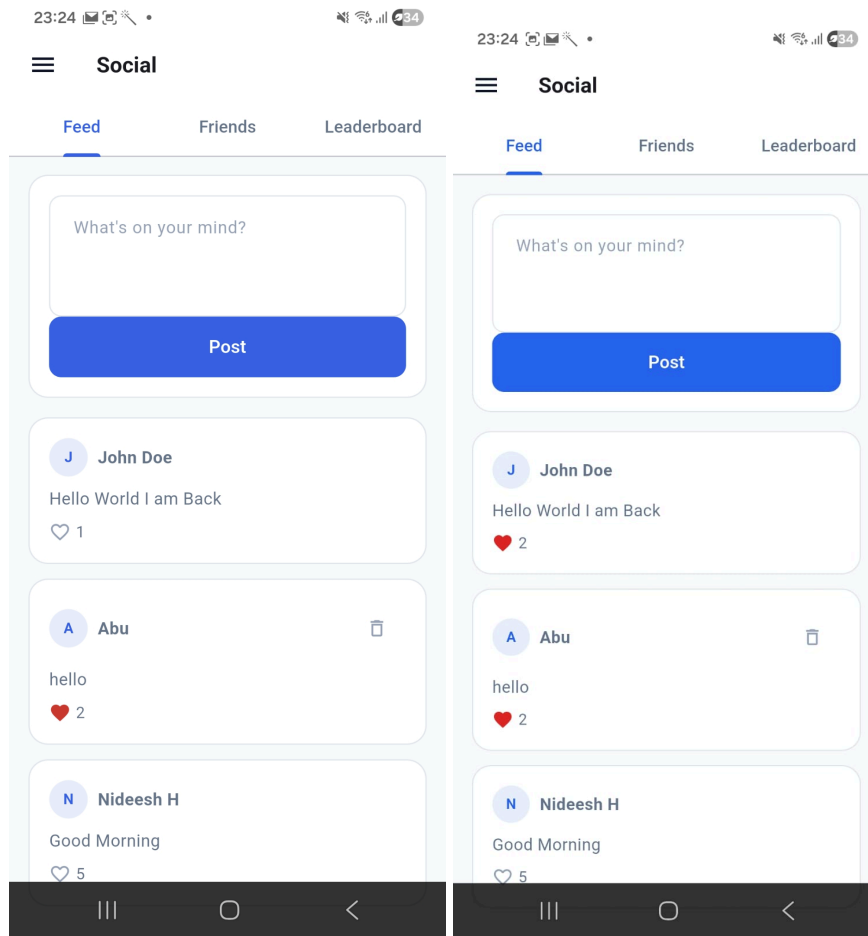
Verify that a user can scroll through the social feed, "like" a friend's post, and visually confirm that their interaction was registered.

Test Owner: Krishna, Rushali and Nideesh

Test Date: 04/02/2026

Test Results: User navigated to the "Feed" tab and scrolled down to view recent posts from friends. User tapped the heart/like icon on a friend's post. The heart icon instantly changed color (e.g., turned red) and the "Like" counter immediately went up by 1.

Additional Comments: The social feed feels highly responsive



15. **Requirement:** F6 (Social Connectivity)

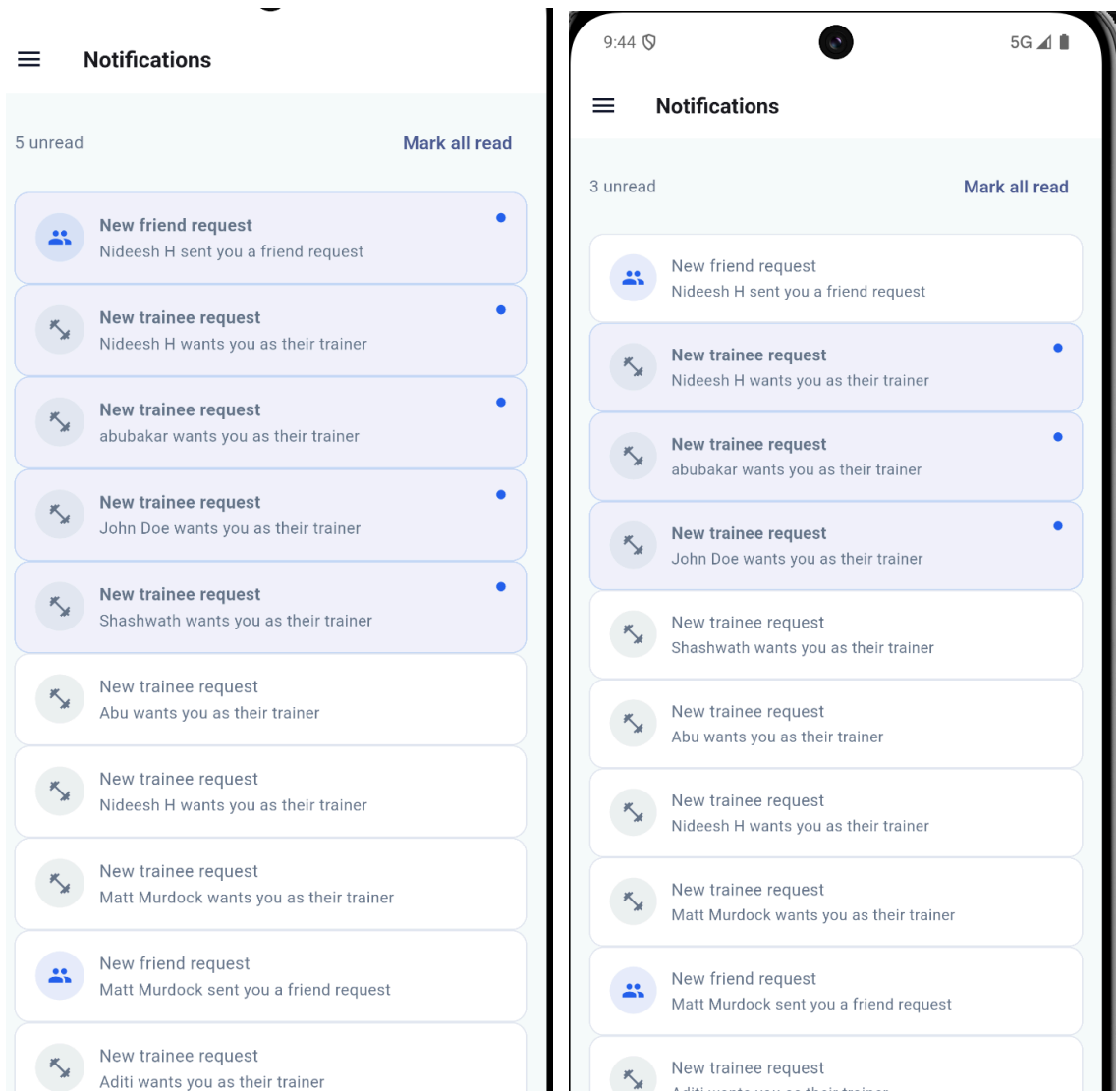
Verify that receiving a new alert triggers a visual badge on the notifications icon, and that reading the alerts clears the badge.

Test Owner: Ayush, Aditi and Aman M.

Test Date: 04/02/2026

Test Results: A test friend request was sent to the user. A red dot (unread badge) successfully appeared on the bell icon in the top right corner of the user's dashboard. The user tapped the bell icon to open the Notifications list, tapped "Mark all read," and the red dot successfully disappeared.

Additional Comments: Visual cues for unread alerts are functioning perfectly to catch the user's attention.



....

8 Conclusion

How Effective and Exhaustive was the Testing?

The testing process executed for TrackMate was highly effective in securing the application's core user journeys, though it was not entirely exhaustive due to the sheer volume of features introduced during the development cycle. With the rapid implementation of complex, interconnected modules, ranging from role-based dashboards and daily analytics to real-time messaging and deep-linked notifications, our strategy had to prioritize *effectiveness* over absolute *exhaustiveness*. We focused our manual testing bandwidth on validating the most critical pathways: accurate macro calculations, database synchronization, and the essential handshakes between Trainees and Trainers. While this targeted approach successfully identified and resolved high-severity blockers (such as application crashes and data desyncs), the rapid expansion of the app's feature set simply outpaced our capacity for exhaustive, edge-to-edge coverage. Ultimately, the testing was highly successful in ensuring a stable, functional product for launch, but we must acknowledge that deeper edge cases within secondary features were not exhaustively mapped out or stressed during this cycle.

Which Components Have Not Been Tested Adequately?

Despite the exhaustive nature of the nutrition and scheduling modules, certain components within the TrackMate ecosystem have not been tested adequately and require further dedicated QA cycles. Primarily, the Exercise and sensor-integration modules have only undergone surface-level simulation rather than rigorous real-world hardware testing. The complexities of background pedometer tracking, OS-level battery optimization interruptions on Android, and the precise timing mechanisms of the active Cardio Timer have not been fully stressed under variable, real-world physical conditions. Additionally, the Admin User interaction flows have been critically under-tested. While we verified the highest-level dashboard rendering for administrative accounts, we have not adequately executed test cases surrounding user suspension protocols, global system health overrides, database management, or the resolution of flagged content within the Social feed. This lack of deep integration testing on the Admin panel leaves a blind spot regarding system-wide moderation and administrative control mechanisms.

What Difficulties Have You Faced During Testing?

A primary difficulty encountered during the testing lifecycle was the logistical complexity of simulating and coordinating multiple users simultaneously to validate real-time features. Because TrackMate relies heavily on cross-role interactions, such as a Trainee sending a coaching request, the Trainer accepting it, and the underlying messaging and social graphs instantly updating; coordinating these asynchronous events across different test environments proved highly challenging. Verifying WebSocket stability and push

notification deep-linking required juggling multiple authenticated sessions and physical devices concurrently to ensure the backend bridges functioned correctly. Furthermore, the development lifecycle was highly dynamic, with a significant number of features and UI refinements being introduced actively during the implementation phase. This continuous integration of new functionalities forced the QA process to constantly adapt, requiring us to continuously rewrite test cases, expand our structural coverage, and ensure our testing remained exhaustive despite the rapidly expanding scope of the application. Keeping the test documentation and execution synchronized with a moving target of new features was a persistent administrative and technical hurdle.

How Could the Testing Process Be Improved?

To significantly improve the testing process moving forward, the most critical enhancement would be the introduction of a robust Automated Testing framework. Currently, the heavy reliance on manual functional testing creates a bottleneck, especially when rigorous regression testing is required after new features are implemented. Integrating end-to-end automation tools (such as Appium for mobile UI) would allow us to rapidly execute core user flows, like food logging or role authentication, automatically on every new code build. Additionally, to resolve the multi-user coordination challenges, the engineering team should develop a dedicated suite of mock-data generation scripts. These scripts could programmatically simulate dozens of Trainee requests, chat messages, or social feed updates simultaneously, allowing QA to stress-test WebSocket concurrency and database locking without needing manual device juggling. Finally, for the hardware-dependent features like the pedometer and workout timers, utilizing a cloud-based physical device farm would allow us to test background sensor behavior and OS-level limitations across a much wider variety of Android devices, ensuring true hardware compatibility rather than relying purely on software simulations.

Appendix A - Group Log

Date	Duration	Minutes
27th March	1 Hour	Discussion About How To go about unit testing and how to progress to integration and system testing
28th March	4 Hour	Identified Tests in depth for Unit, System and Integration Testing
1st April	2 Hour	Started The First Draft Of Testing Document and User Manual Document Identifying all working use cases, test cases and bugs/boundary conditions to check
2nd April	6 Hour	Completed and Reviewed the Final Draft of the User Manual and went through all the important cases of Unit Testing, Checking and testing their results and finalized all relevant test cases for System and Integration Testing
3rd April	8 Hour	Completed Testing Procedure Checking the results of all Unit System and Integration Testing test cases and made the final draft of the Testing Document and the User Manual